

SELÇUK ÜNİVERSİTESİ
MÜHENDİSLİK-MİMARLIK FAKÜLTESİ
ELEKTRİK-ELEKTRONİK MÜHENDİSLİĞİ BÖLÜMÜ

LOJİK DEVRE TASARIM
DERS NOTLARI

Konya- 2012

KONULAR

1. Ardışıl lojik devreler, senkron ardışıl lojik devreler ve analizi
2. Durum sayısının azaltılması, durum diyagramı, durum tablosu ve uygulama tablolarının elde edilmesi
3. Ardışıl devrelerin sentezi, değişik tipte senkron sayıcı tasarımları, dizi yakalayıcılar
4. Registerlar, kaydırıcı registerlar, paralel yüklemeli registerlar, iç register transfer, bus transfer, registerlar arası veri transferi
5. Hafıza elemanları ile senkron ardışıl devre tasarımı, zamanlama diyagramları
6. ALU, genel özellikleri, kaydırıcı tasarımı, durum registeri
7. Akümülatör register tasarımı
8. Genel işlemci tasarımı, kontrol birimlerinin yapıları ve genel özellikleri
9. Algoritmik durum makinaları (ASM), ASM şeması, durum kutusu, karar kutusu, koşul kutusu, ASM bloğu
10. Kontrol biriminde her bir durum için bir flip-flop kullanılması, decoder kullanılması ile oluşan tasarım, uygulamalı örnekler
11. Kontrol biriminde; decoder, multiplexer ve PLA kullanarak oluşan tasarım, uygulamalı örnekler
12. Lojik kapı devrelerinde fan-out hesabı

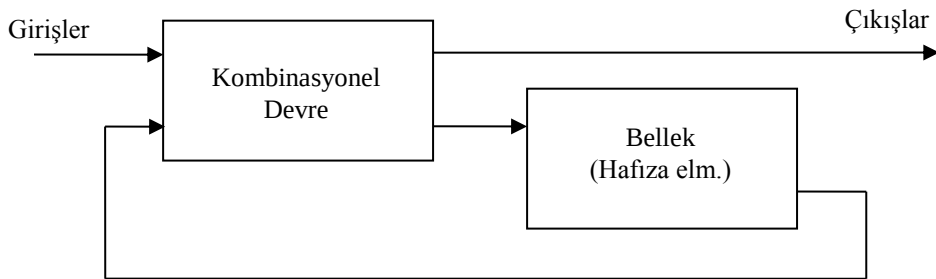
KAYNAKLAR

1. Sayısal Tasarım, Morris MANO, MEB Yayınları. (Dijital Tasarım)
2. Lojik Devreler, Prof. Dr. Emin ÜNALAN, İTÜ Yayınları.
3. Lojik Devreler, Prof. Dr. Emre HARMANCI, İTÜ Yayınları.
4. Digital Design Principles&Practices, John F. Wakerly, Prentice Hall.
5. The Design of Digital Systems, J.B. Reatman, McGraw-Hill.
6. Principles of Digital Design, Daniel D. Gajski, Prentice Hall.

1. ARDIŞIL DEVRELER

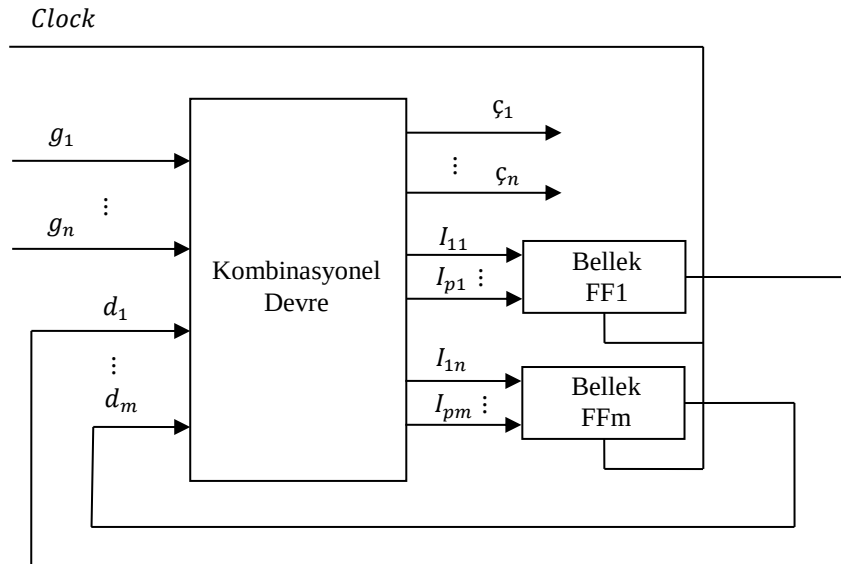
Şimdiye kadar ele alınan sayısal devrelerin çıkışları tamamen o anki devre girişlerine bağlı olan kombinasyonel devrelerdi. Örneğin VE kapısı bir kombinasyonel devredir. Bazı lojik devrelerde girişteki bilgi değişse bile saat darbesi gelmedikçe çıkış konum değiştirmez ise bu devrelere “*ardışıl devre (sequential logic)*” denir. 7 rakamlı bir telefon numarasında 6 rakam çevirdiğimizde telefon çalmaz, 7. rakam çevrilince telefon çalar, burada 7. rakam clock darbesidir.

Kombinasyonel devre + Bellek = Ardışıl devre



Şekil 1.1 Ardışıl devre blok şeması

Ardışıl devrenin durumu, bellekteki durumun değişmesi ile değişir. Yani kombinasyonel devre, girişleri belirler, belleğin durumu ise çıkışları belirler.



Şekil 1.2 Genel ardışıl devre şeması

$g_1, \dots, g_n$: n tane giriş , bağımsız büyüklük

$z_1, \dots, z_n$: n tane çıkış , bağımlı büyüklük, bellek çıkışlarına bağlıdır

$d_1, \dots, d_m$: Durum büyüklüğü, Bellek (FF) çıkışı

$I_{11}, \dots, I_{pm}$: FF uyarma girişleri

İki çeşit ardışıl devre vardır ve bunların sınıflandırılması devrelerin sinyal zamanlamasına bağlıdır. Başka bir şekilde ifade edilecek olursa ardışıl devreler girişlerinde olan değişikliğin çıkışlarına yansıtılma zamanına göre sınıflandırılırlar. Birinde girişlerinde olan değişiklik beklemeksizin yalnızca kullanılan kapı vs. gibi birimlerin gecikme süresi kadar sonra çıkışa yansıtılır. Burada herhangi bir senkronlama işareti yoktur. Dolayısıyla bu çalışma şekli “*asenkron*” olarak adlandırılır. Diğerinde ise girişlerinde olan değişiklik çıkışlara hemen yansıtılmaz; değişikliğin yansıtılması için bir senkronlama işareti kullanılır. Bu işaret aktif olduğunda girişlerinde olan değişiklik çıkışlara yansıtılır, burada da tabi ki devrede kullanılan kapı vs. eleman gecikmeleri yine söz konusudur. Bu şekilde çalışma şekliyse “*senkron*” olarak adlandırılır.

Flip-Flop Uyarma Tabloları

$Q(t)$	$Q(t+1)$	S	R	J	K	D	T
0	0	0	X	0	X	0	0
0	1	1	0	1	X	1	1
1	0	0	1	X	1	0	1
1	1	X	0	X	0	1	0

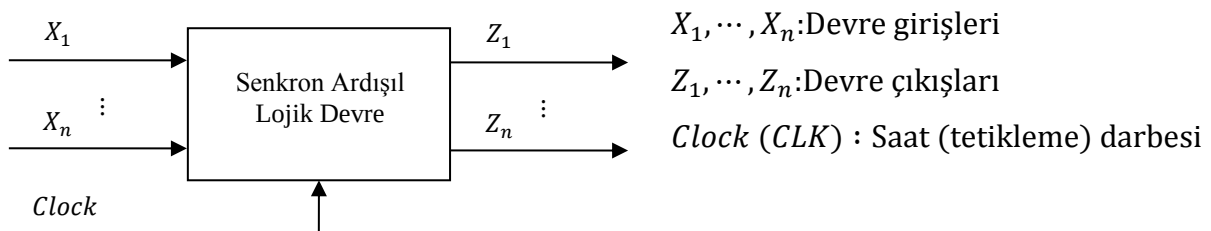
1.1. Senkron Ardışıl Lojik Devreler

Ardışıl devreler senkron ve asenkron olarak ikiye ayrılmaktadır. Yaygın olarak çok kullanıldığı için senkron ardışıl devreler (zamanlanmış sıralı devreler, senkron ardışıl lojik) anlatılacaktır. Ardışıl senkron devreler iki aşamada incelenecektir:

- Senkron ardışıl devre analizi
- Senkron ardışıl devre sentezi

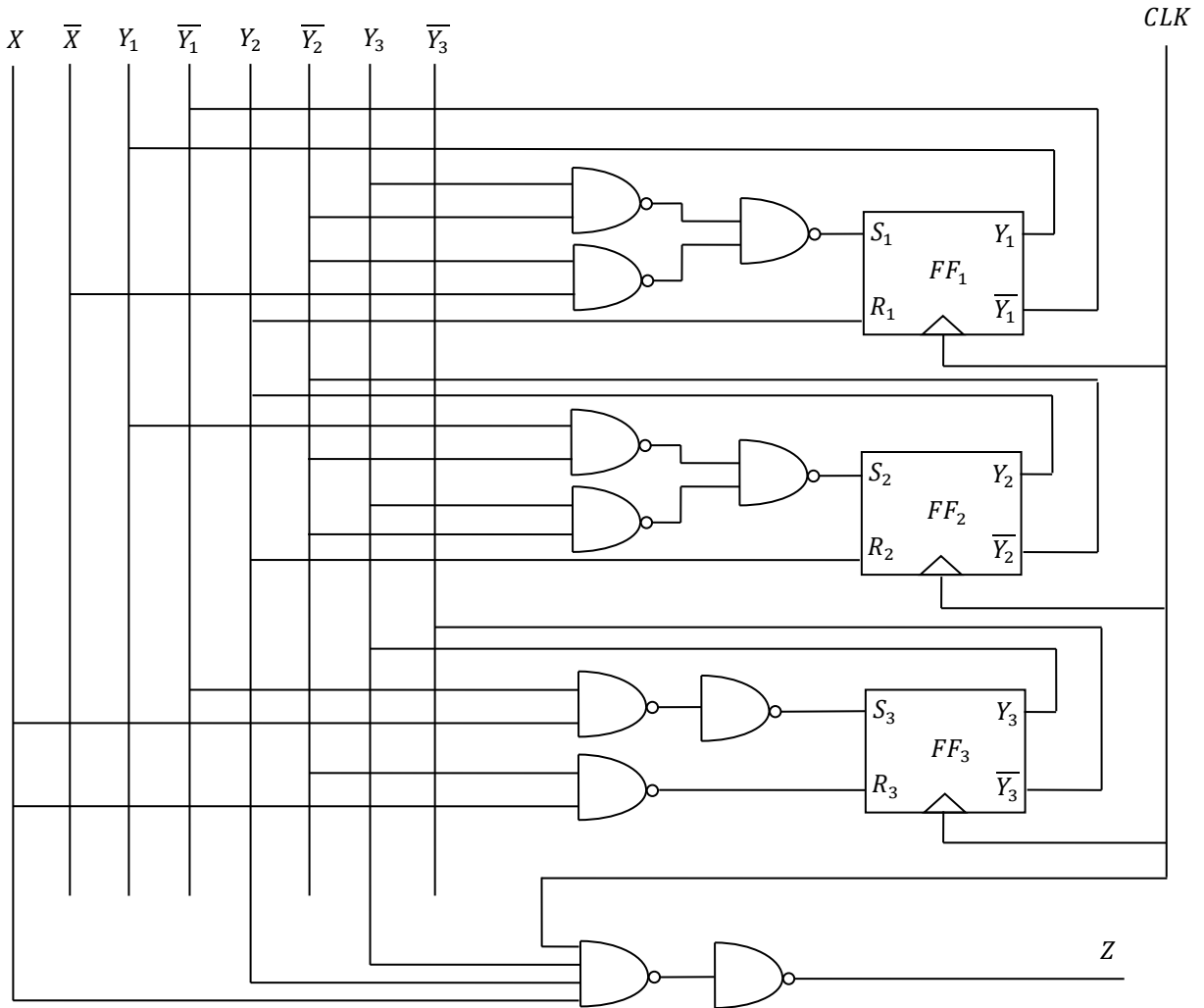
1.1.1. Senkron ardışıl devre analizi

Senkron ardışıl devrelerde çıkışlar ve bir sonraki durum, girişlerin ve mevcut durumların bir fonksiyonudur. Böyle bir devrenin blok şeması aşağıda gösterilmiştir:



Şekil 1.3. Senkron ardışıl devre şeması

Örnek 1.1. Şekil 1.4’ de verilen devrenin analizini gerçekleştiriniz.



Şekil 1.4. Senkron ardışıl devre analizi- Örnek devre1
(Devrede kullanılan tüm kapılar NAND’ dır)

Devrenin bir girişi ve bir çıkışı vardır: X girişi ve Z çıkışı yanında dışarıdan CLK senkronlama işareti verilmektedir. Şekil 1.4’ deki devreden Z çıkış için;

$$Z = X \cdot Y_2 \cdot Y_3 \cdot CLK$$

fonksiyonu bulunur. Clock darbesi geldiğinde $X \cdot Y_2 \cdot Y_3 = 111$ ise $Z = 1$ ’ dir. Diğer yandan FF uyarma fonksiyonları şu şekilde yazılabilir:

$$S_1 = (\overline{Y_2 Y_3} \cdot \overline{X Y_2}) \Rightarrow S_1 = \overline{Y_2} Y_3 + \overline{X} \overline{Y_2}, R_1 = Y_2$$

$$S_2 = Y_1 \overline{Y_2} + \overline{Y_2} Y_3, R_2 = Y_2$$

$$S_3 = \overline{Y_1} X, R_3 = Y_2 + \overline{X}$$

CLK darbelerine göre FF' ler değişik durumda olabilirler. $Y_1Y_2Y_3 = 000$ durumunu FF' lar başlangıç durumu olarak ele alalım ve (1) numaralı durum olarak gösterelim.

Bağımsız X değişkeninin 0 ve 1 olması halinde her CLK darbesinde $Y_1Y_2Y_3$ bağımlı değişkenlerinin Z çıkışının hangi değerler alacağı aşağıda gösterilmiştir.

$X = 1$ değeri için:

$$S_1 = \overline{Y_2}Y_3 + \overline{X}\overline{Y_2} = 0.1 + 0.1 = 0, R_1 = Y_2 = 0$$

$S_1R_1 = 00$ olduğu için FF_1 ' in çıkışı durum değiştirmez ve CLK' dan sonra yine $Y_1 = 0$ olur.

$$S_2 = Y_1\overline{Y_2} + \overline{Y_2}Y_3 = 0, R_2 = Y_2 = 0$$

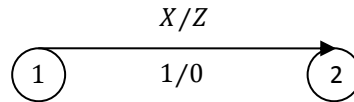
$S_2R_2 = 00$ olduğu için FF_2 ' nin çıkışı durum değiştirmez ve CLK' dan sonra yine $Y_2 = 0$ olur.

$$S_3 = \overline{Y_1}X = 1.1 = 1, R_3 = Y_2 + \overline{X} = 0 + 0 = 0$$

$S_3R_3 = 10$ olduğu için FF_3 ' ün çıkışı durum değiştirir ve CLK' dan sonra yine $Y_3 = 1$ olur.

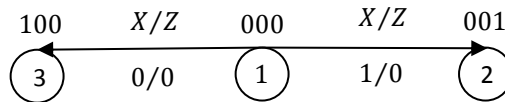
$$Z = X \cdot Y_2 \cdot Y_3 \cdot CLK = 1.0.0 = 0$$

O halde gelen CLK darbesi sonunda FF çıkışları olan bağımlı değişkenler $Y_1Y_2Y_3 = 000$ durumundan $Y_1Y_2Y_3 = 001$ durumuna geleceklerdir. Bu yeni durumu (2) ile gösterirsek devrenin durum diyagramının bir parçası Şekil 1.5' de gösterildiği gibi elde edilebilir.



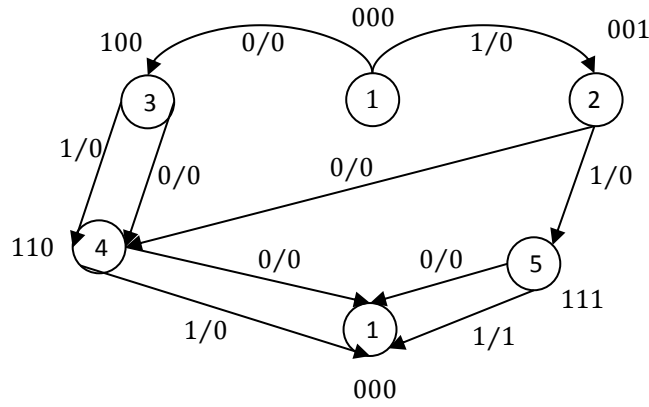
Şekil 1.5. Durum diyagramının parçası-1

Bu şekle göre, t anında (1) durumunda olan ardışıl devreye $X = 1$ girişi uygulanırsa Z çıkışında 0 oluşur ve t^+ anında devre (2) durumuna geçer. Aynı şartlarda $X=0$ olduğu kabul edilirse CLK darbesinden sonra FF çıkışlarının $Y_1Y_2Y_3 = 100$ olacağı ve devre çıkışı $Z = 0$ kalacağı görülür. Bu yeni durumu (3) ile gösterirsek devrenin durum diyagramının bir parçası Şekil 1.6' daki gibi olur.



Şekil 1.6. Durum diyagramının parçası-2

Aynı şekilde işleme devam edilirse devreye ait durum diyagramının tamamı Şekil 1.7' deki gibi olacaktır.



Şekil 1.7. Durum diyagramının parçası-3

Durum diyagramından $Y_1Y_2Y_3 = 000$ başlangıç durumunda iken arka arkaya gelen 3 CLK darbesinde $X = 1$ ise Z çıkışı 3. CLK darbesi sonunda 1 olur, X' in diğer durumlarında $Z = 0$ olur.

Belirsiz durumların önüne geçmek için FF' ların çıkışlarını başlangıçta sıfırlamak gerekir. Şekil 1.7' deki durum diyagramından Çizelge 1.1' deki durum tablosu elde edilir.

Çizelge 1.1. Durum Tablosu

Şimdiki Durum	Gelecek Durum		Çıkış (Z)	
	X=0	X=1	X=0	X=1
1	3	2	0	0
2	4	5	0	0
3	4	4	0	0
4	1	1	0	0
5	1	1	0	1

Durum tablosu, durum sayısı minimum olan bir tablodur. Yani tekrarlanmış durumlar yoktur. Durum sayısı minimum olan Çizelge 1.1' den uygulama tablosu elde edilir. Uygulama tablosu, durum geçiş tablosu, çıkış tablosu ve FF uyarma tablolarından oluşmaktadır. Uygulama tablosuna ait sözü edilen tablolar ayrı ayrı yazılacağı gibi Çizelge 1.2' deki gibi de gösterilebilir.

Çizelge 1.2. Uygulama Tablosu

Durum	Ş.D.			Giriş	G.D.			Durum	FF Girişleri						Çıkış
	Y_1	Y_2	Y_3		Y_1	Y_2	Y_3		S_1	R_1	S_2	R_2	S_3	R_3	
1	0	0	0	0	1	0	0	3	1	0	0	X	0	X	0
1	0	0	0	1	0	0	1	2	0	X	0	X	1	0	0
2	0	0	1	0	1	1	0	4	1	0	1	0	0	1	0
2	0	0	1	1	1	1	1	5	1	0	1	0	X	0	0
3	1	0	0	0	1	1	0	4	X	0	1	0	0	X	0
3	1	0	0	1	1	1	0	4	X	0	1	0	0	X	0
4	1	1	0	0	0	0	0	1	0	1	0	1	0	X	0
4	1	1	0	1	0	0	0	1	0	1	0	1	0	X	0
5	1	1	1	0	0	0	0	1	0	1	0	1	0	1	0
5	1	1	1	1	0	0	0	1	0	1	0	1	0	1	1

Uygulama tablosunda durumlar $X=0$ ve $X=1$ için birer kere yazılır. FF uyarma giriş değerleri SR FF' a ait geçiş tablosundan faydalanılarak elde edilir. Örneğin 1 durumunda $X=0$ için 3 durumuna geçilmiştir ve Y_1 çıkışı 0' dan 1' e geçmiştir. Bu durumda $S_1R_1 = 10$ olmalıdır.

Uygulama tablosundan yararlanılarak bağımlı ve bağımsız değişkenlere göre FF uyarma girişleri ve Z çıkışı için Karnaugh diyagramı çıkarılır. Uygulama tablosunda gösterilmeyen durumlar yani kullanılmayan durumlar Karnaugh diyagramında keyfi olarak gösterilir.

$\begin{matrix} XY_1 \\ Y_2Y_3 \end{matrix}$	00	01	11	10
00	1	X	X	0
01	1			1
11		0	0	
10		0	0	

$$S_1 = \bar{X}\bar{Y}_2 + \bar{Y}_2Y_3$$

$\begin{matrix} XY_1 \\ Y_2Y_3 \end{matrix}$	00	01	11	10
00	0	0	0	X
01	0			0
11		1	1	
10		1	1	

$$R_1 = Y_2$$

$\begin{matrix} XY_1 \\ Y_2Y_3 \end{matrix}$	00	01	11	10
00	0	0	0	0
01	0			0
11		0	1	
10		0	0	

$$Z = Y_2Y_3X$$

Benzer şekilde işleme devam edilirse aşağıdaki sonuçlar bulunur:

$$S_2 = Y_1\bar{Y}_2 + \bar{Y}_2Y_3, \quad R_2 = Y_2, \quad S_3 = X\bar{Y}_1, \quad R_3 = Y_2 + \bar{X}$$

Durum tablosunda kullanılmayan kombinasyonların herhangi bir kargaşaya neden olmaması için, bu kombinasyonların bir CLK darbesi sonra belirli bir konuma döndürülmeleri gerekir. Bu konum $Y_1Y_2Y_3 = 000$ yani başlangıç konumu olsun. Enerji verilince FF çıkışlarında kullanılmayan komutlardan $Y_1Y_2Y_3 = 010$ oluşursa, bu durumun birinci CLK darbesi sonunda $Y_1Y_2Y_3 = 000$ konumuna dönmesi gerekir. Bu durumda yalnız Y_2 çıkışı konum değiştirir, Y_1 ve Y_3 ise konum değiştirmez. O halde $S_1R_1 = 0X, S_2R_2 = 01, S_3R_3 = 0X$ olur. Kullanılmayan kombinasyonlardan $Y_1Y_2Y_3 = 101$ kombinasyonunun birinci CLK darbesi sonunda $Y_1Y_2Y_3 = 000$ konumuna gelmesi için $S_1R_1 = 01, S_2R_2 = 0X, S_3R_3 = 01$ olmalıdır.

Bu değerler için kullanılan uyarma tabloları ve bu tablolardan elde edilen uyarma fonksiyonları Çizelge 1.3' de verilmiştir. Önceki tablolarda kullanılmayan durumların $Y_1Y_2Y_3 = 000$ konumuna dönmeleri $X = 0$ ve $X = 1$ için olur. Önceki tablolara bu durumlar ilave edilir.

Çizelge 1.3. Ek uygulama tablosu

Şimdiki Durum			Giriş	Gelecek Durum		
Y_1	Y_2	Y_3	X	Y_1	Y_2	Y_3
1	0	1	0	0	0	0
1	0	1	1	0	0	0
0	1	1	0	0	0	0
0	1	1	1	0	0	0
0	1	0	0	0	0	0
0	1	0	1	0	0	0

Yeni oluşan son duruma göre Karnaugh diyagramları yeniden hazırlanır. FF uyarma girişleri ve Z çıkışı için aşağıdaki ifadeler elde edilir.

XY_1 Y_2Y_3	00	01	11	10
00	1	X	X	0
01	1	0	0	1
11	0	0	0	0
10	0	0	0	0

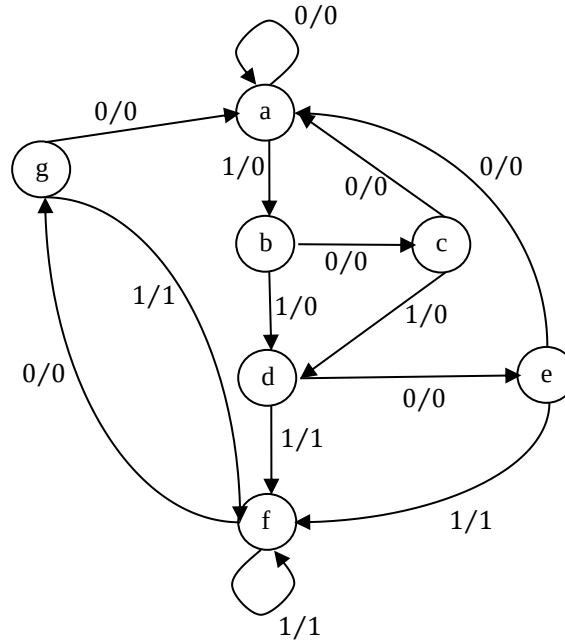
$$S_1 = \bar{X}\bar{Y}_1\bar{Y}_2 + \bar{Y}_1\bar{Y}_2Y_3, \quad R_1 = Y_2 + Y_1Y_3$$

$$S_2 = Y_1\bar{Y}_2\bar{Y}_3 + \bar{Y}_1\bar{Y}_2Y_3, \quad R_2 = Y_2$$

$$S_3 = X\bar{Y}_1\bar{Y}_2, \quad R_3 = Y_1 + Y_2 + \bar{X}$$

1.1.2. Durum Sayısının Azaltılması

Ardışıl devrede kullanılan lojik kapı ve FF sayıları azaltılabilir. m - adet FF' dan 2^m adet durum oluşur. FF sayısının azaltılması hem durumun azaltılması hem de maliyet yönünden gereklidir. Durumlar azaltılırken, giriş ve çıkıştaki isteklerin önceki durumdaki gibi yerine getirilmesi gerekir. Şekil 1.8' deki durum diyagramı ile bu konu daha detaylı açıklanacaktır. Diyagrama ait devrenin bir girişi ve bir çıkışı vardır. Durumlar a, b, c, d, e, f, g dir.



Şekil 1.8. Durum diyagramı

Diyagrama bakarak durumların azaltılıp azaltılamayacağı konusunda bir fikir yürütülemez. Bu yüzden durum diyagramından hareket edilerek durum tablosu oluşturulur. Çizelge 1.4' de elde edilen durum tablosu görülmektedir.

Çizelge 1.4. Durum tablosu

Şimdiki Durum	Gelecek Durum		Çıkış (Z)	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1

Durumların azaltılabilmesi için, şimdiki duruma ait sonraki durumun ve çıkışın eşit olması gerekir. Bu özelliği sağlayan durumlar “e” ve “g” dir. O halde “g” durumu silinip

bunun yerine “e” yazılabilir. “g” yerine “e” yazılırsa “f” durumuna ait gelecek durum $x = 0$ için “e”, $x = 1$ için “f” olur. Bu ise “d” durumuna ait gelecek durumu ile aynıdır, çıkışlarda aynı olduğu için “f” yerine “d” yazılabilir. Böylece Çizelge 1.4’ deki durum tablosundan “f” ve “g” silinerek bunların yerlerine “d” ve “e” yazılabilir.

Durum sayısının azaltılmasını gösteren tablo Çizelge 1.5’ de verilmiştir.

Çizelge 1.5. Durum tablosunun azaltılması

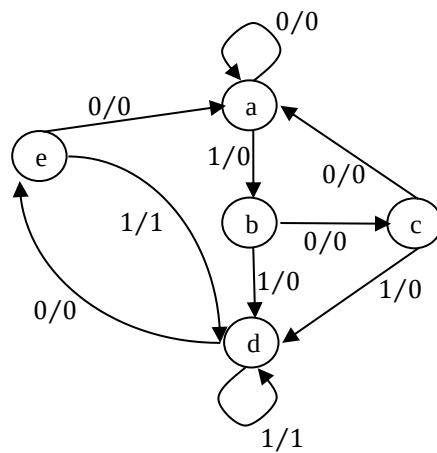
Şimdiki Durum	Gelecek Durum		Çıkış (Z)	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1
g	a	f	0	1

İndirgenmiş durum tablosu Çizelge 1.6’ da görülmektedir.

Çizelge 1.6. İndirgenmiş durum tablosu

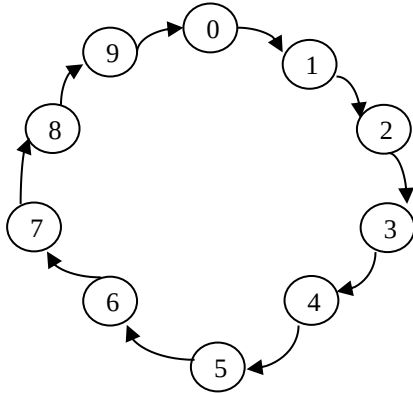
Şimdiki Durum	Gelecek Durum		Çıkış (Z)	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1

İndirgenmiş durum tablosuna uygun yeni akış diyagramı Şekil 1.9’ da görülmektedir.



Şekil 1.8. İndirgenmiş durum diyagramı

Örnek 1.1. T flip-flop kullanarak 4 bitlik bir sayıcı tasarlayınız. Sayıcı başlangıçta 0 gösterecek ve 9 sayısını saydıktan sonra tekrar 0' a dönecektir.



$$\left. \begin{array}{l} 0 \Rightarrow 0000 \\ \vdots \\ 9 \Rightarrow 1001 \end{array} \right\} 4 \text{ FF kullanılacak}$$

Başlangıçta bütün FF' lar resetlenerek, FF çıkışları 0 yapılır. Buna göre uygulama tablosu aşağıda verilmiştir. Sayıcının herhangi bir girişi ve çıkışı (X ve Z) gibi yoktur.

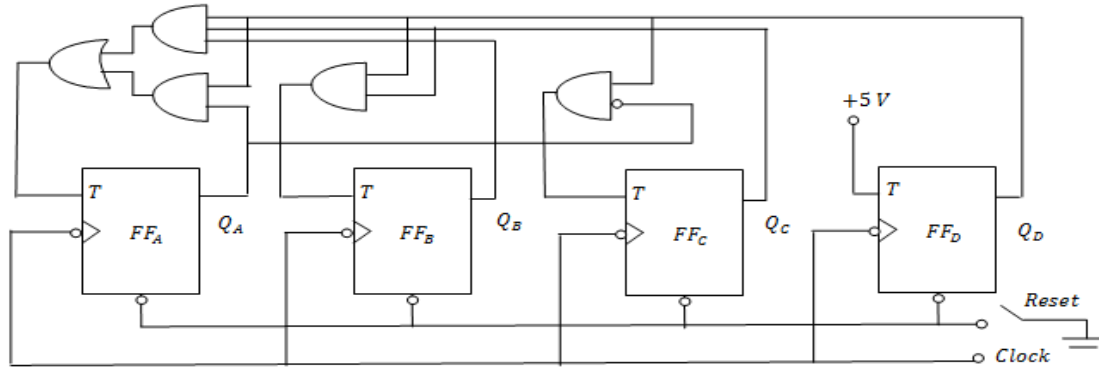
Şimdiki Durum	Sonraki Durum	FF Uyarma Girişleri
$A B C D$	$A B C D$	$T_A T_B T_C T_D$
0 0 0 0	0 0 0 1	0 0 0 1
0 0 0 1	0 0 1 0	0 0 1 1
0 0 1 0	0 0 1 1	0 0 0 1
0 0 1 1	0 1 0 0	0 1 1 1
0 1 0 0	0 1 0 1	0 0 0 1
0 1 0 1	0 1 1 0	0 0 1 1
0 1 1 0	0 1 1 1	0 0 0 1
0 1 1 1	1 0 0 0	1 1 1 1
1 0 0 0	1 0 0 1	0 0 0 1
1 0 0 1	0 0 0 0	1 0 0 1

Kullanılmayan durumlar: 1010, 1011, 1100, 1101, 1110, 1111 $\Rightarrow$ Keyfi değer

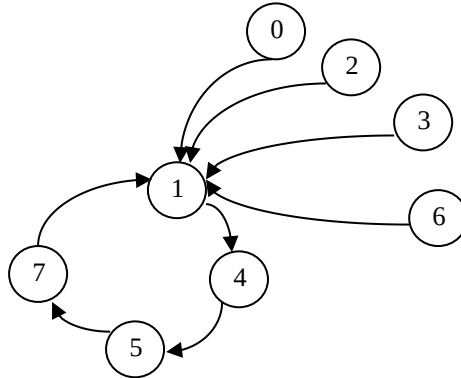
$CD \backslash AB$	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	X	X	X	X
10	0	1	X	X

$$T_A = BCD + AD, T_B = CD$$

$$T_C = \bar{A}D, T_D = 1$$



Örnek 1.2. 1, 4, 5, 7 sayan ve tekrar 1' e dönen bir sayıcı tasarlayınız. Kullanılmayan durumlarda sayıcı 1' e ayarlanacak ve JK FF kullanılacaktır (Başlangıçta FF çıkışlarının 0 konumunda olduğu kabul edilecek).



Durum diyagramından görüleceği gibi durumlar 3 bit kullanılarak ifade edilebilir.

Şimdiki Durum	Sonraki Durum	FF Uyarma Girişleri
A B C	A B C	$J_A K_A$ $J_B K_B$ $J_C K_C$
0 0 0	0 0 1	0 X 0 X 1 X
0 0 1	1 0 0	1 X 0 X X 1
0 1 0	0 0 1	0 X X 1 1 X
0 1 1	0 0 1	0 X X 1 X 0
1 0 0	1 0 1	X 0 0 X 1 X
1 0 1	1 1 1	X 0 1 X X 0
1 1 0	0 0 1	X 1 X 1 1 X
1 1 1	0 0 1	X 1 X 1 X 0

	BC	00	01	11	10
A	0		1		
	1	X	X	X	X

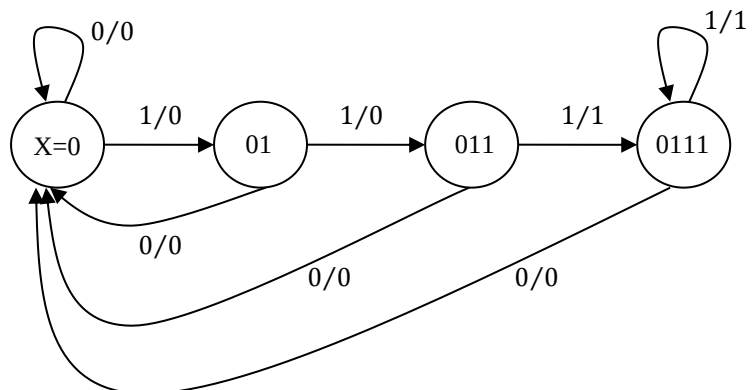
$J_A = C\bar{B}$, $K_A = B$
 $J_B = AC$, $K_B = 1$
 $J_C = 1$, $K_C = \bar{A}\bar{B}$

Örnek 1.3. Girişe gelen her 3 adet Lojik 1 değeri için çıkışında Lojik 1 veren tek girişli tek çıkışlı senkron ardışıl devreyi gerçekleştiriniz.

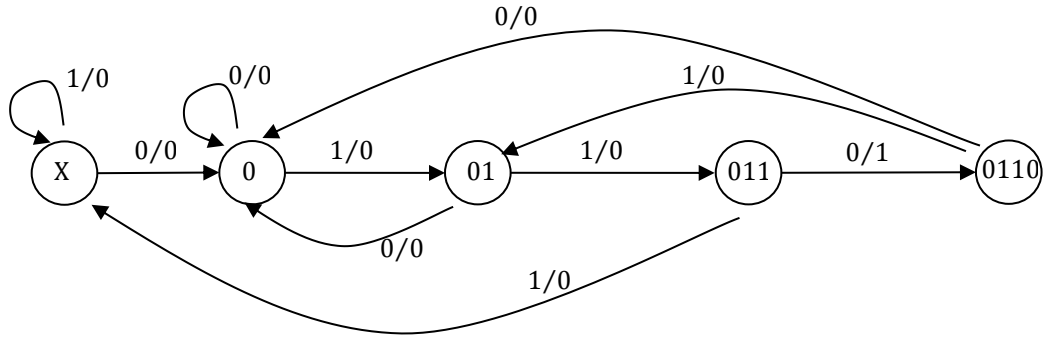
$X \rightarrow 0110010111011110$

$Z \rightarrow 0000000001000110$

X=0 durumundan başlayalım.



Örnek 1.4. 0110 dizisini yakalayan devreyi JK flip-flopları kullanarak gerçekleyiniz.



Şimdiki Durum	Gelecek Durum		Çıkış (Z)	
	x = 0	x = 1	x = 0	x = 1
a	b	a	0	0
b	b	c	0	0
c	b	d	0	0
d	c b	a	1	0
e	b	c	0	0

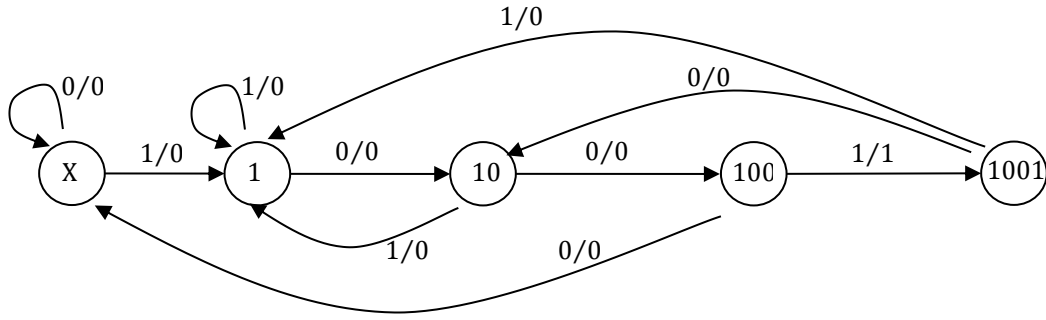
Şimdiki Durum	Giriş	Sonraki Durum	FF Uyarma Girişleri	Çıkış
A B	X	A B	$J_A K_A$ $J_B K_B$	Z
0 0	0	0 1	0 X 1 X	0
0 0	1	0 0	0 X 0 X	0
0 1	0	0 1	0 X X 0	0
0 1	1	1 0	1 X X 1	0
1 0	0	0 1	X 1 1 X	0
1 0	1	1 1	X 0 1 X	0
1 1	0	0 1	X 1 X 0	1
1 1	1	0 0	X 1 X 1	0

$\begin{matrix} BX \\ A \end{matrix}$	00	01	11	10
0	0	0	1	0
1	X	X	X	X

$$J_A = BX, \quad K_A = \bar{X} + B$$

$$J_B = \bar{X} + A, \quad K_B = X, \quad Z = AB\bar{X}$$

Örnek 1.5. 1001 dizisini yakalayan devreyi D flip-flopları kullanarak gerçekleyiniz.



Şimdiki Durum	Gelecek Durum		Çıkış (Z)	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	b	0	0
c	d	b	0	0
d	a	e	0	1
e	c	b	0	0

Şimdiki Durum	Giriş	Sonraki Durum	FF Uyarma Girişleri		Çıkış
$A B$	X	$A B$	D_A	D_B	Z
0 0	0	0 0	0	0	0
0 0	1	0 1	0	1	0
0 1	0	1 0	1	0	0
0 1	1	0 1	0	1	0
1 0	0	1 1	1	1	0
1 0	1	0 1	0	1	0
1 1	0	0 0	0	0	0
1 1	1	0 1	0	1	1

$BX \backslash A$	00	01	11	10
0	0	0	0	1
1	1	0	0	0

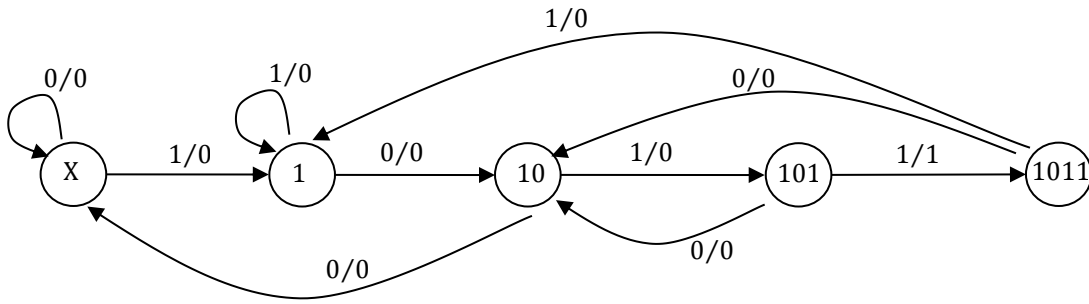
$$D_A = \bar{A}B\bar{X} + A\bar{B}\bar{X}$$

$BX \backslash A$	00	01	11	10
0	0	1	1	0
1	1	1	1	0

$$D_B = X + A\bar{B}$$

$$Z = ABX$$

Örnek 1.6. 1011 dizisini yakalayan devreyi T flip-flopları kullanarak gerçekleyiniz.



Şimdiki Durum	Gelecek Durum		Çıkış (Z)	
	x = 0	x = 1	x = 0	x = 1
a	a	b	0	0
b	c	b	0	0
c	a	d	0	0
d	c	b	0	1
e	c	b	0	0

Şimdiki Durum	Giriş	Sonraki Durum	FF Uyarma Girişleri		Çıkış
A B	X	A B	T _A	T _B	Z
0 0	0	0 0	0	0	0
0 0	1	0 1	0	1	0
0 1	0	1 0	1	1	0
0 1	1	0 1	0	0	0
1 0	0	0 0	1	0	0
1 0	1	1 1	0	1	0
1 1	0	1 0	0	1	0
1 1	1	0 1	1	0	1

BX \ A	00	01	11	10
0	0	0	0	1
1	1	0	1	0

$$T_A = A\bar{B}\bar{X} + ABX + \bar{A}B\bar{X}$$

BX \ A	00	01	11	10
0	0	1	0	1
1	0	1	0	1

$$T_B = \bar{B}X + B\bar{X} = B \oplus X$$

$$Z = ABX$$

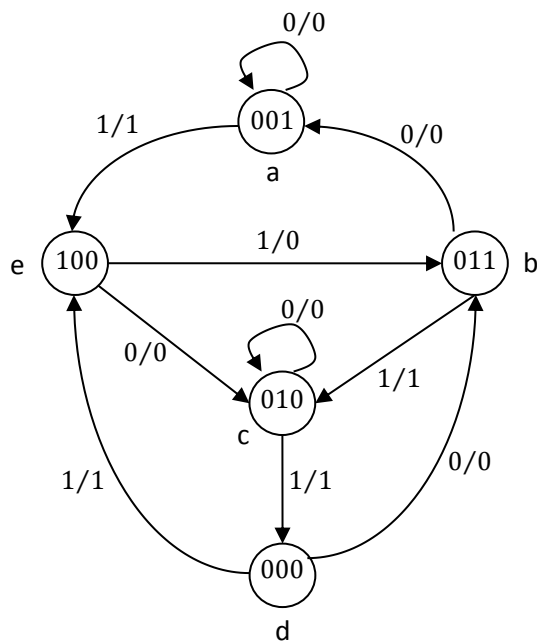
1.1.3. Senkron Ardışıl Devre Sentezi

Ardışıl devrenin tasarımı sentez işleminin konusudur. Tasarım işlemi için aşağıdaki işlem adımları takip edilir.

- Devrenin davranışı sözle tanımlanır. Bu tanımlamaya uygun olarak durum diyagramı çıkartılır. Bu işlem için zaman diyagramı veya diğer bilgilerden faydalanılabilir.
- Durum diyagramından faydalanılarak, mümkünse durumlar azaltılır.
- Kullanılacak FF sayısı ve tipi belirlenir.
- Durum diyagramında kullanılmayan durumlar da göz önüne alınarak uygulama tablosu çıkartılır.
- Uygulama tablosundan FF uyarma tablosu ve çıkış tablosu çıkartılır. Bu tablolarda kullanılan değişkenler devrenin bağımlı ve bağımsız büyüklükleridir.
- Uyarma ve çıkış tablolarından lojik fonksiyon elde edilir.
- Lojik fonksiyonlara uygun olarak devre çizilir.

Kullanılmayan durumlar için bir istek belirtilmemişse bu durumlar tasarım sırasında göz önüne alınmazlar. Bazı devrelerde çıkış olmayabilir, sadece giriş olabilir.

Örnek 1.4. SR FF kullanarak verilen durum diyagramına ait lojik devreyi gerçekleştiriniz.



$Q(t)$	$Q(t+1)$	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

Durum tablosu şu şekilde olur:

Şimdiki Durum	Gelecek Durum		Çıkış (Z)	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	e	0	1
b	a	c	0	1
c	c	d	0	1
d	b	e	0	1
e	c	b	0	0

Tablo incelenirse durum sayısının azaltılamayacağı görülür. 5 durum olduğu için tasarım için 3 FF gerekir.

Durum	Ş.D.			Giriş	G.D.			Durum	FF Uyarma Girişleri						Çıkış
	A	B	C		A	B	C		S_A	R_A	S_B	R_B	S_C	R_C	
d	0	0	0	0	0	1	1	b	0	X	1	0	1	0	0
d	0	0	0	1	1	0	0	e	1	0	0	X	0	X	1
a	0	0	1	0	0	0	1	a	0	X	0	X	X	0	0
a	0	0	1	1	1	0	0	e	1	0	0	X	0	1	1
c	0	1	0	0	0	1	0	c	0	X	X	0	0	X	0
c	0	1	0	1	0	0	0	d	0	X	0	1	0	X	1
b	0	1	1	0	0	0	1	a	0	X	0	1	X	0	0
b	0	1	1	1	1	0	0	c	0	X	X	0	0	1	1
e	1	0	0	0	0	1	0	c	0	1	1	0	0	X	0
e	1	0	0	1	0	1	1	b	0	1	1	0	1	0	0

Kullanılmayan durumlar: 101, 110, 111 $\Rightarrow$ Keyfi değer

$\begin{matrix} CX \\ AB \end{matrix}$	00	01	11	10
00	0	1	1	0
01	0	0	0	0
11	X	X	X	X
10	0	0	X	X

$$S_A = \bar{A} \bar{B} X, R_A = A$$

$$S_B = \bar{C} \bar{X} + \bar{A} B, \quad R_B = \bar{A} \bar{C} X + C \bar{X}$$

$$S_C = \bar{A} \bar{B} \bar{X} + A X, \quad R_C = A X$$

$\begin{matrix} CX \\ AB \end{matrix}$	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	X	X	X	X
10	0	0	X	X

$$Z = \bar{A} X$$

Örnek 1.5. JK FF' lar ile Mod 4 sayıcısı aşağıdaki özelliklere göre gerçekleştirilecektir:

a) Sayıcı, iki girişli (X_1, X_2) ve iki çıkışlı (Z_1, Z_2) ileri- geri sayıcıdır

b) İki girişin ikisi birden 1 olmamaktadır.

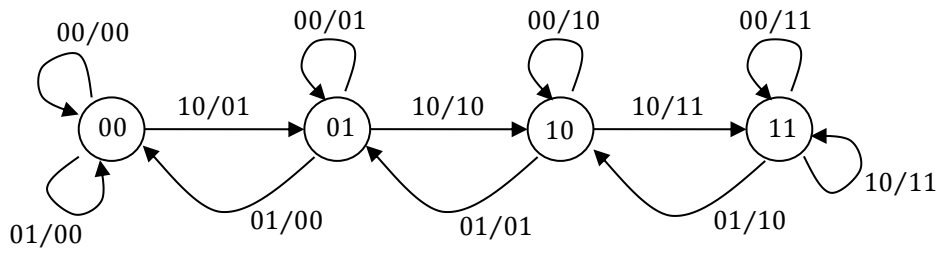
c) $X_1 = 1$ olduğunda ileri saymakta fakat çıkış maksimum sayıyı gösterince sabit kalmaktadır
 $X_2 = 1$ olduğunda geri saymakta fakat çıkış minimum sayıyı gösterince sabit kalmaktadır.
 $Z_1 Z_2 = AB$ ifadesi FF' ların sonraki durumunu göstermektedir.

Sayılacak sayılar 0, 1, 2, 3 olduğu için 2 FF yeterlidir.

$$Z_1 Z_2 = AB \quad X_1 X_2 / Z_1 Z_2 = X_1 X_2 / AB$$

$AB = Z_1 Z_2 = 00$ başlangıç durumudur.

X_1	X_2	Sayıcı Durumu
0	0	Değişmeyecek
0	1	Geri sayacak
1	0	İleri sayacak
1	1	Keyfi



Şimdiki Durum	Giriş	Gelecek Durum	FF Uyarma Girişleri		Çıkış
$A \ B$	$X_1 \ X_2$	$A \ B$	$J_A \ K_A$	$J_B \ K_B$	$Z_1 \ Z_2$
0 0	0 0	0 0	0 X	0 X	0 0
0 0	0 1	0 0	0 X	0 X	0 0
0 0	1 0	0 1	0 X	1 X	0 1
0 0	1 1	X X	X X	X X	X X
0 1	0 0	0 1	0 X	X 0	0 1
0 1	0 1	0 0	0 X	X 1	0 0
0 1	1 0	1 0	1 X	X 1	1 0
0 1	1 1	X X	X X	X X	X X
1 0	0 0	1 0	X 0	0 X	1 0
1 0	0 1	0 1	X 1	1 X	0 1
1 0	1 0	1 1	X 0	1 X	1 1
1 0	1 1	X X	X X	X X	X X
1 1	0 0	1 1	X 0	X 0	1 1
1 1	0 1	1 0	X 0	X 1	1 0
1 1	1 0	1 1	X 0	X 0	1 1
1 1	1 1	X X	X X	X X	X X

$\begin{matrix} AB \\ X_1 X_2 \end{matrix}$	00	01	11	10
00	0	0	X	X
01	0	0	X	X
11	X	X	X	X
10	0	1	X	X

$$J_A = BX_1$$

$\begin{matrix} AB \\ X_1 X_2 \end{matrix}$	00	01	11	10
00	X	X	0	0
01	X	X	0	1
11	X	X	X	X
10	X	X	0	0

$$K_A = X_2 \bar{B}$$

$\begin{matrix} AB \\ X_1 X_2 \end{matrix}$	00	01	11	10
00	0	X	X	0
01	0	X	X	1
11	X	X	X	X
10	1	X	X	1

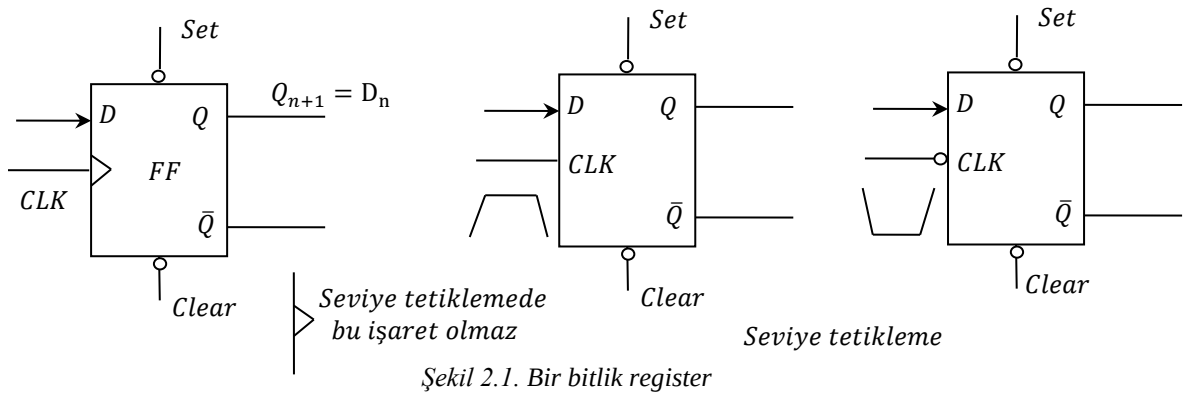
$$J_B = X_1 + X_2 A$$

$\begin{matrix} AB \\ X_1 X_2 \end{matrix}$	00	01	11	10
00	X	0	0	X
01	X	1	1	X
11	X	X	X	X
10	X	1	0	X

$$K_B = X_1 \bar{A} + X_2$$

2. REGİSTERLAR

Bir sayısal sistemin saklayabileceği en küçük bilgi birimi 1 veya 0 lojik değerine sahip bir ikili (binary) bilgi veya bir bittir. Bir veri biti, FF veya bir bit saklayıcı (register) olarak adlandırılan elektronik devrede saklanır. Bir bitlik bir FF hafıza hücresi olarak adlandırılır. Çalışma gücü kesilmediği ve sinyaller ile durumun değişmediği takdirde, süresiz kalabileceği iki kararlı duruma sahiptir. Aşağıdaki şekilde bir bitlik bir register görülmektedir.

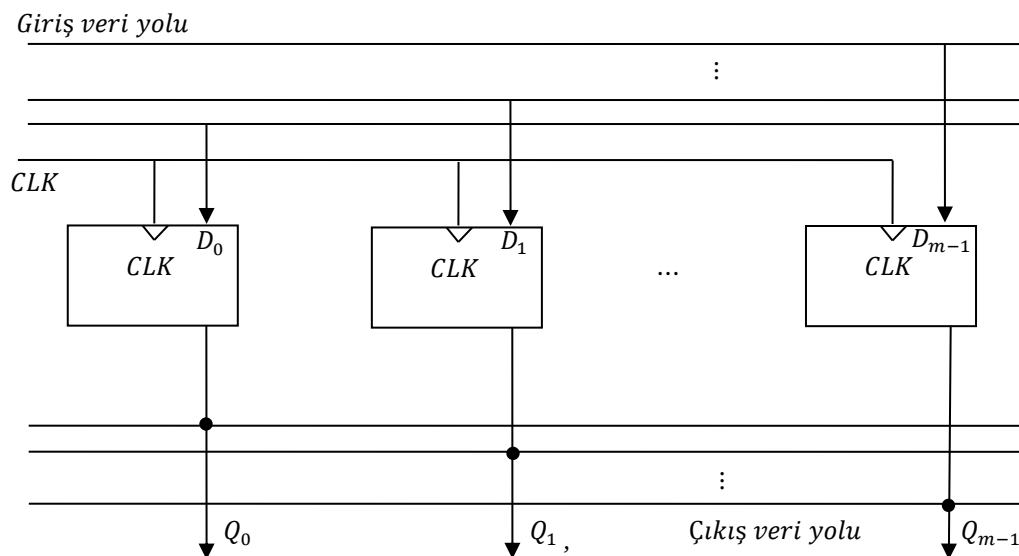


CLK tetikleme seviyesinde kaldığı müddetçe çıkış girişi takip eder.

D tipi FF mikroişlemcili sistemlerde oldukça yoğun olarak kullanılmaktadır. Diğer FF' larda kullanılabilir.

2.1.m-bit register

Birden çok veri bitini aynı anda saklamak için D tipi FF' ların saat girişleri m-bit register oluşturacak şekilde paralel olarak birleştirilir.



Şekil 2.2. m-tane D tipi FF' tan oluşan m-bit register

bit: binary digit (0 ya da 1)

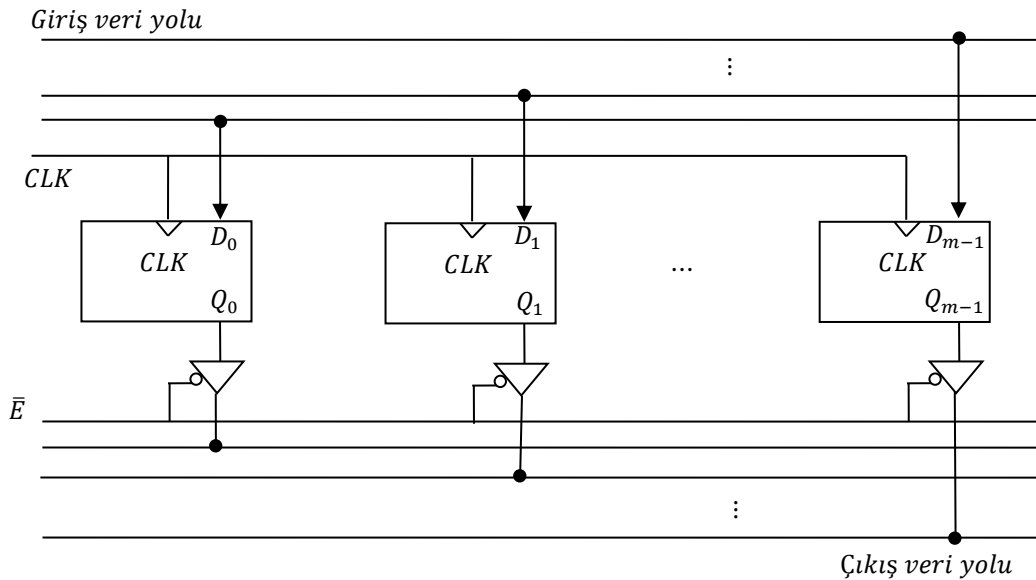
nibble: 4 bitlik veri paketi ($0_{10} - 15_{10}$ (16 adet) 0000 – 1111)

byte: 8 bitlik veri paketi ($0_{10} - 255_{10}$ (256 adet) 00000000 – 11111111)

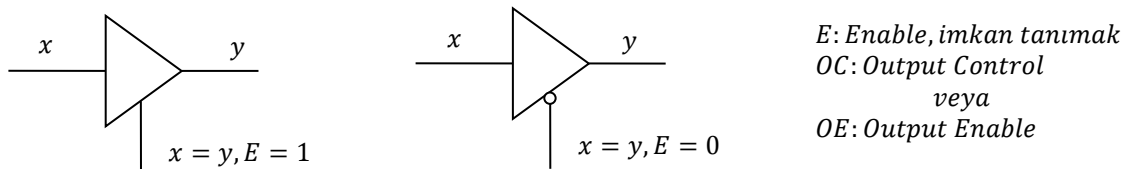
Bir byte'lık bir alan 256 adet veri kaydedilebilir.

Genelde mikroişlemci sistemlerde temel veri yolu uzunluk birimi olduğundan değişik yapılarda 8 bit registerlar üretilmiştir. Örneğin 74X273 yukarıdaki şekilde görüldüğü gibi 8 tane D tipi FF içerir.

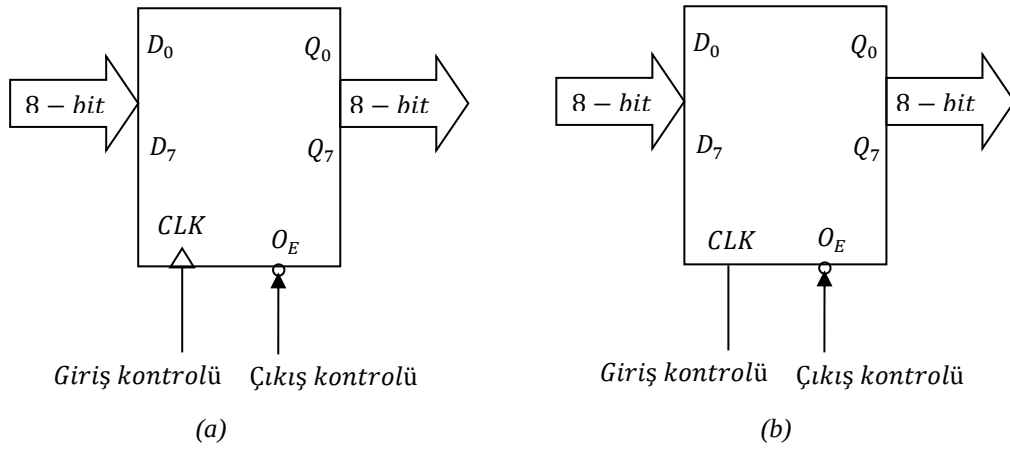
Genelde FF çıkışlarındaki bilgilerin bir kontrol mekanizmasına bağlı olarak ilgili birimlere aktarılması istenir. Bu durumda FF'ların veri çıkışlarına 3 durumlu buffer eklenir. Böylece paralel olarak veri aktarımı kontrol altına alınmış olur.



Şekil 2.3. Üç durum çıkışlara sahip m-bir register

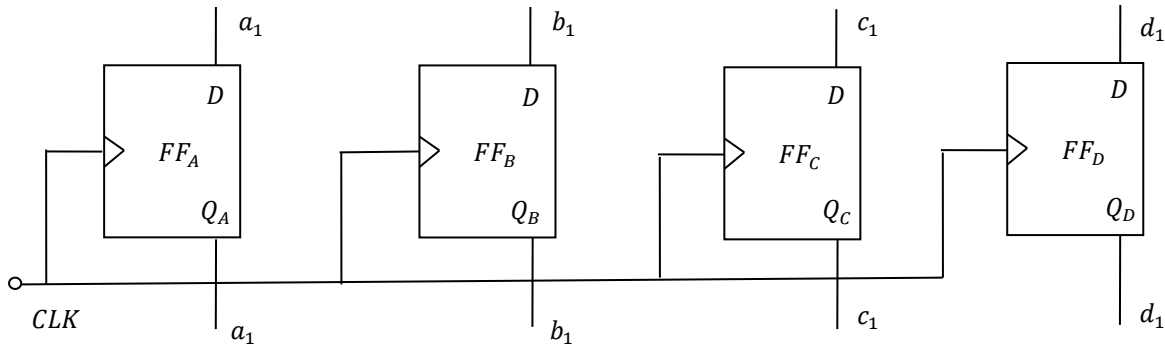


74X374, 74X574 entegreleri 8 bitlik pozitif kenar tetiklemeli 3 durumlu çıkışlara sahip entegrelerdir.



Şekil 2.4. Üç durum çıkışlı 8-bit (octal) register (a) Kenar tetiklemeli, (b) Seviye tetiklemeli

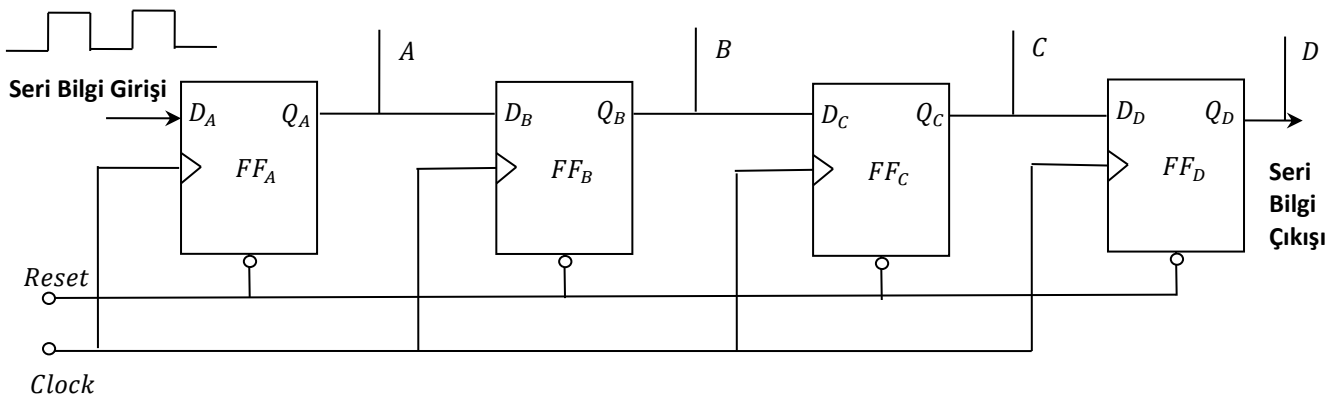
2.2. Registerlarda paralel bilgi aktarımı



Şekil 2.5. Registerlarda paralel bilgi aktarımı

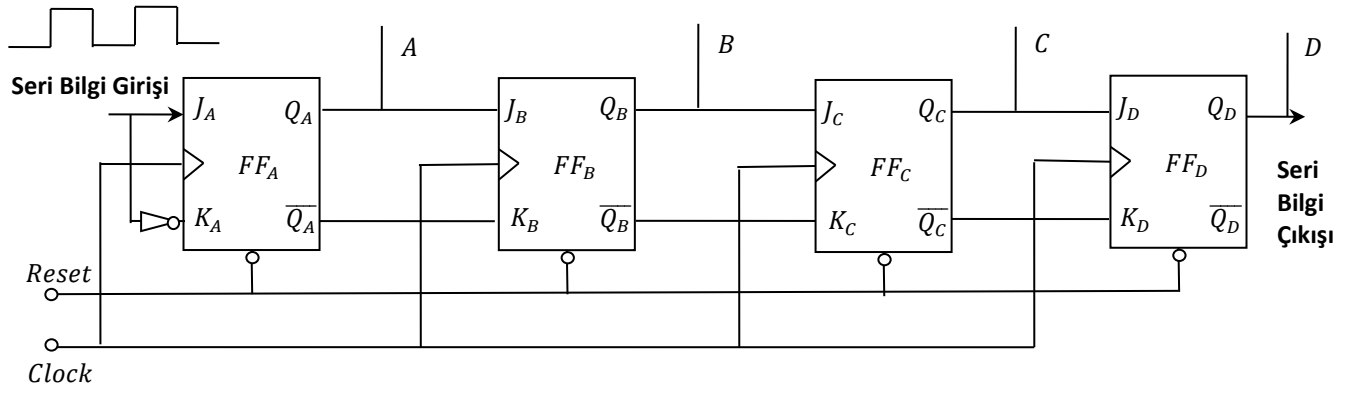
Seri bilgi aktarımı shift (kaydırıcı) register ile yapılır.

2.3. Sağa ötelemeli register



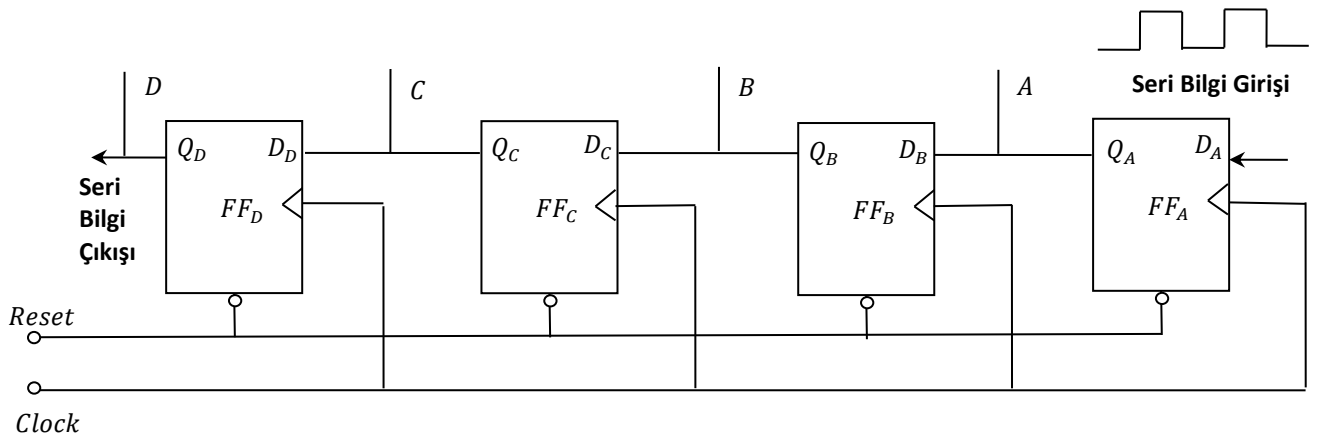
Şekil 2.6. 4-bitlik sağa ötelemeli register (shift register)

4 CLK darbesi ile 4 bitlik bilgi yerleşir. Başka bir deyişle 4 CLK darbesinden sonra girişe gelen ilk bilgi çıkıştan alınır.



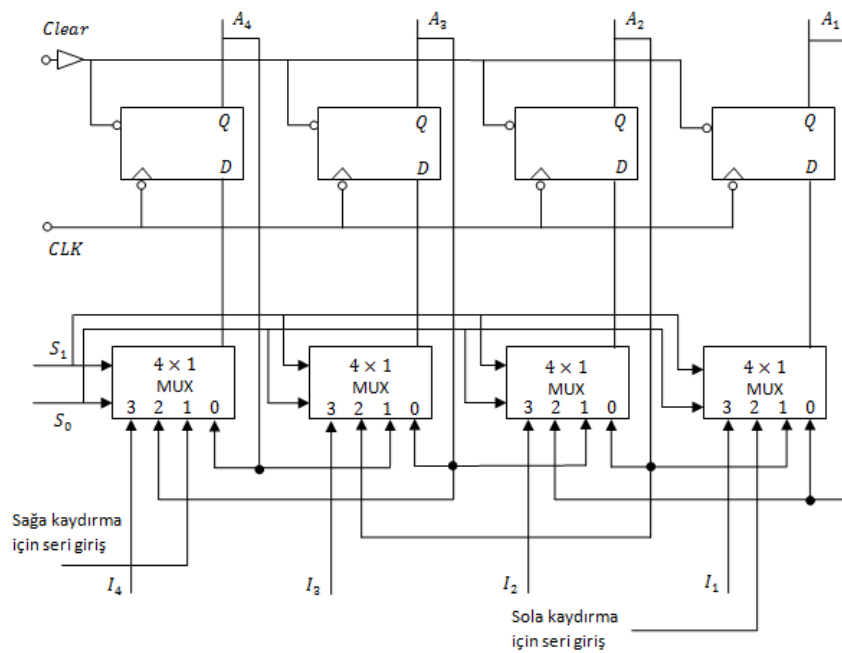
Şekil 2.7. JK FF ile 4-bitlik sağa ötelemeli register (shift register)

2.4. Sola ötelemeli register



Şekil 2.8. 4-bitlik sola ötelemeli register (shift register)

2.5. Çift yönlü paralel yüklemeli registerlar



Şekil 2.9. Paralel yüklemeli 4-bitlik iki yönlü ötelemeli register (74194)

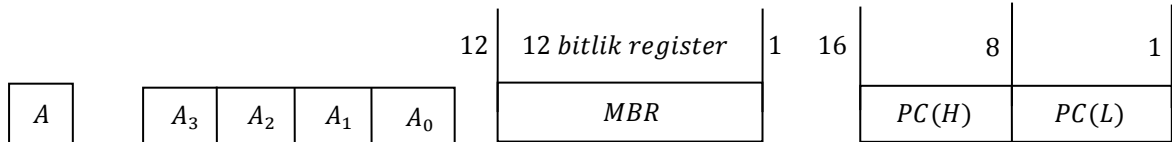
S_1	S_0	İşlem
0	0	Değişme yok
0	1	Sağa kaydır
1	0	Sola kaydır
1	1	Paralel yükle

2.6. Registerlar arası veri transferi

Registerlar arası veri transferi, verinin okunacağı bir kaynak (source) register ile verinin yazılacağı bir hedef (destination) register gerektirir. Bu iki register bir veri yolu ile birbirine bağlanmalıdır. Bir mikroişlemcili sistemde pek çok kaynak ve hedef register bulunabileceği için, her kaynak ve hedef çiftini kendilerine ayrılmış bir veri yolu ile bağlamak mümkün değildir. Bu yüzden mikroişlemcili sistemler paylaşılan veri yolu kullanır.

2.6.1. İç register transfer

Registerlar sembolik olarak aşağıdaki şekillerde olduğu gibi gösterilir:

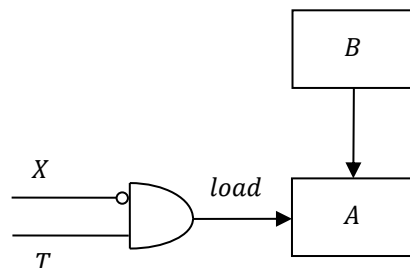


A(8) ile belirleniyorsa 8 bitlik register söz konusudur (MBR ve PC mikroişlemcilerdeki özel register isimleri, PC: Program counter).

$$MBR(16)=PC(16) \quad PC(L)=PC(1-8), \quad PC(H)=PC(9-16)$$

Bir registerdan diğerine veri transferi yer değiştirme operatörü ile gösterilir. $A \leftarrow B$ ifadesi B' den A' ya veri transferini gösterir. Transfer işleminin zamanını belirleyen kontrol fonksiyonları bir boolean fonksiyonudur ve bu şekilde olan işlemler aşağıdaki gibi gösterilir.

$X'T_1: A \leftarrow B$ ifadesi $X = 0, T_1 = 1$ olması durumunda B' deki bilginin A' ya transferi mevcut.

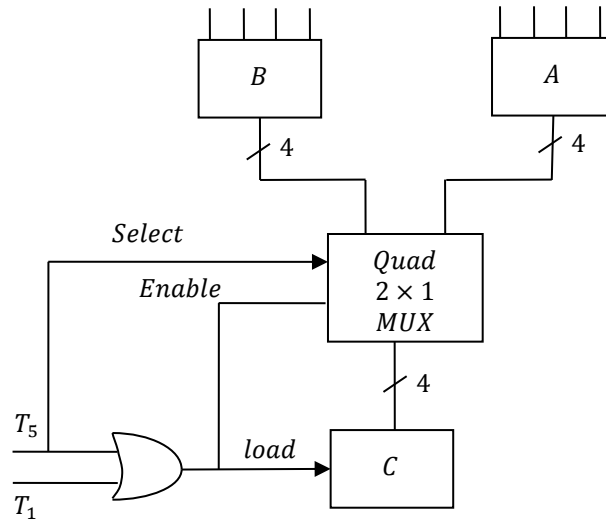


Sembol	Tanım	Örnek
Harf (Rakam)	Register	A, MBR, R_2
İndis	Register biti	A_2, B_6
()	Register parçası	$PC(H), MBR(L)$
$\leftarrow$	Veri transferi ve yönü	$A \leftarrow B$
:	Kontrol fonksiyonları sınırlamalı	$X'T_0: A \leftarrow B$
,	İki mikroişlemi ayırma	$A \leftarrow B, B \leftarrow A$ $T_3: A \leftarrow B, B \leftarrow A$
[]	Memory transferi için adresin belirlenmesi	$MBR \leftarrow M[MAR]$

Bazı durumlarda herhangi bir register farklı iki kaynaktan (register) aynı anda olmamak şartı ile bilgi alabilir.

$T_1: C \leftarrow A$

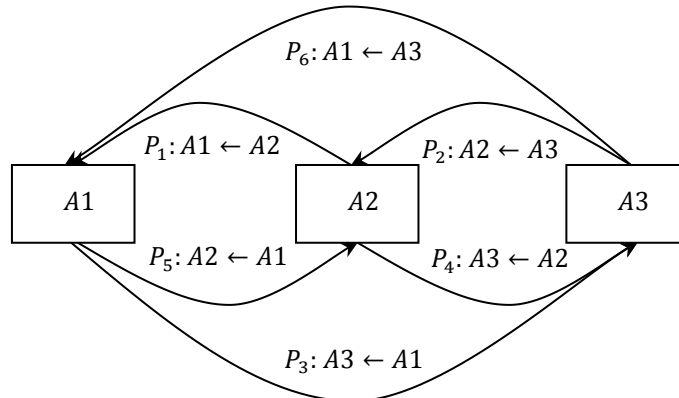
$T_5: C \leftarrow B$ A, B : Kaynak register, C : Hedef register



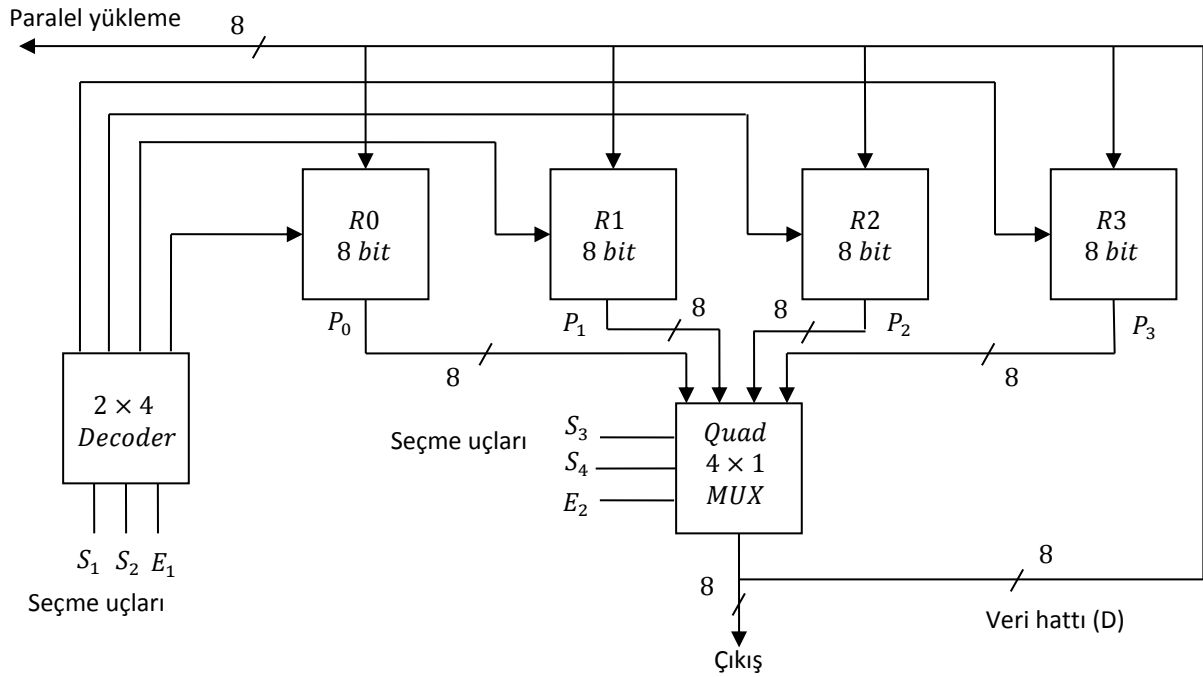
Şekil 2.10. $T_1: C \leftarrow A$ MUX kullanarak iki kaynaktan tek bir registra bilgi aktarımı

2.6.2. Bus transfer

Dijital sistemler birçok registra ve birçok registerdan diğerine veri transferinde birçok iletim hatlarına sahiptir.



Örnek 2.2. 4 registerdan oluşmuş bir bus organizasyonunun incelenmesi



E_1, E_2 : İzin uçları, aktif olduğu zaman buna ait eleman (entegre devre) çalışır.

Sistemin çalışması için E_1 ve E_2 ' yi sürekli aktif tutmalıyız.

S_1	S_2	Dekoder çıkışı
0	0	0 ($R0$ registeri etkin)
0	1	1 ($R1$ registeri etkin)
1	0	2 ($R2$ registeri etkin)
1	1	3 ($R3$ registeri etkin)

S_3	S_4	MUX çıkışı
0	0	$R0$ çıkışa aktarılır
0	1	$R1$ çıkışa aktarılır
1	0	$R2$ çıkışa aktarılır
1	1	$R3$ çıkışa aktarılır

$R_3 \leftarrow R_0$ işlemi yapmak istiyoruz

1. P_0 kontrol sinyali ile $D \leftarrow R0$ işlemi yapılır. Buna göre $R0$ ' ı dataya aktarabilmek için $S_3S_4 = 00$ ve $E_2 = 1$ olmalı. P_0 kontrol kelimesi $S_3S_4E_1 = 001$ ' dir.

2. Data $R3$ ' e aktarılır ($R3 \leftarrow D$ işlemi yapılıyor)

Bunu yapmak için dekoderin 3 nolu çıkışına aktif yapmalıyız. Bunun için $S_1S_2 = 11$, $E_1 = 1$ olmalıdır. P_3 kontrol kelimesi 111 olmalıdır.

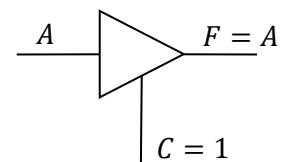
Böylece $R_3 \leftarrow R_0$ mikroişlemi için kontrol kelimesi

P_3	P_0
111	001

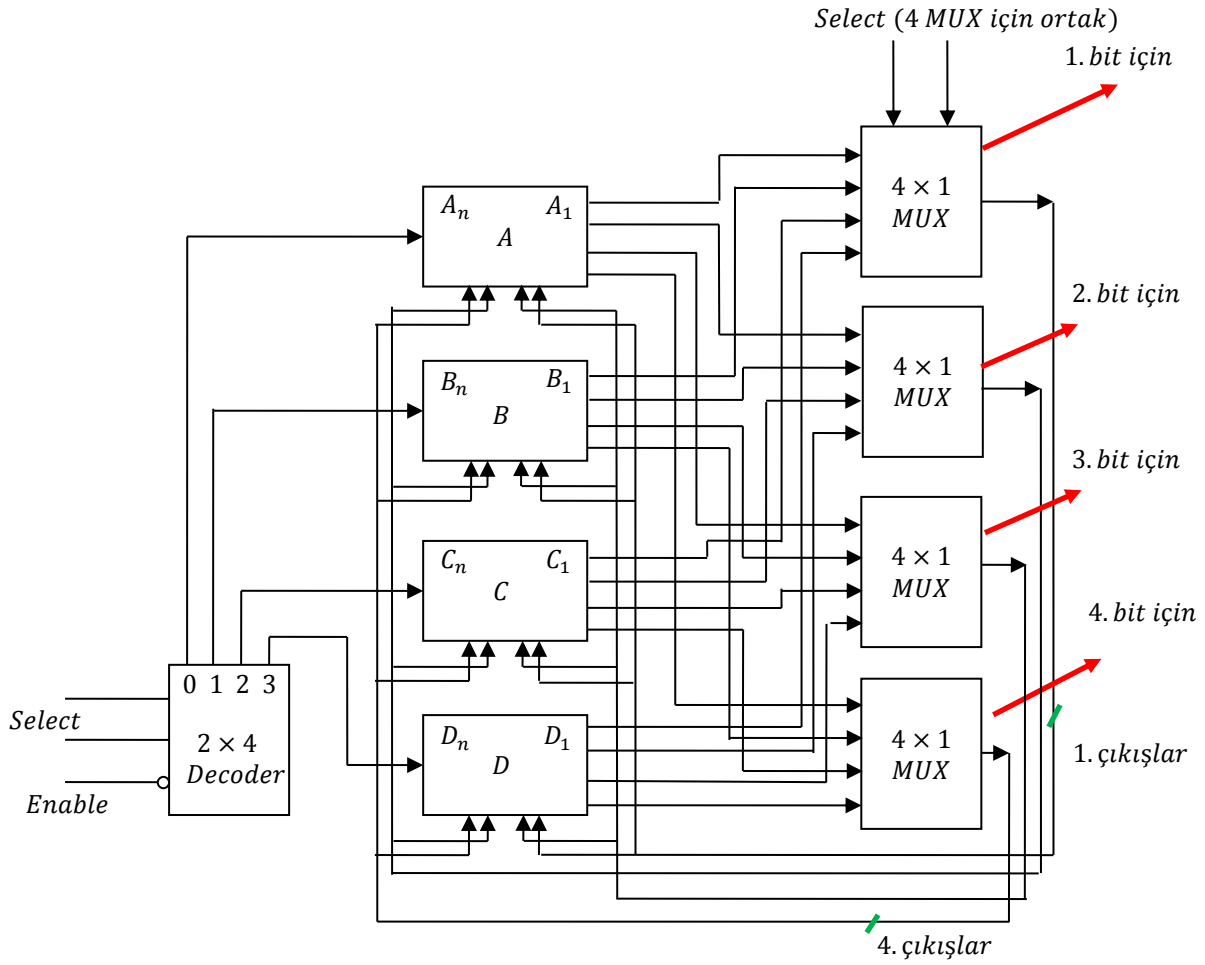
E_1 ve E_2 aktif olmazsa elemanların çıkışları yüksek empedans gösterir.

$C = 1$ ise $F = A$

$C = 0$ ise A ile F arası açık devre (yüksek empedans)

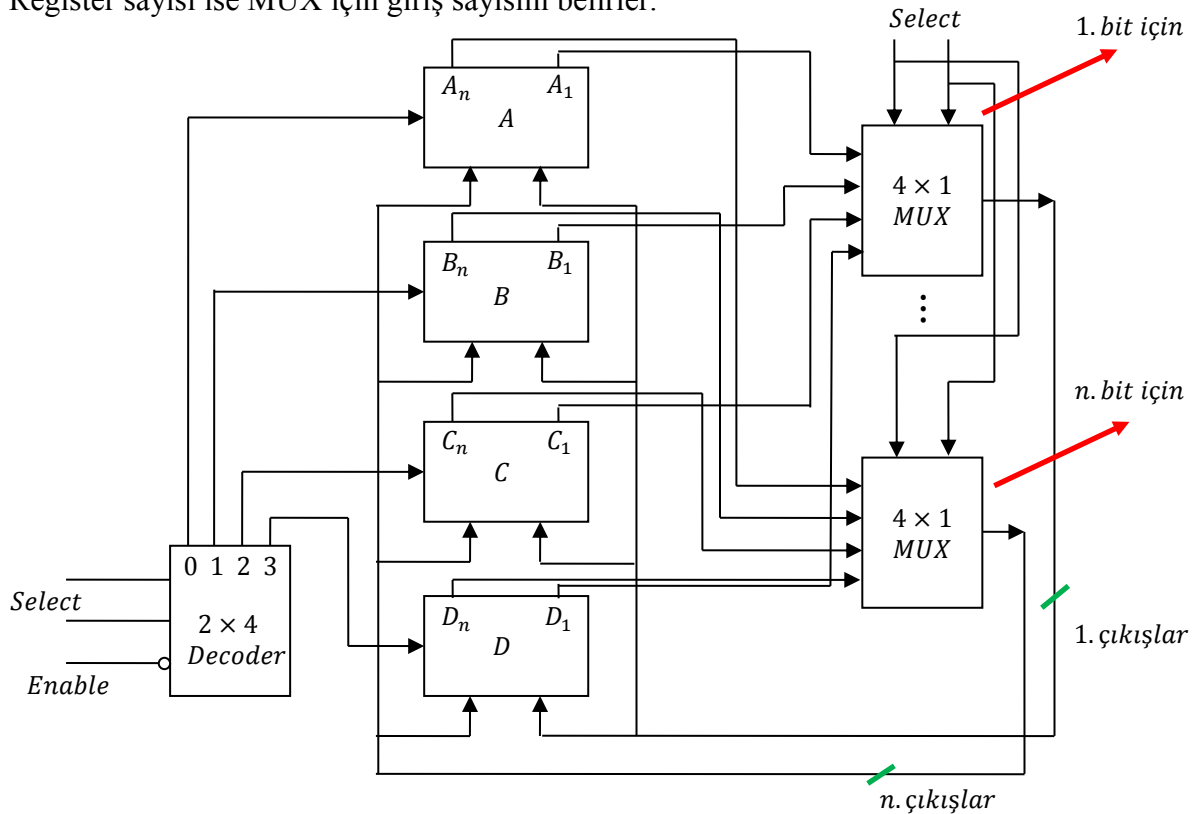


Yukarıdaki 4 register için ortak bus' ı daha detaylı çizersek aşağıdaki devreyi elde ederiz.



Bit sayısı kadar MUX gereklidir.

Register sayısı ise MUX için giriş sayısını belirler.

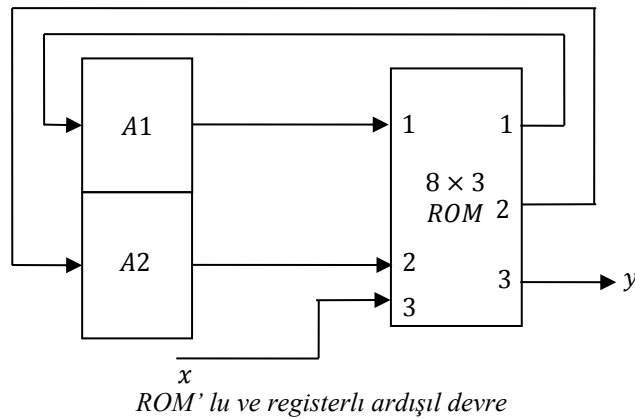


Örnek 2.4. Yukarıdaki örneği ROM kullanarak dizayn ediniz.

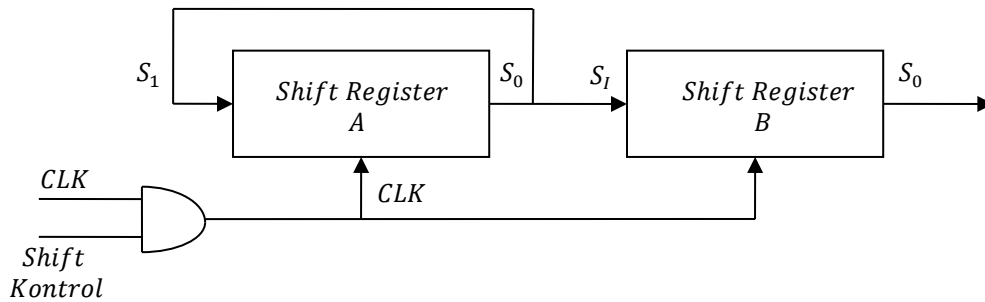
Adres							Çıkışlar			Depolu bilgi						
Adres							Çıkışlar			Depolu bilgi						
1	2	3	1	2	3		1	2	3							
0	0	0	0	0	0	1	0	0	0	0	0	0	4	5	...	n
0	0	1	0	0	1	2	0	1	0	...	...	...	...	...	...	...
0	1	0	0	0	1	3	0	1	0	...	...	...	...	...	...	...
0	1	1	0	0	0	4	1	0	0	...	...	...	...	...	...	...
1	0	0	1	0	0	5	0	1	0	...	...	...	...	...	...	...
1	0	1	0	0	1	6	1	1	0	...	...	...	...	...	...	...
1	1	0	1	0	1	7	0	0	1	...	...	...	...	...	...	...
1	1	1	0	1	1	...				...	...	...	...	...	...	...
1	1	1	1	0	0	n				...	...	...	...	...	...	...

Çıkışlar hafıza bit sayısını belirtir. Girişler adres hattını verir. Burada 8×3 ROM gerekir.

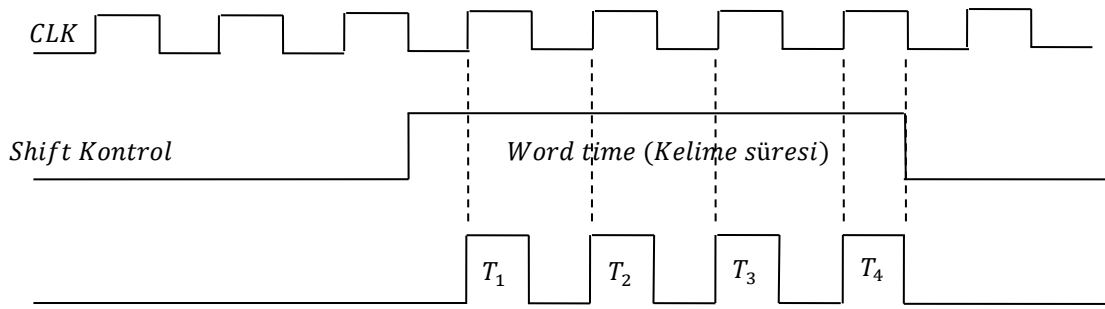
ROM' a yapılan giriş sayısı FF' ların sayısı ve harici girişlerin sayısının toplamı kadardır. ROM çıkışlarının sayısı ise FF' ların sayısı ile harici çıkışların sayılarının toplamı kadardır. Bu durumda ROM büyüklüğünün 8×3 olması gerekir (3 giriş, 3 çıkış var).



2.8. Seri Transfer



Şekil 2.12. Seri transfer blok gösterim

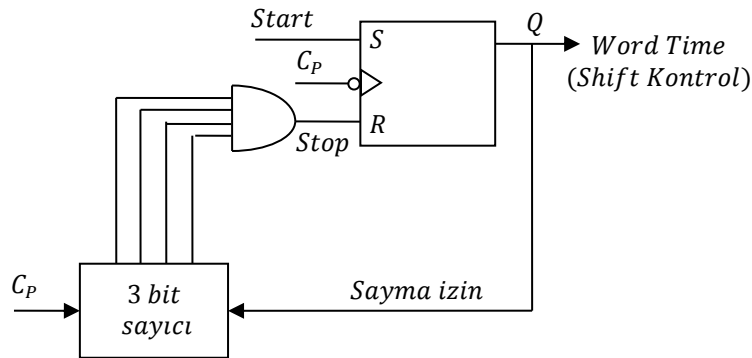


Şekil 2.13. Zamanlama diyagramı

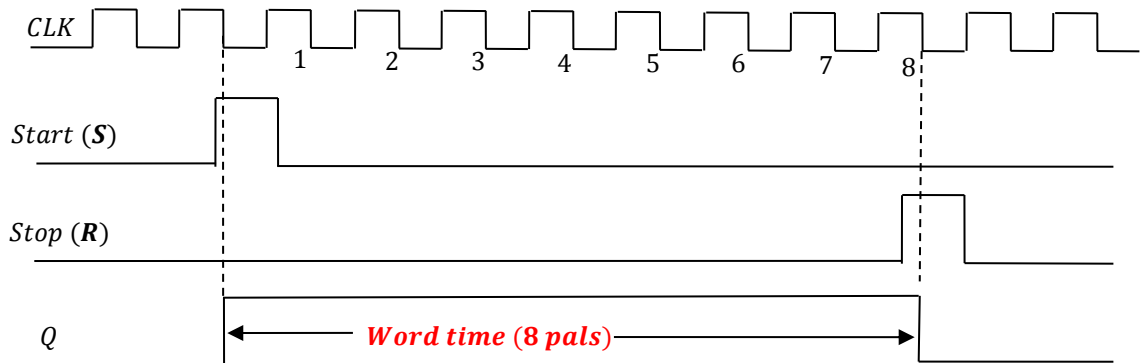
Word time: Registerdaki bilginin kaydırılması için geçen süre

Zamanlama darbesi	Shift Register-A	Shift Register-B	B' nin seri çıkışı
Başlangıç değeri	1011	0010	Başlangıç
T_1 ' den sonra	1101	1001	1
T_2 ' den sonra	1110	1100	0
T_3 ' den sonra	0111	0110	0
T_4 ' den sonra	1011	1011	1

Word Time sinyalinin üretilmesi



Şekil 2.14. Word Time sinyalini üreten devre

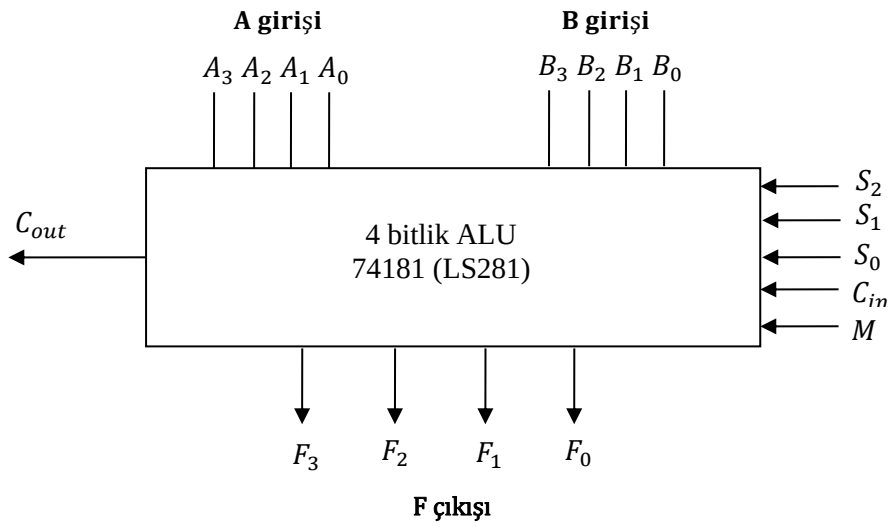


3. ALU TASARIMI

ALU iki kısımdan oluşur:

- Aritmetik Ünite: Aritmetik işlemlerin yapıldığı ünite
- Lojik Ünite: Lojik işlemlerin yapıldığı ünite

4 bitlik bir ALU için blok şeması aşağıdaki gibidir:



Şekil 3.1. 4 bitlik bir ALU için blok gösterim

C_{in} : Elde girişi, bir önceki ALU' dan geleni alır.

S_1, S_0 : Fonksiyon seçme uçları

S_2 : Mod seçme ucu (Aritmetik yada lojik işlemlerin hangisinin seçileceğini belirler)

C_{in} ' e kendimizde giriş verebiliriz. Eğer C_{in} girişlerini hiçbir yerde kullanmaz isek, C_{in} girişi de mod seçici giriş olarak kullanılır ve 4 tane seçme girişi olur.

Burada işlemler bit bit yapılır.

$$A + B = A_3A_2A_1A_0 + B_3B_2B_1B_0$$

$$A = 1111, \quad B = 1001, \quad A + B = 11000 = C_{out}F_3F_2F_1F_0$$

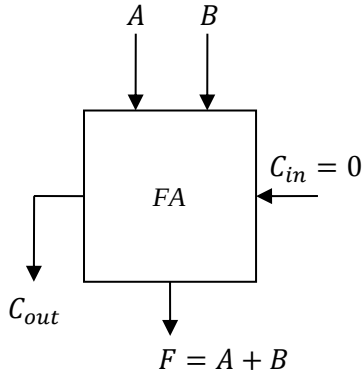
74181 entegresinde (ALU), $S_0 - S_3$: Fonksiyon seçme uçları, M : Aritmetik/Lojik seçme ucu,

C_n : Aritmetik işlemler.

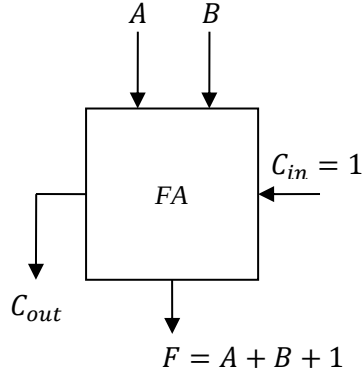
3.1. Aritmetik işlemler

Aritmetik işlemler tam toplayıcı (full adder) ile yapılır. FA iki tane biti toplayabilir.

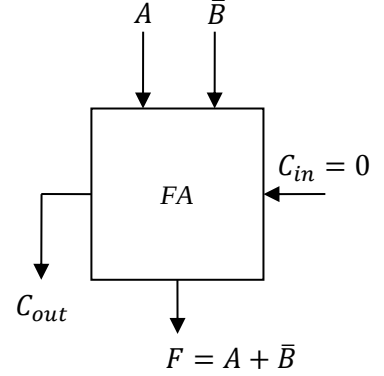
1. Toplama işlemi



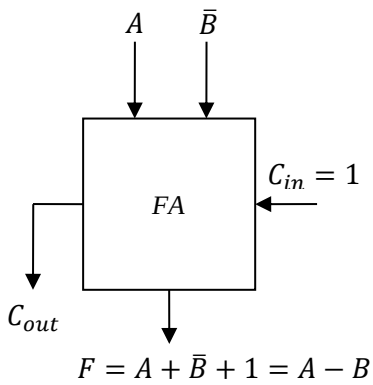
2. Eldeli toplama işlemi



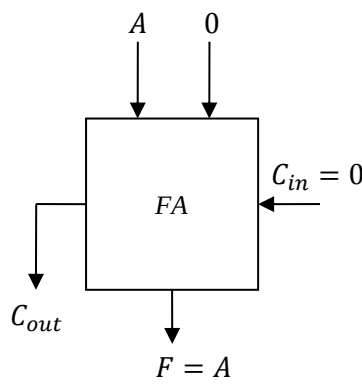
3. A' yı B̄ ile toplama



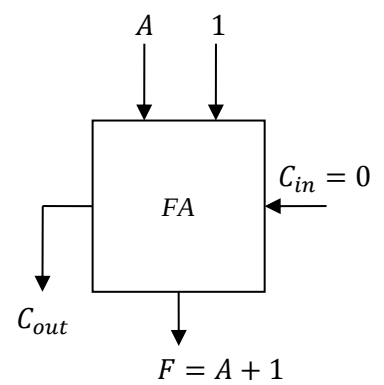
4. Çıkarma işlemi



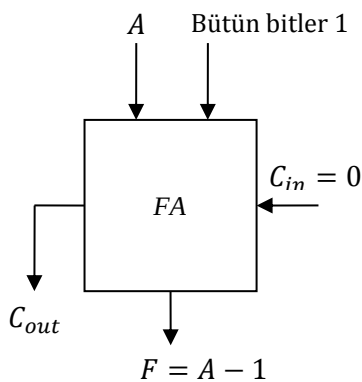
5. A' nin aktarılması



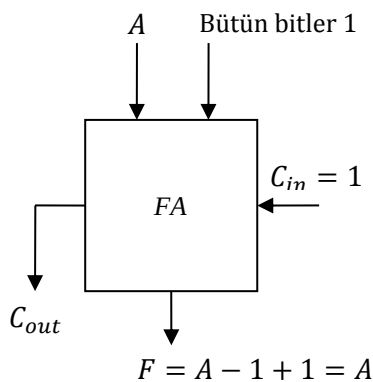
6. A' nin 1 artırılması



7. A' nin 1 azaltılması



8. A' nin aktarılması (II. method)



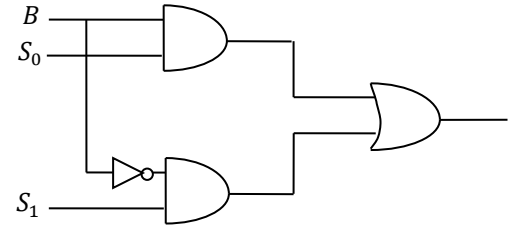
B girişlerini oluşturan devre

S_1	S_0	Çıkış
0	0	0
0	1	B
1	0	$\bar{B}$
1	1	1

S_1	S_0	B	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

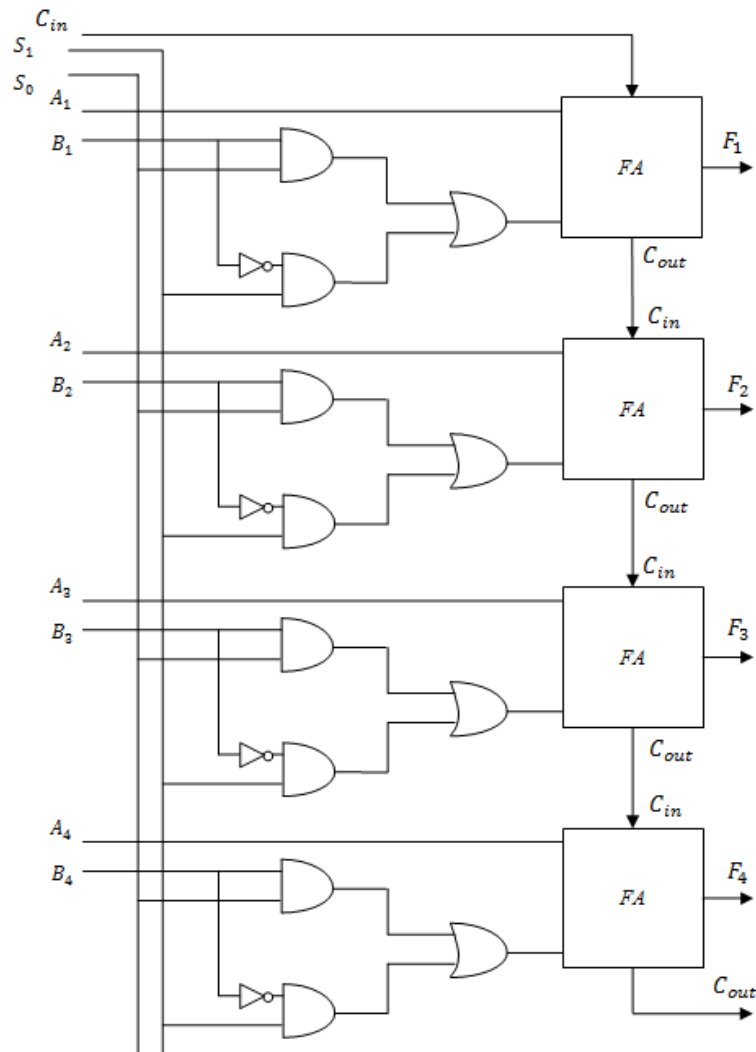
$S_0 \backslash S_1$	00	01	11	10
0			1	
1	1		1	1

$$Y = BS_0 + \bar{B}S_1$$



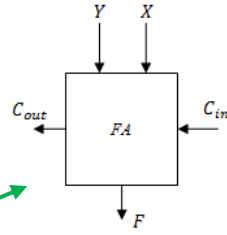
3.2. 4 Bitlik Aritmetik İşlemci Tasarımı

4 tane FA kullanarak oluşturabiliriz.



Şekil 3.2. 4 bitlik aritmetik işlemci

C_{in} girişini seçme girişi olarak kullanalım.



S_1	S_0	C_{in}	Çıkış Y	F	$C_{out} = 1$ olma şartları (C_{out} ne zaman 1 olur?)	İşlem
0	0	0	0	A	$C_{out} = 0$	A 'nın transferi
0	0	1	0	$A + 1$	$A = 2^n - 1 \Rightarrow C_{out} = 1$	Increment A
0	1	0	B	$A + B$	$A + B \geq 2^n \Rightarrow C_{out} = 1$	Toplama
0	1	1	B	$A + B + 1$	$A + B \geq 2^n - 1 \Rightarrow C_{out} = 1$	Eldeli toplama
1	0	0	$\bar{B}$	$A + \bar{B}$	$A > B \Rightarrow C_{out} = 1$	A 'nın $\bar{B}$ ile toplanması
1	0	1	$\bar{B}$	$A + \bar{B} + 1$ $= A - B$	$A \geq B \Rightarrow C_{out} = 1$	Çıkarma
1	1	0	1	$A - 1$	$A \neq B \Rightarrow C_{out} = 1$	Decrement A
1	1	1	1	A	Her zaman $C_{out} = 1$	A 'nın aktarımı

F : Aritmetik işlemci çıkışı (1): Bütün bitler 1

$Y = BS_1 + \bar{B}S_0$ ifadesinde $S_1S_0 = 11$ yazılırsa $Y = B + \bar{B} = 1$ olur.

$C_{out} = 1$ olma şartlarından bazılarını inceleyelim:

1. $F = A$ durumunda $C_{out} = 0$ olur. Çünkü $C_{in} = 0$ olduğunda elde çıkışı da toplam çıkışı da 0 olur.
2. $F = A + 1$ durumunda $C_{out} = 1$ olur. Çünkü $A = 2^n - 1 \Rightarrow A$ 'daki bitlerin hepsinin 1 olması demektir (n =bit sayısı). Bu değere 1 eklenirse $C_{out} = 1$ olur.

$A = (1111)_2, n = 4 \Rightarrow A = 2^n - 1 = 15$ $A + 1 = 10000 \Rightarrow C_{out} = 1$ olur.

3. $F = A + B$ durumunda $A + B \geq 2^n \Rightarrow C_{out} = 1$ olur.

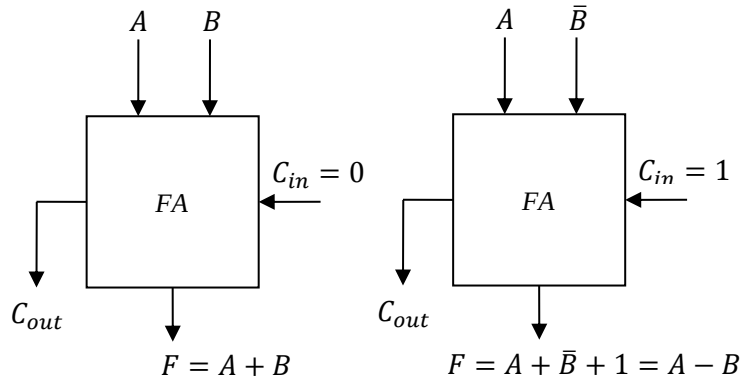
$$\begin{array}{r}
 A = (1111)_2 = (15)_{10} \\
 + \quad B = (0001)_2 = (1)_{10} \\
 \hline
 A + B = (10000)_2 = (16)_{10}
 \end{array}
 \left. \vphantom{\begin{array}{r} A \\ B \end{array}} \right\} C_{out} = 1 \text{ ve } A + B \geq 2^n \text{ 'dir.}$$

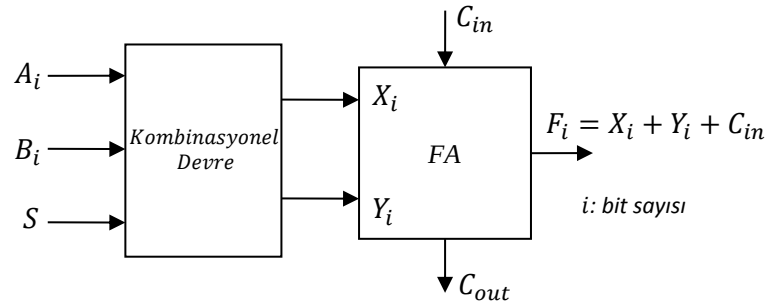
4. $F = A + B + 1$ durumunda $A + B \geq 2^n - 1$ olduğunda $C_{out} = 1$ olur.

Örnek 3.1. Girişleri A ve B seçme girişi S olan toplama ve çıkarma işlemlerini gerçekleştiren 2 bitlik aritmetik işlem birimini tasarlayınız.

$S = 0$ için $F = A + B$ olsun

$S = 1$ için $F = A - B$ olsun





$S = C_{in}$	A_i	B_i	X_i	Y_i
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	0
1	1	0	1	1
1	1	1	1	0

$\Rightarrow X_i = A_i, Y_i = B_i$
 $\Rightarrow X_i = A_i, Y_i = \bar{B}_i$

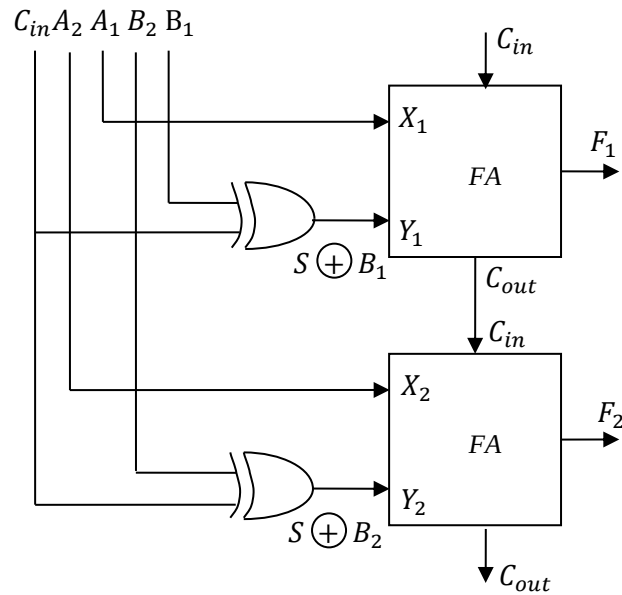
$A_i B_i$	00	01	11	10
C_{in}				
0			1	1
1	1		1	1

$X_i = A_i$

$$Y_i = S \oplus B_i$$

$A_i B_i$	00	01	11	10
C_{in}				
0		1	1	
1	1			1

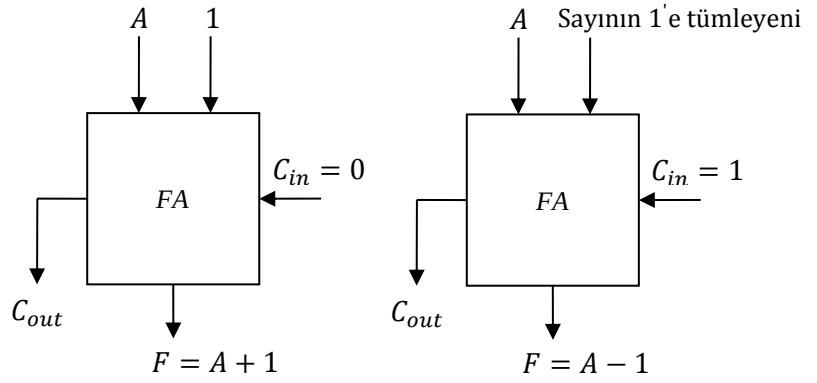
Bu durumda devre aşağıdaki gibi çizilir:



Ödev. Girişleri A ve B olan seçme girişi S olan bir devrede “increment A” ve “decrement A” işlemi yapılacaktır. Bu devreleri gerçekleştiriniz.

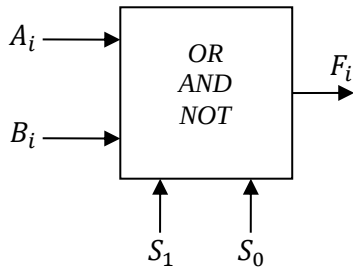
$S = 0$ için $F = A + 1$ olsun

$S = 1$ için $F = A - 1$ olsun



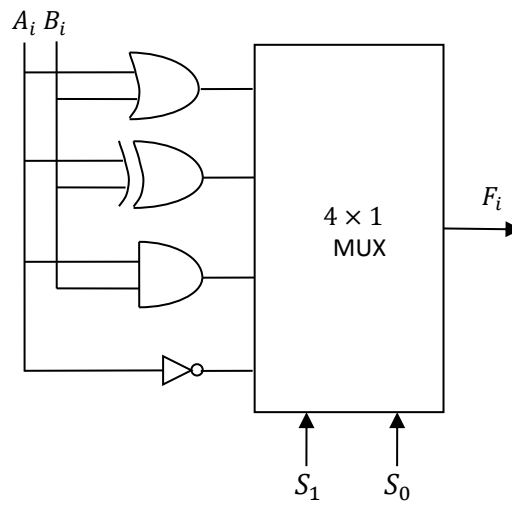
3.3. Lojik Devrenin Tasarımı

Temel lojik işlemler: AND, OR ve NOT işlemi. Diğer lojik işlemler bu 3 temel lojik işlemin toplamından oluşur.

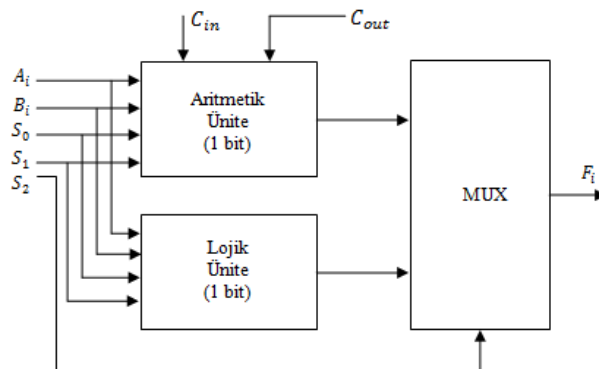


S_1	S_0	F_i	Lojik İşlem
0	0	$A_i + B_i$	OR işlemi
0	1	$A_i \oplus B_i$	XOR işlemi
1	0	$A_i \cdot B_i$	AND işlemi
1	1	$\bar{A}_i$	NOT işlemi

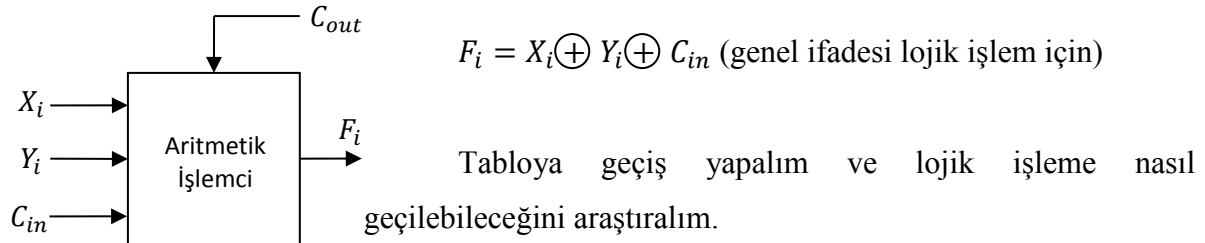
1 bitlik lojik ünite



3.4. 1 Bitlik ALU Tasarımı



$S_2 = 0$ iken aritmetik işlem yapılır, $S_2 = 1$ iken lojik işlem yapılır (MUX 2×1 ' liktir). Aritmetik işlemci ve lojik işlemciyi ayrı ayrı yapmak karışık ve masraflı olacağından bunları tek işlemciye yaptırabiliriz. Bu iş için aritmetik işlemci kullanılabilir.



X_i	Y_i	C_{in}	F_i	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full – Adder Toplama

$$F_i = X_i + Y_i + C_{in}$$

Şimdi buradan lojik işleme çevirmeye çalışalım.

$C_{in} = 0$ olduğunda $F_i = X_i + Y_i + 0 = X_i + Y_i$ değeri $F_i = X_i \oplus Y_i$ lojik işlemini verir mi?

$X_i \oplus Y_i = X_i \bar{Y}_i + \bar{X}_i Y_i$ (XOR işlemi) ifadesinde tablodan değer verip baktığımızda $C_{in} = 0$ için aritmetik işlemci ile lojik işlem değerleri eşit olur.

$S_2 = 1$ yapıldığında (lojik işlem seçildiğinde) $C_{in} = 0$ yapılabilirse ALU, XOR işlemi yapar.

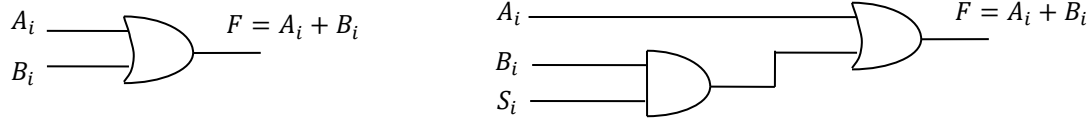
$C_{in} = 0$ durumunda aritmetik işlemci fonksiyon tablosu şöyledir (Sayfa 35' deki tablodan):

S_1	S_0	C_{in}	F
0	0	0	A
0	1	0	$A + B$
1	0	0	$A + \bar{B}$
1	1	0	$A - 1$

$S_2 S_1 S_0$	$X_i Y_i C_{in}$	$F = X_i + Y_i = X_i \oplus Y_i$	Lojik işlemler	Lojik işlemin adı
1 0 0	$A_i 0 0$	A_i	$A_i + B_i$	OR
1 0 1	$A_i B_i 0$	$A_i \oplus B_i$	$A_i \oplus B_i$	XOR
1 1 0	$A_i \bar{B}_i 0$	$A_i \odot B_i$	$A_i \cdot B_i$	AND
1 1 1	$A_i 1 0$	$\bar{A}_i$	$\bar{A}_i$	NOT

Aritmetik işlemcinin yaptığı XOR ve NOT işlemleri aynı kalırken A' 'nın transferi OR işlemine ve XNOR işlemi AND işlemine çevrilmelidir ki lojik işlemler yapılmış olsun.

A' nın transferinin OR işlemine çevrilmesi:



XNOR işleminin AND işlemine dönüştürülmesi

$S_2 S_1 S_0 = 110$ durumunda XNOR işlemi AND işlemi haline dönüşsün.

$S_2 S_1 S_0 = 110$ durumunda $X_i = A_i$, $Y_i = \bar{B}_i$ olduğu tablodan görülmektedir. XNOR işleminin AND' e dönüştürülmesi için bu değerler

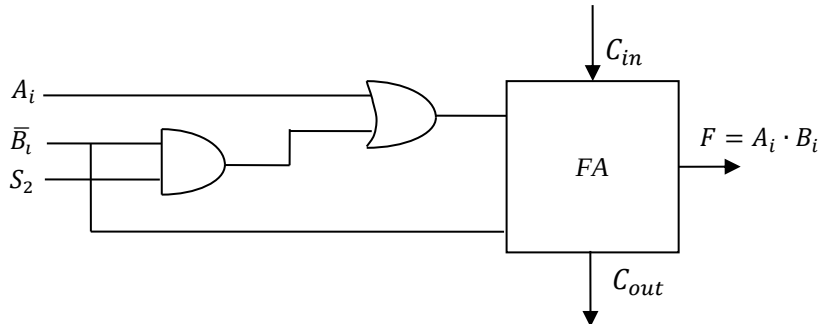
$F_i = X_i \oplus Y_i$ ifadesinden $F_i = A_i \cdot B_i$ değerinin elde edilmesi gerekmektedir.

Bu sebepten X_i ve Y_i değişkenleri F_i fonksiyonunda yerine yazılırken k_i değişkeni kullanılır.

$$F_i = (A_i + k_i) \oplus \bar{B}_i$$

$F_i = \bar{A}_i \bar{k}_i \bar{B}_i + A_i B_i + k_i B_i$ elde edilir. Bu ifadeden $F_i = A_i \cdot B_i$ ifadesine ulaşmak için ilk ve son lojik ifadelerin sıfırlanması gerekir. Bunun için $k_i = \bar{B}_i$ olarak seçilmelidir.

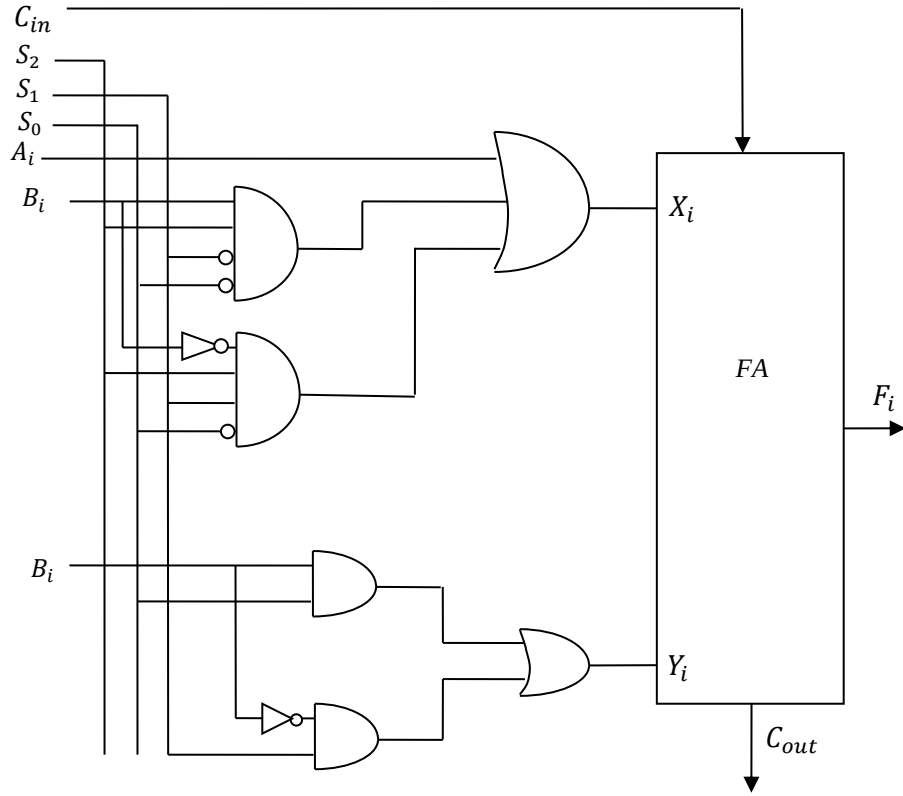
$F_i = A_i B_i + \bar{B}_i B_i + \bar{A}_i \bar{B}_i \bar{B}_i \Rightarrow F_i = A_i B_i$ olarak elde edilir, yani işlem AND' e dönüşür.



ALU' nun yaptığı işlemler

Seçme Girişleri $S_2 S_1 S_0 C_{in}$	Çıkış F_i	İşlem
0 0 0 0	A	A' nın transferi
0 0 0 1	$A + 1$	A' nın 1 fazlası
0 0 1 0	$A + B$	A ile B' nin toplanması
0 0 1 1	$A + B + 1$	A ile B' nin toplam sonucunun 1 fazlası
0 1 0 0	$A + \bar{B}$	A' nın $\bar{B}$ ile toplanması
0 1 0 1	$A + \bar{B} + 1 = A - B$	A ile B' nin farkının alınması
0 1 1 0	$A - 1$	A' nın 1 eksiği
0 1 1 1	A	A' nın transferi
1 0 0 0	OR	$A \cup B$
1 0 1 0	XOR	$A + B$
1 1 0 0	AND	$A \cap B$
1 1 1 0	NOT	$\bar{A}$

Böylece tasarlanan lojik ünite ile aritmetik ünite birleştirilirse Şekil 3.3' deki 1 bitlik ALU elde edilir.



Şekil 3.3. 1 bitlik ALU

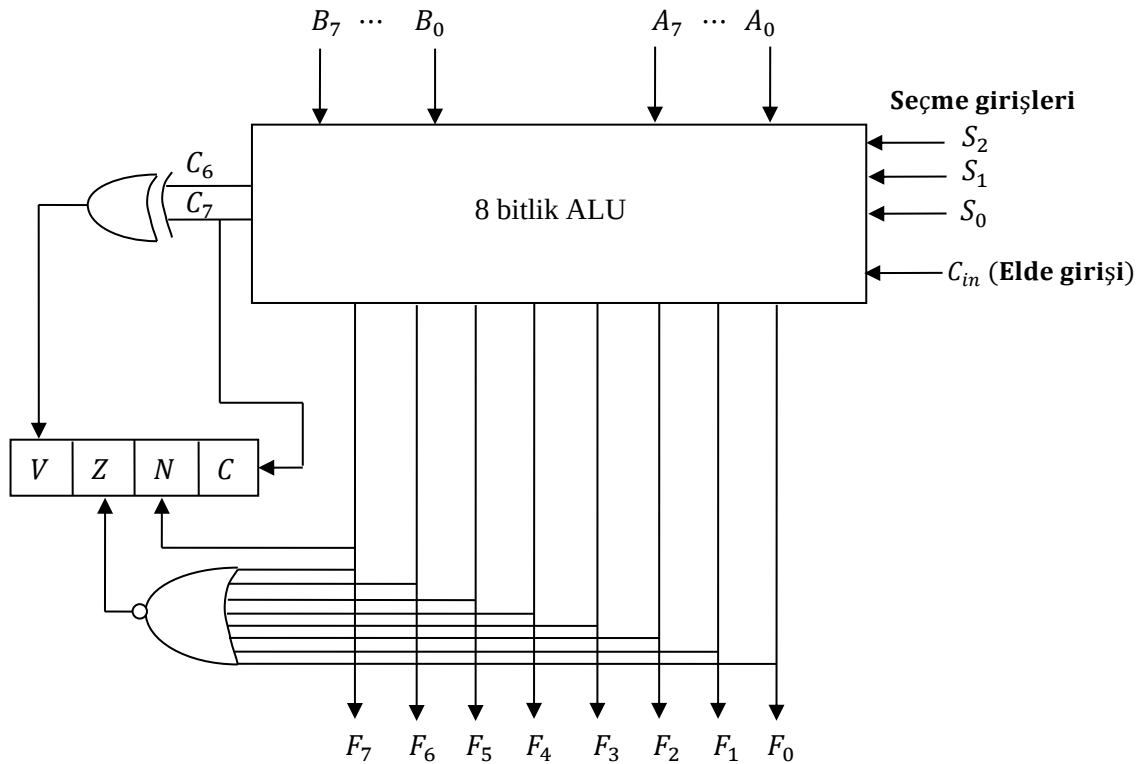
4. DURUM REGİSTERİ (Status Register)

ALU' da yapılan aritmetik işlemler hakkında bilgi verir. Örneğin elde var mı?, taşma var mı?, sonucu sıfır mı? gibi...

Bu işlemlerin 4 tanesini ele alacağız:

- Elde (Carry) C ile gösterilir.
- Sayı sıfır (Zero) Z ile gösterilir.
- Sayı negatif (Negative) N ile gösterilir.
- Taşma (Overflow) V ile gösterilir.

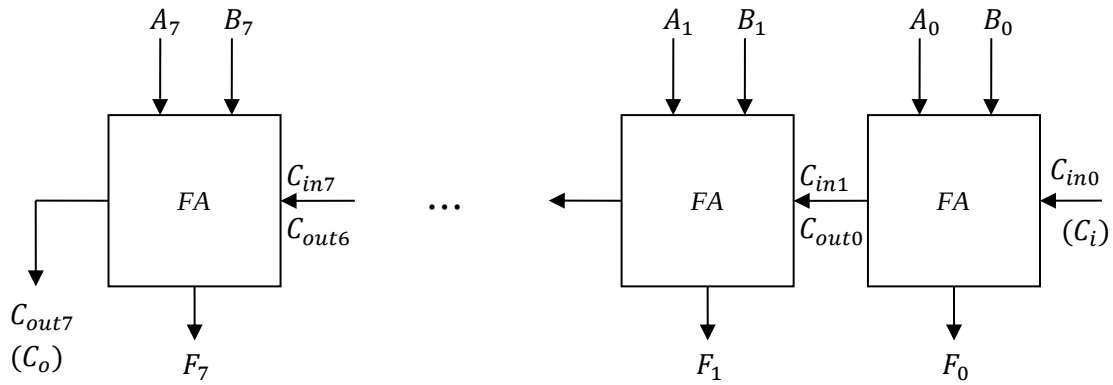
C, Z, N, V: Her biri 1 bit yani 1 FF' dur.



Şekil 4.1. 4 FF' li durum registeri

4 durumu birden gösteren register ise 4 FF' lu durum registeri olur.

Z: Zero	N: Negatif	C: Elde (Carry)	V: Taşma
Z = 1 ise sayı = 0	N = 1 ise sayı = neg	C = 1 ise elde var	V = 1 ise taşma var
Z = 0 ise sayı ≠ 0	N = 0 ise sayı ≠ neg	C = 0 ise elde yok	V = 0 ise taşma yok



Şekil 4.2. ALU' nun içinde tam toplayıcıların detaylı çizimi

- **N: İşaret biti' dir.** $N = 1$ ise işlem sonucu negatif, $N = 0$ ise işlem sonucu pozitifdir. $F_7 F_6 \dots F_0$: işlem sonucu (çıkış)' dur. F_7 : en anlamlı bit, $F_7 = 1$ ise sonuç negatif, $F_7 = 0$ ise sonuç pozitifdir. F_7 çıkışı N bitine bağlanır.
- **Aritmetik işlemde elde varsa $C = 1$, yoksa $C = 0$ olur.** Eldenin olup olmadığı en son elde biti ile belli olur. Onun için son tam toplayıcının elde çıkışı durum registerinin C bitine bağlanır.
- **Z (zero)—sıfır biti:** $Z = 1$ ise sayı sıfırdır, $Z = 0$ ise sayı sıfır değildir. Burada sayı işlem sonucudur. $Z = \overline{(F_0 + F_1 + \dots + F_7)} = \overline{F_0} \overline{F_1} \dots \overline{F_7}$ Buna göre çıkışlar NOR işleminden sonra Z bitine bağlanır.
- **V: Taşma bitidir.** Taşma biti işlem sonucunda sayının işaret değiştirdiğini gösterir.

Taşmanın olması için üç durumdan birinin oluşması gerekir:

- Aynı işaretli iki sayının toplamı farklı işaretli çıkıyorsa taşma vardır.
 - A ile B sayılarının işareti farklı ise bunların toplamları sonucunda hiçbir zaman taşma olmaz. Örneğin $n - \text{bitlik}$ bir sayıda değer (-2^{n-1}) ' den büyük $2^n - 1$ ' den küçük ise
 $n = 4$ bitlik ise sayı = 15 $-2^{n-1} = -2^3 = -8$
 $-8 \leq A, B \leq 15$ olur.
 $A \rightarrow 01101110$ A pozitif
 $+ \quad B \rightarrow 11100101$ B negatif

 111010011
 $C_7 C_6 C_5 C_4 C_3 C_2 C_1 C_0 = 11101100 \rightarrow C_i = C_{in} = 0 = A_0 + B_0$
 $C_7 \oplus C_6 = 0$ olduğu için taşma yoktur, $V = 0$
 (XOR' dan dolayı V girişi $C_7 \oplus C_6$ ' dir.)

b. İşaretili sayılarda toplama işlemi:

İki negatif sayının toplamı pozitif ise taşma vardır.

İki pozitif sayının toplamı negatif ise taşma vardır.

$$\begin{array}{r}
 A \rightarrow 11010110 \quad A \text{ negatif} \\
 + \quad B \rightarrow 10111001 \quad B \text{ negatif} \\
 \hline
 110001111 \quad \text{Sonuç negatif} \quad C_7C_6C_5C_4C_3C_2C_1C_0 = 11110000 \\
 \text{Elde} = C_7 = 1 \quad C_7 \oplus C_6 = 0 \quad \text{taşma yok} \quad V = 0
 \end{array}$$

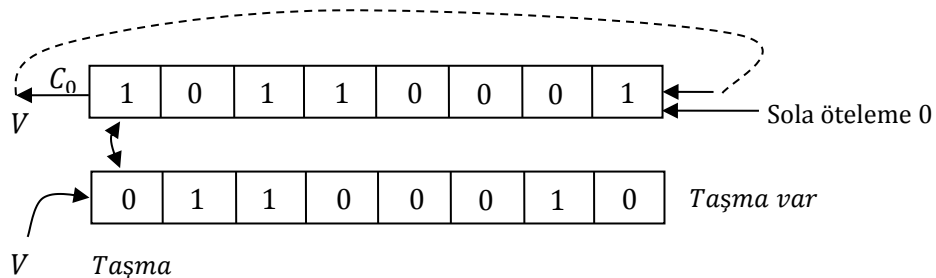
$$\begin{array}{r}
 A \rightarrow 01100111 \quad A \text{ pozitif} \\
 + \quad B \rightarrow 00111001 \quad B \text{ pozitif} \\
 \hline
 10100000 \quad \text{Sonuç negatif} \quad C_7C_6C_5C_4C_3C_2C_1C_0 = 01111111 \\
 \text{Elde} = C_7 = 0 \quad C_7 \oplus C_6 = 1 \quad \text{taşma var} \quad V = 1
 \end{array}$$

$$\begin{array}{r}
 A \rightarrow 00101011 \quad A \text{ pozitif} \\
 + \quad B \rightarrow 00100101 \quad B \text{ pozitif} \\
 \hline
 01010000 \quad \text{Sonuç pozitif} \quad C_7C_6C_5C_4C_3C_2C_1C_0 = 00101111 \\
 \text{Elde} = C_7 = 0 \quad C_7 \oplus C_6 = 0 \quad \text{taşma yok} \quad V = 0
 \end{array}$$

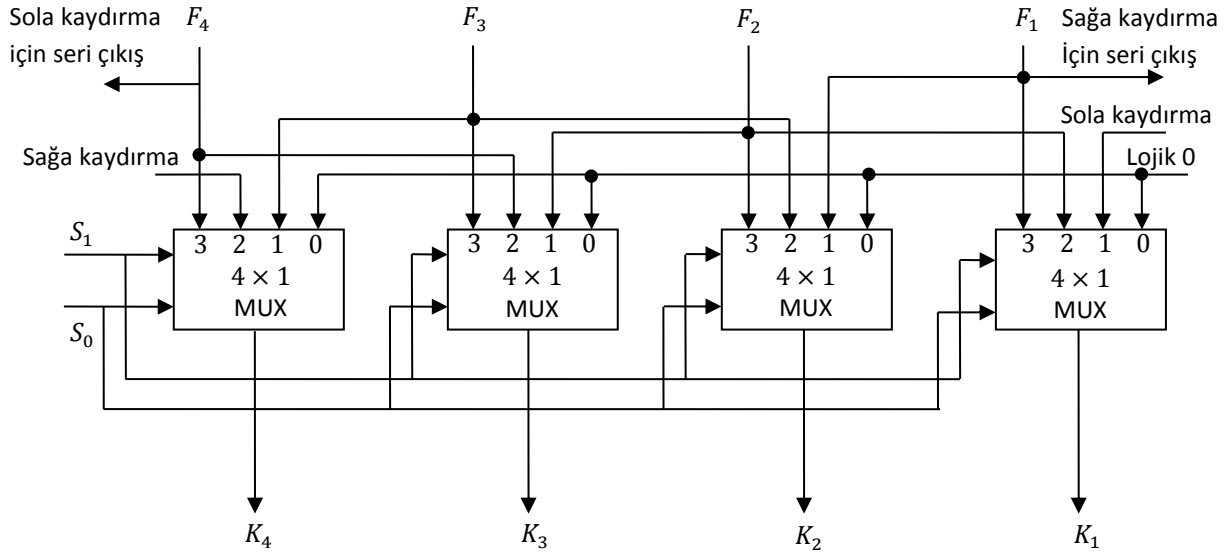
$$\begin{array}{r}
 A \rightarrow 10010000 \quad A \text{ negatif} \\
 + \quad B \rightarrow 11010100 \quad B \text{ negatif} \\
 \hline
 101100100 \quad \text{Sonuç pozitif} \quad C_7C_6C_5C_4C_3C_2C_1C_0 = 10010000 \\
 \text{Elde} = C_7 = 1 \quad C_7 \oplus C_6 = 1 \quad \text{taşma var} \quad V = 1
 \end{array}$$

C_7	C_6	V
0	0	$0 \rightarrow \text{Taşma yok}$
0	1	$1 \rightarrow \text{Taşma var}$
1	0	$1 \rightarrow \text{Taşma var}$
1	1	$0 \rightarrow \text{Taşma yok}$

- ii. $A - B$ işleminde $A > B$ olduğu halde sonuç B 'nin işareti ile aynıysa taşma vardır.
- iii. Sağa veya sola kaydırmada (ötelemede) sayı işaret değiştiriyorsa taşma vardır.

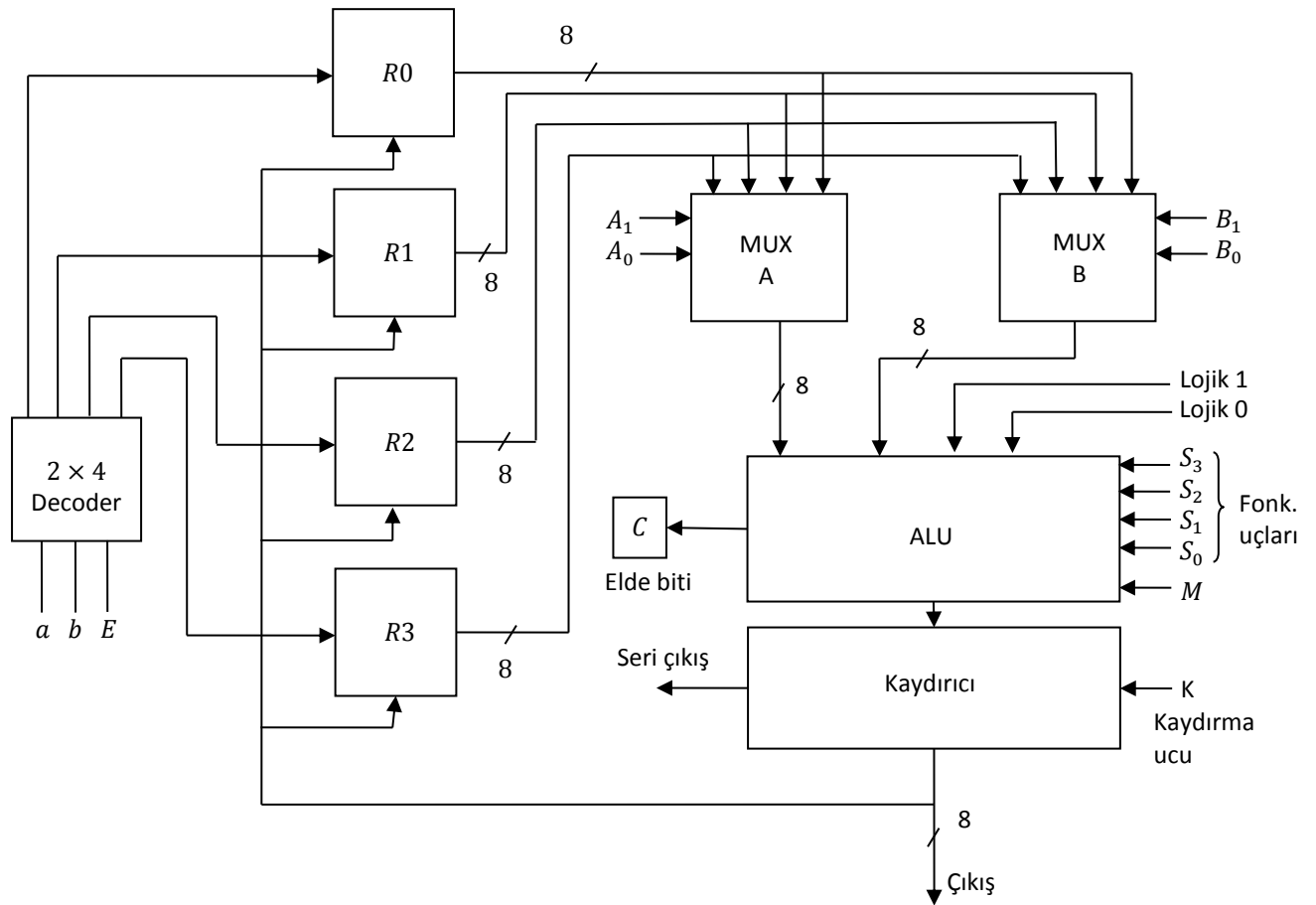


5. KAYDIRICI (ÖTELEYİCİLER-SHIFTER)



S_1	S_0	İşlem	İşlem açıklaması
0	0	$K \leftarrow 0$	K' yı Lojik 0 yap
0	1	$K \leftarrow \text{Sola kaydır} (F)$	F' nin sola kaydırılmış halini K' ya aktar
1	0	$K \leftarrow \text{Sağa kaydır} (F)$	F' nin sağa kaydırılmış halini K' ya aktar
1	1	$K \leftarrow F$	F' yi K' ya aktar

6. ALU İLE BUS ORGANİZASYONU



Kaydırma işlemi paraleli seriye dönüştürmek için kullanılır.

$M = 0$ ise aritmetik işlem yapılır.

$M = 1$ ise lojik işlem yapılır.

$K = 0$ ise kaydırma yok. $K = 1$ ise kaydırma var.

$C = 1$ ise elde var. $C = 0$ ise elde yok (Elde biti bir FF' dan yapılır).

Örnek 6.1. $R2 \leftarrow (R0 + R1)$ mikroişlemi için gerekli kontrol kelimesi nedir?

1. $P_1 = A_1A_0 = 00$ bu durumda $R0$, MUX A çıkışına aktarılır.
2. $P_2 = B_1B_0 = 01$ bu durumda $R1$, MUX B çıkışına aktarılır.
3. $P_3 = M = 0$ yani aritmetik işlem seçilir.
4. $P_4 = S_3S_2S_1S_0 = 0011$ toplama işleminin kontrol kelimesi
5. $P_5 = K = 0$ kaydırma yok
6. $P_6 = abE = 101$ $R2$ registerini etkin hale getirir.

Sonuçta bu mikroişlem için üretilmesi gereken kontrol kelimesi

P_6	P_5	P_4	P_3	P_2	P_1
101	0	0011	0	01	00

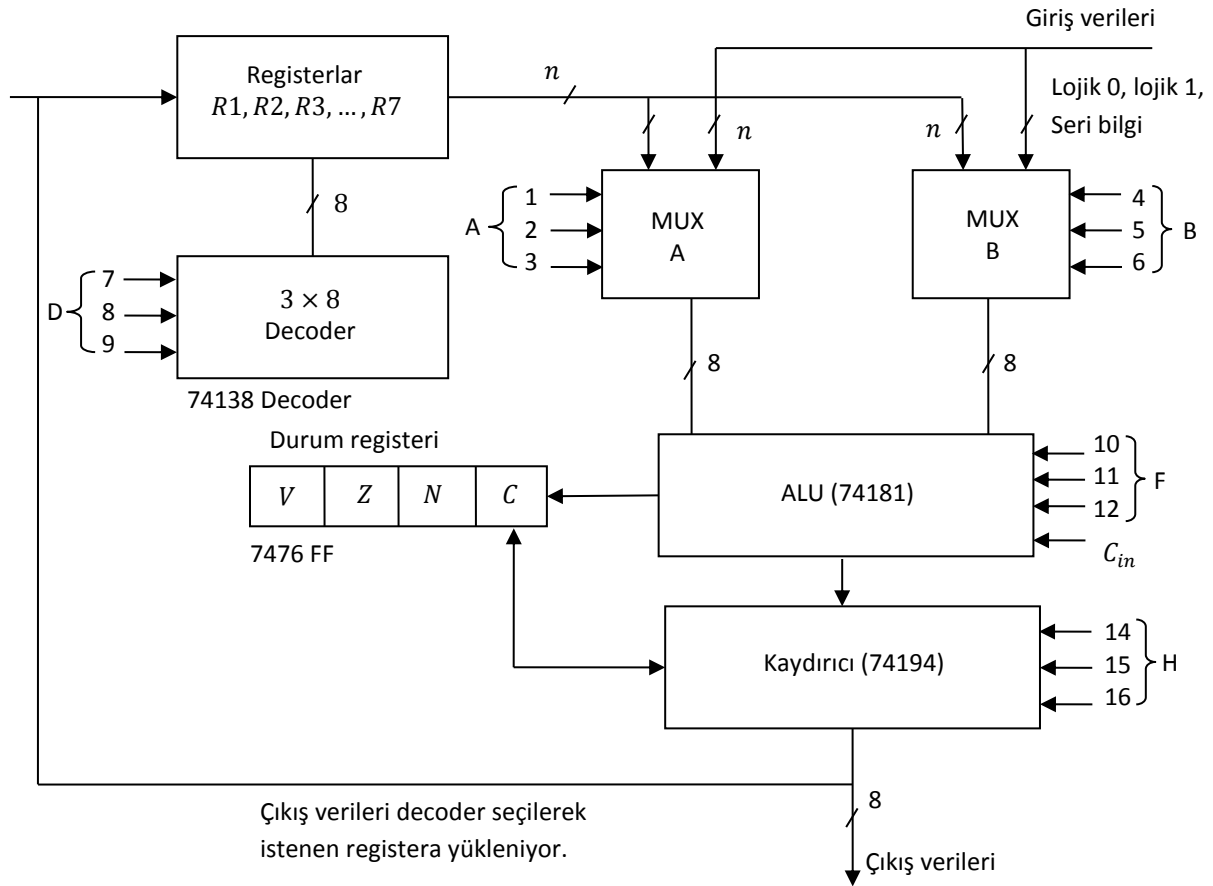
Örnek 6.2. $R2 \leftarrow (R0 - R1)$ mikroişlemi için gerekli kontrol kelimesi nedir?

1. $P_1 = A_1A_0 = 00$ bu durumda $R0$, MUX A çıkışına aktarılır.
2. $P_2 = B_1B_0 = 01$ bu durumda $R1$, MUX B çıkışına aktarılır.
3. $P_3 = M = 0$ yani aritmetik işlem seçilir.
4. $P_4 = S_3S_2S_1S_0 = 0110$ çıkarma işleminin kontrol kelimesi
5. $P_5 = K = 0$ kaydırma yok
6. $P_6 = abE = 101$ $R2$ registerini etkin hale getirir.

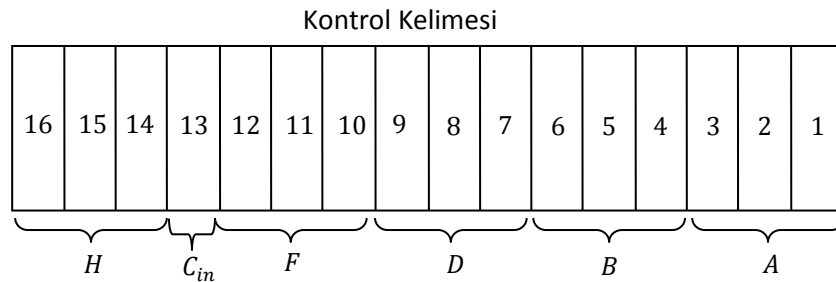
Sonuçta bu mikroişlem için üretilmesi gereken kontrol kelimesi

P_6	P_5	P_4	P_3	P_2	P_1
101	0	0110	0	01	00

7. İŞLEMÇİ ÜNİTESİ



A, B, D, H, F girişleri kontrol sinyalleri C ile sağa döndürme, C ile sola döndürme için kontrol sinyalleri vardır. Kontrol sinyalleri toplamına kontrol kelimesi (control word) denir.



Clock darbesi ile hepsine belli bir süre verilir ve bu sürede gerekli işlem yapılmış olur (Örneğin $R1 \leftarrow R3 + R4$ gibi).

İşlemci ünitesinin yaptığı işlemlere ait tablo şöyledir:

Seçme girişleri	Seçilen yol ve yapılan işlem					
$S_2 S_1 S_0$	A	B	D	$F (C_{in} = 0)$	$F (C_{in} = 1)$	H
0 0 0	Giriş	Giriş	Seçilmiyor	$A, C \leftarrow 0$	$A + 1$	Kaydırma yok
0 0 1	$R1$	$R1$	$R1$	$A + B$	$A + B + 1$	Sağa kaydırma
0 1 0	$R2$	$R2$	$R2$	$A - B - 1$	$A - B$	Sola kaydırma
0 1 1	$R3$	$R3$	$R3$	$A - 1$	$A, C \leftarrow 1$	Çıkışı sıfırla
1 0 0	$R4$	$R4$	$R4$	$A + B$ (OR)	-	-

1 0 1	R5	R5	R5	$A \oplus B$	-	C ile sağa döndürme
1 1 0	R6	R6	R6	$A \cdot B$ (AND)	-	C ile sola döndürme
1 1 1	R7	R7	R7	$\bar{A}$	-	-

A: MUX A' nin girişine gelen bilgi

B: MUX B' nin girişine gelen bilgi

D: 3 × 8 decoderin seçtiği registerlar

F: ALU' nun çıkışı

H: Kaydırıcının çıkışı

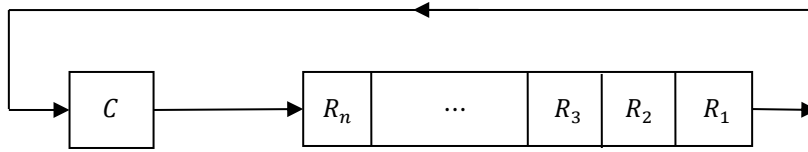
Seçme girişleri 001 ($S_2S_1S_0$) ise MUX A girişi= R1, MUX B girişi= R1 ve decoder çıkışı=R1 registerını seçer.

$C_{in} = 0$ ise ALU çıkışı $A + B$ olur. Kaydırıcı sağa kaydırma işlemi yapar.

$A, C \leftarrow 0$: Lojik 0 değeri MUX A girişine gelir ve aynı zamanda C biti 0 olur.

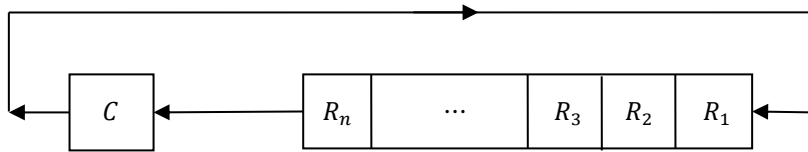
$A, C \leftarrow 1$: Lojik 1 değerini MUX A' nin girişine ve C' ye yükler.

C ile Sağa Döndürme (2 ile Bölme)



$R \leftarrow C$ ile sağa döndürme, $R_n \leftarrow C$, $C \leftarrow R_1$ olur

C ile Sola Döndürme (2 ile Çarpma)



$C \leftarrow R$ ile sola döndürme, $C \leftarrow R_n$, $C \leftarrow R_1$ olur

Mikroişlem	A	B	D	F	C_{in}	H	İşlem (Fonk)
$R2 \leftarrow R4 + R1$	100	001	010	001	0	000	R4 ile R1' i topla, R2' ye koy
$R3 \leftarrow R3 - R1$	011	001	011	010	1	000	R3' den R1' i çıkart, R3' e koy
$R7 \leftarrow R2$	111	010	000	010	1	000	R7 ve R2' i karşılaştır
$R5 \leftarrow \text{Giriş}$	000	000	101	000	0	000	Giriş verilerini R5' e koy
$R4 \leftarrow R1$	001	000	100	000	0	000	R1' i R4' e yükle.
$\text{Çıkış} \leftarrow R1$	001	000	000	000	0	000	R1' i çıkışa aktar
$R2 \leftarrow R2, C \leftarrow 0$	010	000	010	000	0	000	C' yi sıfırla
$R6 \leftarrow 0$	000	000	110	000	0	011	R6' yı sıfırla
$R6 \leftarrow R6 \oplus R6$	110	110	110	101	0	000	R6' yı sıfırla
$R1 \leftarrow \text{sola kaydır } R1$	001	000	001	000	0	010	R1' i sola kaydır
$R2 \leftarrow \text{sağa döndür } R2$	010	000	010	000	0	101	R2' yi sağa döndür

- Girişten bir registera, bir registerdan diğerine ve bir registerdan çıkışa bilgi aktarımında ALU' da ve kaydırıcıda işlem yapılmamaktadır.
- Bir registerın değeri tekrar kendisine aktarılarak durum registerındaki elde biti (C) sıfır yapılabilir.
- Kaydırıcıdaki sıfırlama fonksiyonu seçilerek bir registerın değeri sıfırlanabilir. Aynı işlem $R6 \leftarrow R6 \oplus R6$ şeklinde de yapılabilir.

Örnek 7.1. Aşağıdaki mikroişlemlerin yapılması için işlem biriminin kontrol kelimesinin değerini bulunuz.

- a) $R4 \leftarrow R7 + R3$ b) $R2 \leftarrow R4 - 1$ c) $R1 \leftarrow R1 + 1$ d) $R2 \leftarrow \overline{R2}$
e) $R2 \leftarrow R5$ f) $R3 \leftarrow R1 \vee R2$ (veya) g) $R3 \leftarrow \text{Sağa kaydır } R3$
h) $R6 \leftarrow C \text{ ile Sola döndür } R6$

Mikroişlem	A	B	D	F	C_{in}	H
$R4 \leftarrow R7 + R3$	111	011	100	001	0	000
$R2 \leftarrow R4 - 1$	100	000	010	011	0	000
$R1 \leftarrow R1 + 1$	001	000	001	000	1	000
$R2 \leftarrow \overline{R2}$	010	000	010	111	0	000
$R2 \leftarrow R5$	101	000	010	000	0	000
$R3 \leftarrow R1 \vee R2$	001	010	011	101	0	000
$R3 \leftarrow \text{Sağa kaydır } R3$	011	000	011	000	0	001
$R6 \leftarrow C \text{ ile Sola döndür } R6$	110	000	110	000	0	110

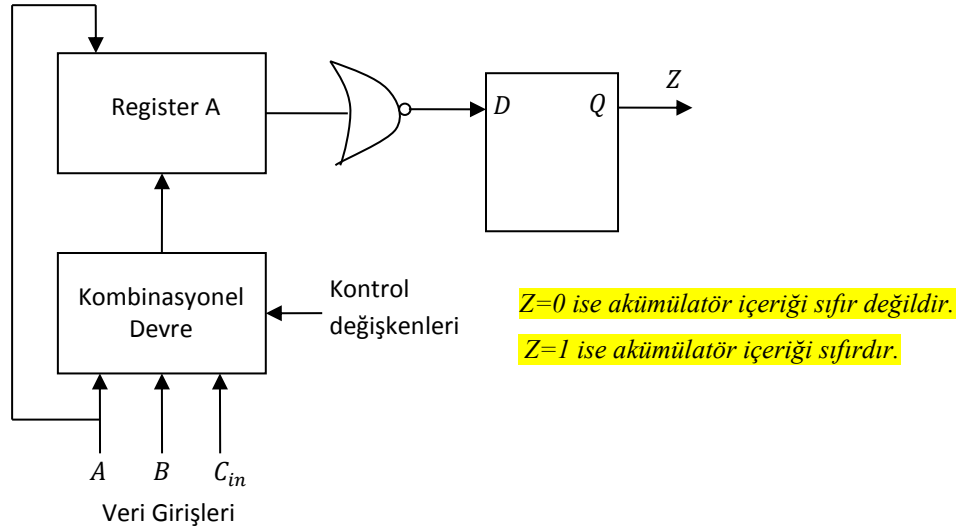
Örnek 7.2. Aynı işlem birimini kullanarak $R1$, $R2$, $R3$ ve $R4$ registerlerindeki işaretsiz sayıların ortalamasını bulacak ve sonucu R_5 registerina koyacak bir mikroprogram düzenleyiniz.

$$(R1 + R2 + R3 + R4)/4 = ((R1 + R2)/2 + (R3 + R4)/2)/2$$

Mikroişlem	A	B	D	F	C_{in}	H
$R5 \leftarrow R1 + R2$	001	010	101	001	0	000
$R5 \leftarrow R5$	101	000	101	000	0	000
$R5 \leftarrow \text{Sağa döndür } R5$	101	000	101	000	0	101
$R6 \leftarrow R3 + R4$	011	100	110	001	0	000
$R6 \leftarrow R6$	110	000	110	000	0	000
$R6 \leftarrow \text{Sağa döndür } R6$	110	000	110	000	0	101
$R5 \leftarrow R5 + R6$	101	110	101	001	0	000
$R5 \leftarrow R5$	101	000	101	000	0	000
$R5 \leftarrow \text{Sağa döndür } R5$	101	000	101	000	0	101

8. AKÜMÜLATÖR REGİSTERİN YAPISI

Akümülatör register içindeki değeri sağa ve sola kaydırabilen, aritmetik mantık biriminin yapabildiği toplama ve mantık işlemlerini yapabilen bir ardışıl devredir. Aşağıdaki şekilde akümülatör registerin blok şeması görülmektedir.



Şekil 8.1. Akümülatör register blok şeması

Örnek bir akümülatör register kontrol biriminden gelen kontrol değişkenlerine göre aşağıdaki tabloda görülen mikroişlemi yapmaktadır.

Çizelge 8.1. Örnek akümülatör mikroişlem tablosu

Kontrol değişkeni	Mikroişlem	Açıklama
P_1	$A \leftarrow A + B$	Toplama
P_2	$A \leftarrow 0$	Sıfırlama
P_3	$A \leftarrow \bar{A}$	A' nın 1' e tümleyenini alma
P_4	$A \leftarrow A \cdot B (A \cap B)$	AND işlemi (VE)
P_5	$A \leftarrow A + B (A \cup B)$	OR işlemi (VEYA)
P_6	$A \leftarrow A \oplus B$	XOR işlemi
P_7	$A \leftarrow \text{Sağa kaydır } A$	Sağa kaydırma
P_8	$A \leftarrow \text{Sola kaydır } A$	Sola kaydırma
P_9	$A \leftarrow A + 1$	A' yı 1 artırma

a) Akümülatörde yapılan toplama işlemini gerçekleyelim: $P_1: A \leftarrow A + B$

Şimdiki Durum	Girişler		Gelecek Durum	FF Uyarma Girişleri		Çıkış
A_i	B_i	C_{in}	A_i	J_{A_i}	K_{A_i}	C_{in+1}
0	0	0	0	0	X	0
0	0	1	1	1	X	0
0	1	0	1	1	X	0
0	1	1	0	0	X	1
1	0	0	1	X	0	0
1	0	1	0	X	1	1
1	1	0	0	X	1	1
1	1	1	1	X	0	1

Karnough' a aktarılması

$B_i C_{in} \backslash A_i$	00	01	11	10
0	0	1	0	1
1	X	X	X	X

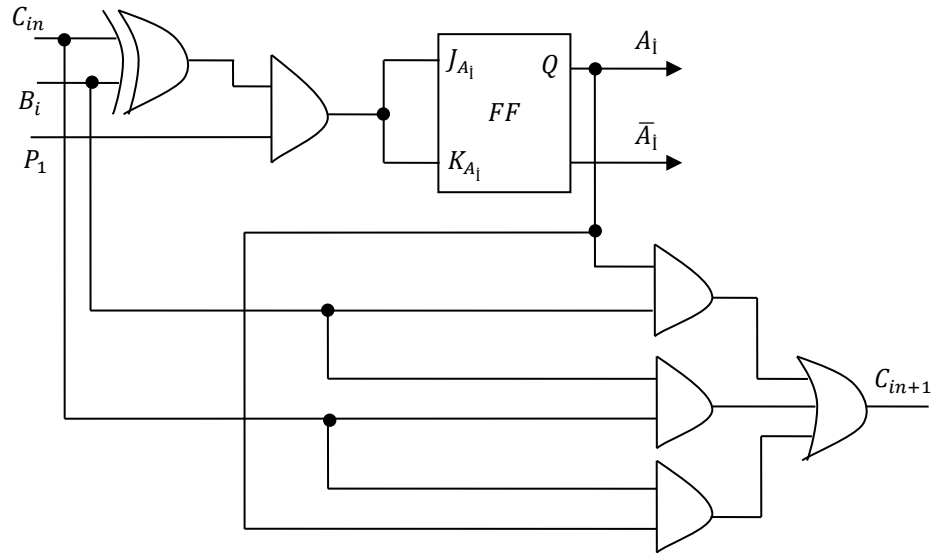
$$J_{A_i} = [B_i \overline{C_{in}} + \overline{B_i} C_{in}] \cdot P_1$$

$B_i C_{in} \backslash A_i$	00	01	11	10
0	X	X	X	X
1	0	1	0	1

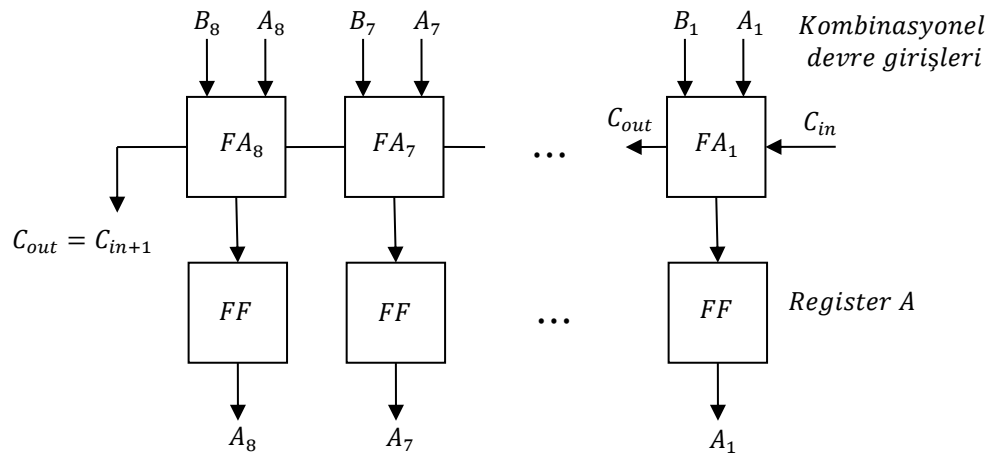
$$K_{A_i} = [B_i \overline{C_{in}} + \overline{B_i} C_{in}] \cdot P_1$$

$B_i C_{in} \backslash A_i$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$C_{in+1} = A_i B_i + A_i C_{in} + B_i C_{in}$$



Şekil 8.2. 1 bitlik kombinasyonel devre (1 bit $A_i \leftarrow A_i + B_i$)

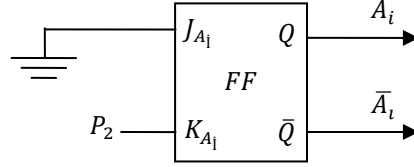


Şekil 8.3. P_1 mikroişlemini gerçekleştiren devreye ait blok diyagramı

b) Akümülatörde yapılan sıfırlama (silme) işlemini gerçekleyelim: $P_2: A \leftarrow 0$

$P_2 = 1$ olduğunda bütün FF' lar sıfırlanacaktır (bütün FF içerikleri silinecektir).

i. bit için $J_{A_i} = 0$ ve $K_{A_i} = 1 \cdot P_2 = P_2$ olduğunda $J_{A_i}K_{A_i} = 01$ olduğu için $A_i = 0$ olur.

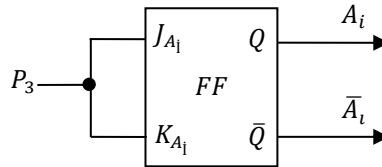


Şekil 8.4. P_2 mikroişlemini gerçekleştiren devre

c) Akümülatörde yapılan 1' e tümlleme işlemini gerçekleyelim: $P_3: A \leftarrow \bar{A}$

$P_3 = 1$ olduğunda A 'nın inversi alınır.

O halde $J_{A_i}K_{A_i} = 11$ olması gerekir. Bunun için $J_{A_i} = 1 \cdot P_3 = P_3$ ve $K_{A_i} = 1 \cdot P_3 = P_3$ olmalıdır ($J_{A_i}K_{A_i} = 11$ ise $Q^+ = Q^-$ olur).



Şekil 8.5. P_3 mikroişlemini gerçekleştiren devre

d) Akümülatörde yapılan VE işlemini gerçekleyelim: $P_4: A \leftarrow A \cdot B$

A_i	B_i	A_i^+	J_{A_i}	K_{A_i}
0	0	0	0	X
0	1	0	0	X
1	0	0	X	1
1	1	1	X	0

Tablodan hareketle $J_{A_i} = 0$ $K_{A_i} = \bar{B}_i P_4$ elde edilir.

e) Akümülatörde yapılan VEYA işlemini gerçekleyelim: $P_5: A \leftarrow A + B$

A_i	B_i	A_i^+	J_{A_i}	K_{A_i}
0	0	0	0	X
0	1	1	1	X
1	0	1	X	0
1	1	1	X	0

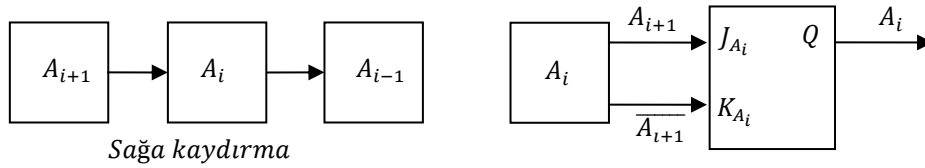
Tablodan hareketle $J_{A_i} = B_i P_5$ $K_{A_i} = 0$ elde edilir.

f) Akümülatörde yapılan XOR işlemini gerçekleyelim: $P_6: A \leftarrow A \oplus B$

A_i	B_i	A_i^+	J_{A_i}	K_{A_i}
0	0	0	0	X
0	1	1	1	X
1	0	1	X	0
1	1	0	X	1

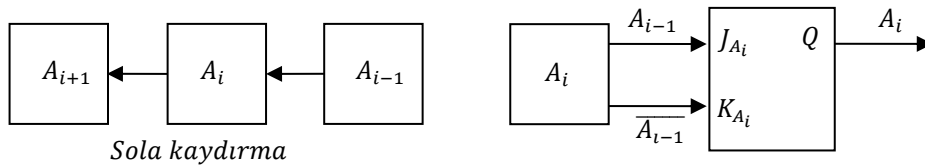
Tablodan hareketle $J_{A_i} = B_i P_6$ $K_{A_i} = B_i P_6$ elde edilir.

g) Akümülatörde yapılan sağa kaydırma işlemini gerçekleyelim: $P_7: A \leftarrow \text{Sağa kaydır } A$



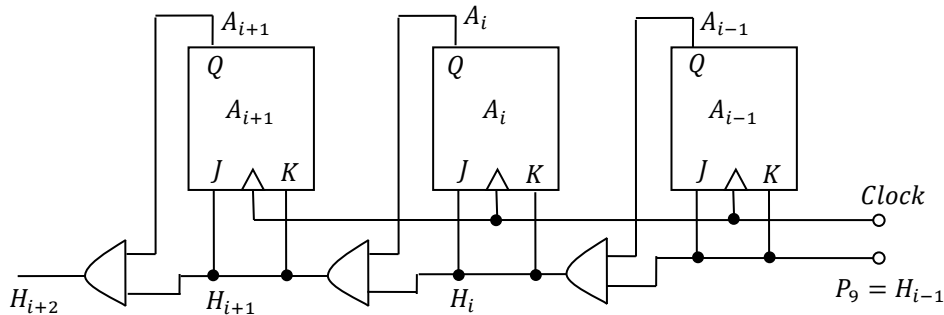
Sağa kaydırmada FF' un gelecek değeri soldaki FF' un şimdiki değeri olduğundan söz konusu mikroişlem için FF uyarma girişleri $J_{A_i} = A_{i+1} P_7$ ve $K_{A_i} = \overline{A_{i+1}} P_7$ olacaktır.

h) Akümülatörde yapılan sola kaydırma işlemini gerçekleyelim: $P_8: A \leftarrow \text{Sola kaydır } A$



Sola kaydırmada FF' un gelecek değeri sağdaki FF' un şimdiki değeri olduğundan söz konusu mikroişlem için FF uyarma girişleri $J_{A_i} = A_{i-1} P_8$ ve $K_{A_i} = \overline{A_{i-1}} P_8$ olacaktır.

i) Akümülatörde yapılan '1 artırma' işlemini gerçekleyelim: $P_9: A \leftarrow A + 1$

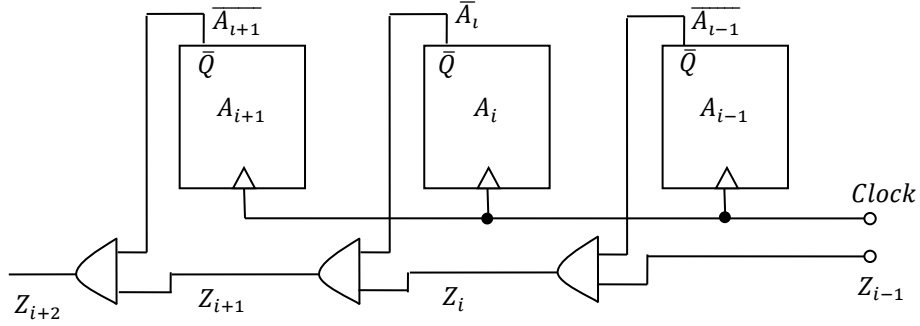


Bu durumda A registeri senkron bir sayıcı gibi tasarlanabilir.

$$J_{A_i} = H_i, \quad K_{A_i} = H_i, \quad H_{i+1} = A_i H_i, \quad H_{i-1} = P_9$$

8.1. Akümülatör registerin (A) sıfır olup olmadığının denetlenmesi

Akümlatör registerdaki sayı 0 ise $Q = 0$ $\bar{Q} = 1$ ' dir. $A = 0$ ise $Z = 1$ ' dir.



8.2. Bir Bitlik Akümülatör Fonksiyonu

Akümlatörün bir bitlik yapısı, her bir kontrol girişi için bulunan devreler birleştirilerek elde edilebilir. Bu durumda;

$$J_{A_i} = B_i \bar{C}_{in} P_1 + \bar{B}_i C_{in} P_1 + P_3 + B_i P_5 + B_i P_6 + A_{i+1} P_7 + A_{i-1} P_8 + H_i \quad H_i = A_{i-1} P_9$$

$$K_{A_i} = B_i \bar{C}_{in} P_1 + \bar{B}_i C_{in} P_1 + P_2 + P_3 + \bar{B}_i P_4 + \bar{B}_i P_6 + \bar{A}_{i+1} P_7 + \bar{A}_{i-1} P_8 + H_i \quad H_i = A_{i-1} P_9$$

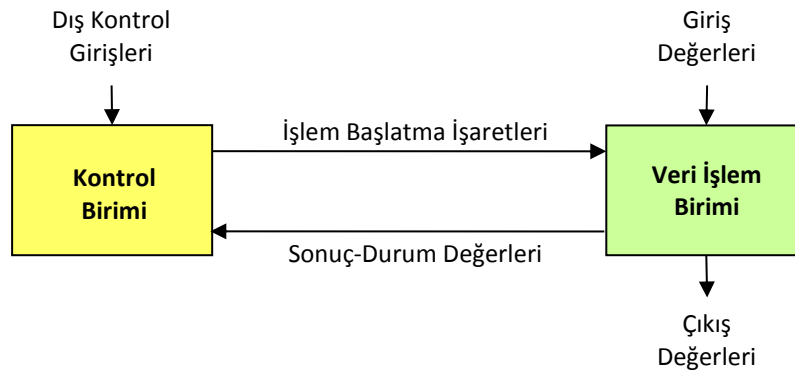
$$C_{in+1} = A_i B_i + A_i C_{in} + B_i C_{in}$$

$$H_{i+1} = H_i \cdot A_i$$

$$Z_{i+1} = Z_i \cdot \bar{A}_i$$

9. KONTROL BİRİMLERİ

Sayısal sistemler, veri işlem (işlemci ünitesi) ve kontrol birimlerinden oluşmaktadır. Veri işlem biriminde, sisteme gelen veriler okunmakta, işlenmekte ve sonuçlar bulunmaktadır. Kontrol biriminde ise veri işlem birimindeki devrelerin aç-kapa (enable) girişlerine uygulanan değerler (işlem başlatma işaretleri) bulunmakta ve böylece yapılacak işlemlerin sırası belirlenmektedir (Şekil 9.1). Bu amaçla kontrol biriminde durumlar saklanmaktadır. Önceden belirlenen algoritmaya göre bir sonraki durum değerleri bulunmaktadır. Bir sonraki durum değeri, o andaki durum değeri ile, dış kontrol girişlerine ve veri işlem biriminden gelen sonuç-durum değerlerine bağlı olarak belirlenmektedir.



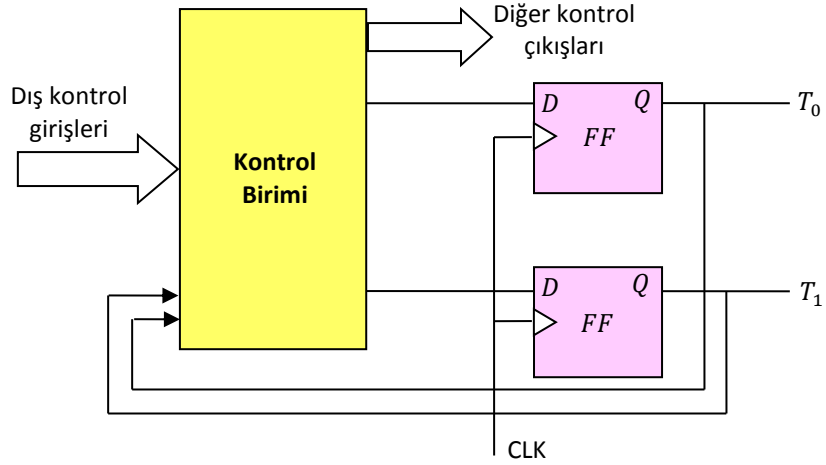
Şekil 9.1. Bir sayısal sistemde kontrol ve veri-işlem birimleri arasındaki bağıntılar

9.1. Kontrol Birimlerinin Yapıları

Sistemde uygulanan algoritmanın büyüklüğüne ve karmaşıklığına göre kontrol biriminin gerçekleştirilmesinde değişik yöntemler uygulanmaktadır. Bu yöntemler başlıca dörde ayrılmaktadır.

1. Her bir durum için bir FF kullanılması

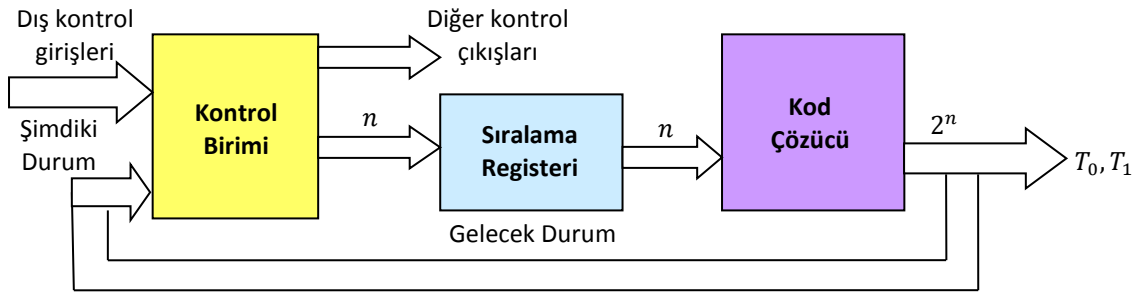
Kontrol biriminin durum diyagramı bulunduktan sonra, her bir durum için bir FF kullanılmakta ve FF çıkışları durumu belirlemektedir. Bu nedenle bir anda yalnız bir FF' un çıkışı 1 olmaktadır (Şekil 9.2). Bu tür kontrol birimleri, karmaşıklığı az olan sistemlerde kullanılmaktadır.



Şekil 9.2. Kontrol biriminde her bir durum için bir FF kullanılması

2. Sıra registeri ve kod çözücü kullanılması

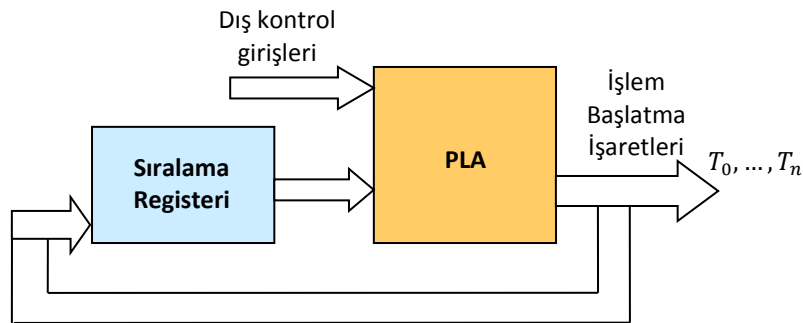
Pou yönteminde FF' lar yerine bir register ve kod çözücü kullanılmaktadır (Şekil 9.3). Bu yöntem biraz daha karmaşık sistemlerde uygulanmaktadır.



Şekil 9.3. Kontrol biriminde sıralama registeri ve decoder kullanılması

3. Programlanabilir Lojik Dizi (PLA-Programmable Logic Array) kullanılması

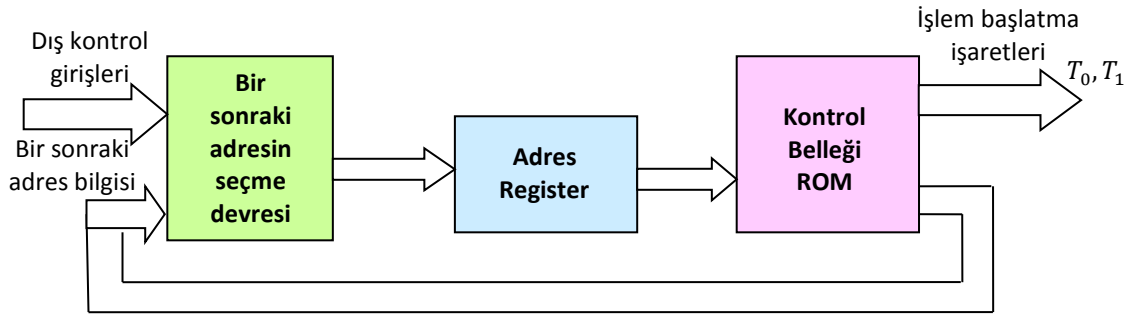
Yukarıdaki yöntemde kullanılan kontrol birimi ve kod çözücünün yerini programlanabilir lojik dizi almaktadır (Şekil 9.4). Bu yöntem genellikle karmaşık sayısal sistemlerde uygulanmaktadır.



Şekil 9.4. Kontrol biriminde PLA kullanılması

4. Mikroprogram kontrol

Bu yöntem daha karmaşık sistemlerde kullanılmaktadır. Burada ROM kullanılmakta ve bir sonraki adres bilgisi ile devre birimlerinin işlem başlatma işaretleri bellekte saklanmaktadır (Şekil 9.5). Devrenin izleyeceği durumlar ve bu durumlarda ve bu durumlarda yapılacak işlemler, bellekteki kelimelerde sırasıyla saklanmaktadır. Bu nedenle belleğin programlanması (içine gerekli bilginin saklanması) söz konusudur. Bu yöntemin kullanılmasının yararı, kontrol devresinin ve içindeki bağlantılarının değiştirilmeden, belleğin yeniden programlanarak yapılacak işlemleri sırasının değiştirilmesidir.



Şekil 9.5. Mikroprogram kontrol birimi

Daha büyük sistemlerin gerçekleştirilmesinde mikroişlemciler kullanılmaktadır.

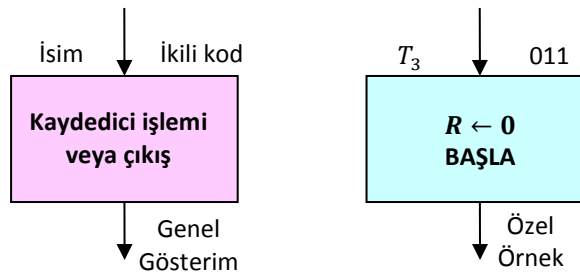
10. ALGORİTMİK DURUM MAKİNALARI

Algoritmik Durum Makinası (Algorithmic State Machine-ASM) ardışıl devrelere verilen ikinci bir isimdir. Sayısal bir sistemin kontrol sırası ve veri işleme görevleri bir donanım algoritmasıyla tanımlanır. Algoritma bir sorunun nasıl çözüleceğini belirten sonlu sayıda işlem basamağından oluşur. Donanım algoritması belli bir cihaz parçasıyla problemi uygulamak için kullanılan bir işlemdir.

Sayısal donanım algoritmalarını tanımlamak için özel olarak geliştirilen akış şemasına ASM şeması denir. ASM şeması klasik akış şemalarına benzer, ancak farklı yorumlanır. Klasik akış şemasında bir algoritmanın işlem basamakları ve karar yolları, zaman ilişkileri dikkate alınmadan tanımlanır. ASM şeması ise hem olayların sırasını hem de sıralı kontrol devresinin durumlarıyla bir durumdan ötekine geçilirken gerçekleşen olaylar arasındaki zamanlama ilişkisini tanımlar.

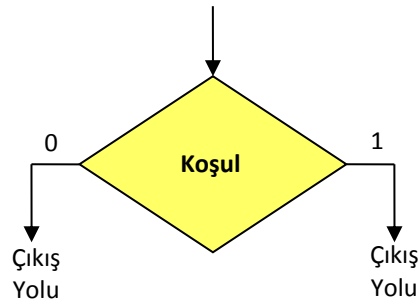
10.1. ASM Şeması

Şema üç temel elemandan oluşur: Durum kutusu, Karar kutusu ve Koşul kutusu. Kontrol sırasındaki bir durum Şekil 10.1’ deki gibi bir durum kutusu ile gösterilir. Kutu içine kaydedici işlemleri ve ilgili durumdayken kontrol devresinin ürettiği çıkış sinyal isimleri yazılan bir dikdörtgen şeklindedir. Sembolik bir adla gösterilen durum, kutunun üst sol köşesine yazılır. Duruma verilen ikili kod ise üst sağ köşeye yazılır (Şekil 10.1).



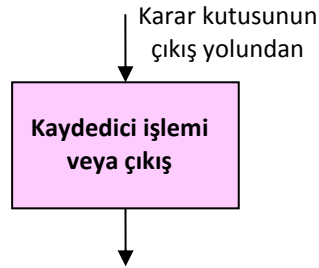
Şekil 10.1. Durum kutusu

Karar kutusu bir girişin kontrol alt sistemi üzerindeki etkisini tanımlar (Şekil 10.2).



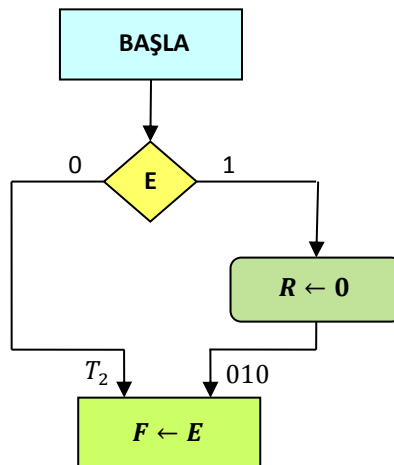
Şekil 10.2. Karar kutusu

Koşul kutusu ise Şekil 10.3’ de görülmektedir. Koşul kutusunun giriş yolunun, karar kutusunun çıkış yollarından birisinden gelmesi gerekir. Koşul kutusunun içinde verilen kaydedici işlemleri veya çıkışlar, giriş koşullarının yerine getirilmesi koşuluyla, belli bir durum sırasında üretilir.



Şekil 10.3. Koşul kutusu

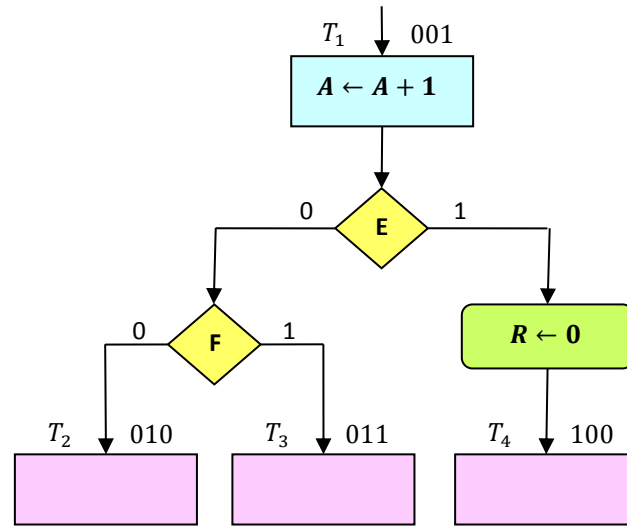
Şekil 10.4’ de koşul kutulu bir örnek verilmiştir. Kontrol devresi, T_1 durumundayken bir BAŞLAT çıkış sinyali üretir. Kontrol T_1 durumundayken E girişinin statüsünü kontrol eder. $E = 1$ ise R silinir (0 yapılır); değilse R aynen kalır. Her iki durumda da sonraki durum T_2 ’ dir.



Şekil 10.4. Koşul kutulu örnek

10.2. ASM Bloğu

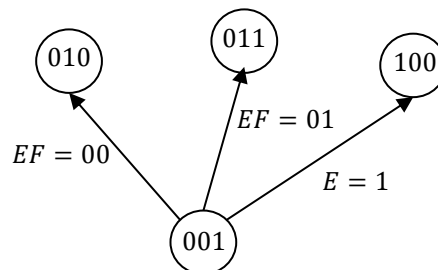
ASM bloğu, bir durum kutusundan ve çıkış yoluna bağlı bütün karar ve koşul kutularından oluşan bir yapıdır. Bir ASM bloğu, bir girişe ve karar kutularının yapısıyla temsil edilen herhangi bir sayıdaki çıkış yoluna sahiptir. ASM şeması, bir veya birbirine bağlı birden fazla bloktan oluşur. Şekil 10.5’ de ASM bloğuna bir örnek verilmiştir.



Şekil 10.5. ASM bloğu

Karar veya koşul kutuları olmayan bir durum kutusu basit bir blok oluşturur. ASM şemasındaki her blok, bir saat darbesi aralığı içindeki sistem durumunu tanımlar. Şekil 10.5’ deki durum ve koşul kutuları içindeki işlemler, sistem T_1 durumundayken ortak bir saat darbesiyle yürütülür. Aynı saat darbesi ayrıca sistem kontrol devresini E ve F ikili değerleriyle belirlendiği şekilde T_2 , T_3 veya T_4 sonraki durumlarından birine anahtarlar.

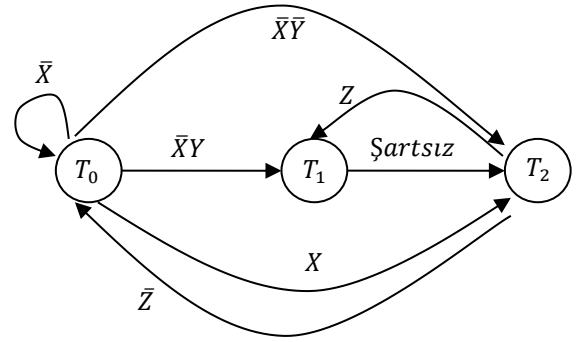
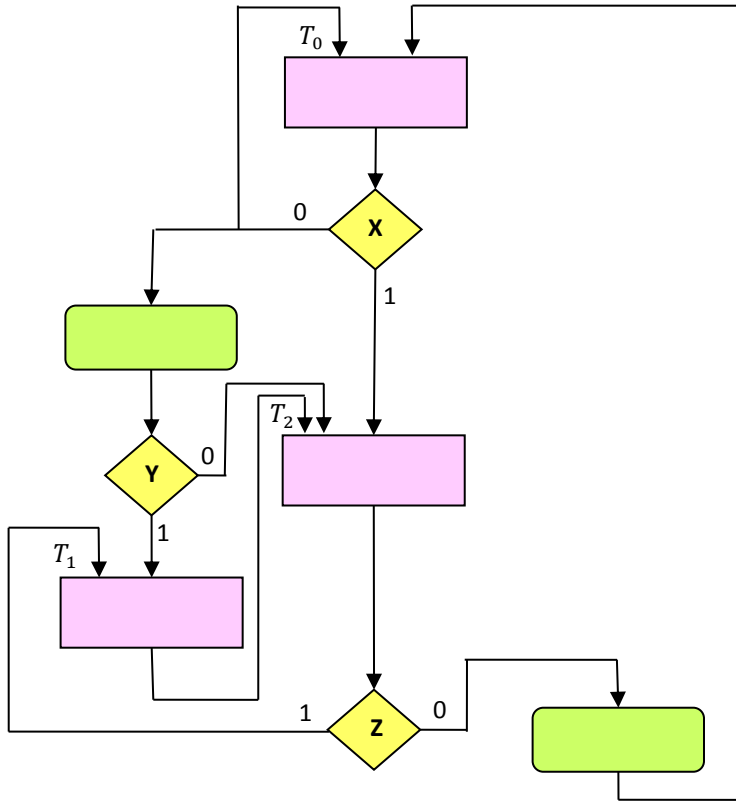
ASM şemasının durum diyagramı şeklinde gösterimi Şekil 10.6’ da verilmiştir.



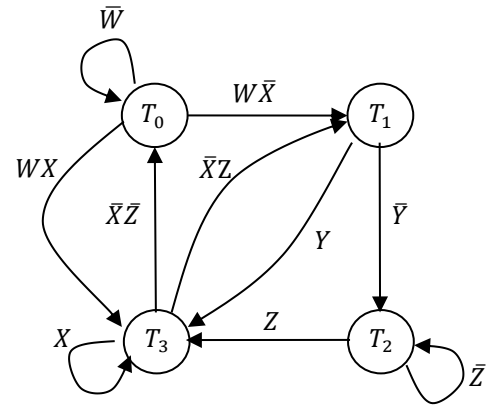
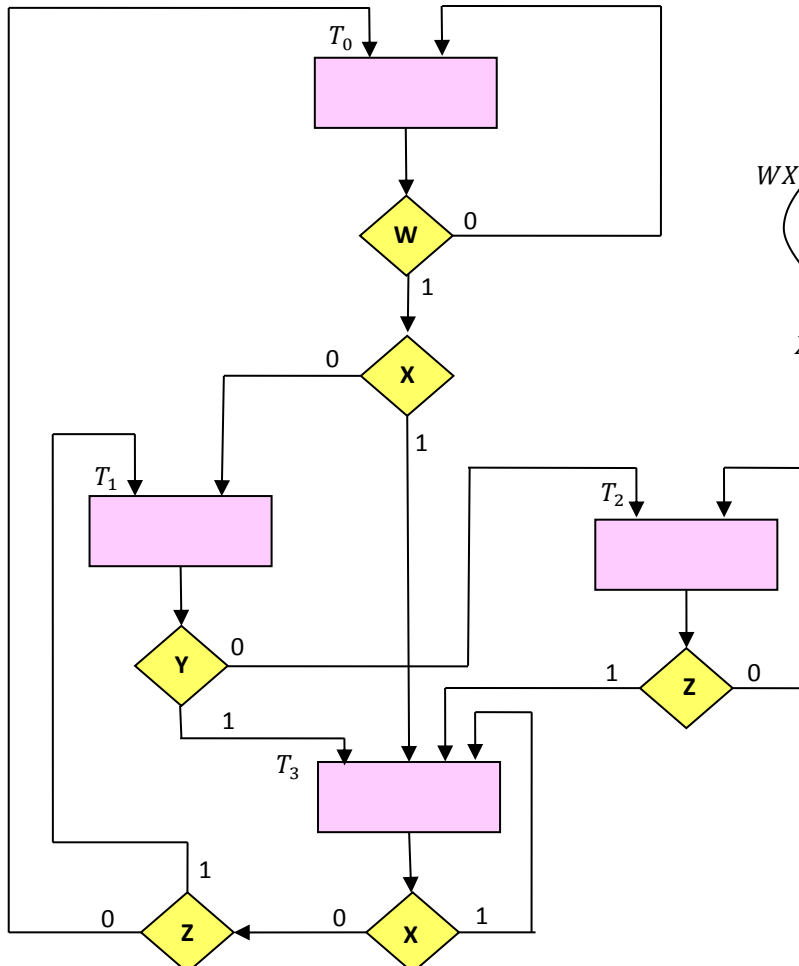
Şekil 10.6. ASM şemasının eşdeğeri olan durum diyagramı (şeması) ile gösterimi

Kontrol birimi tasarlamak için bazen ASM şemasını durum diyagramına çevirip daha sonra da sıralı devre işlemlerini kullanmak daha uygun olmaktadır.

Örnek 10.1. Aşağıda ASM şeması verilmiş devrenin durum diyagramını çıkarınız.



Örnek 10.2. Aşağıda ASM şeması verilmiş devrenin durum diyagramını çıkarınız.



Örnek 10.3. İçinde iki adet flip-flop (E,F) ve bir adet 4 bitlik sayıcı (A)'nın bulunduğu bir devrenin tasarlanması istenmektedir (A sayısının en anlamlı biti A_4 'dür). Başla (S) işareti 1 olduğunda, devre A sayıcısını ve F flip-flobunu sıfırlayarak T_0 durumundan T_1 durumuna geçecek ve çalışmaya devam edecektir. Daha sonra işlemler durduruluncaya kadar, her saat darbesinde sayıcı 1 artırılabacaktır. Sayıcının A_3 ve A_4 bitlerine bağlı olarak işlemler şu şekilde denetlenecektir.

- Eğer $A_3 = 0$ ise E sıfırlanacak ve sayıcı devam edecektir.
- Eğer $A_3 = 1$ ise E birlenecek, sonra eğer $A_4 = 0$ ise sayıcı devam edecek, fakat $A_4 = 1$ ise T_2 durumuna geçerek F birlenecek ve sayım duracaktır.

a) Bu devrenin ASM diyagramını (şemasını) çıkarınız.

b) Devrenin kontrol biriminin durum diyagramını ve (kontrol işaretlerine bağlı olarak) yapılan fonksiyonları gösteriniz.

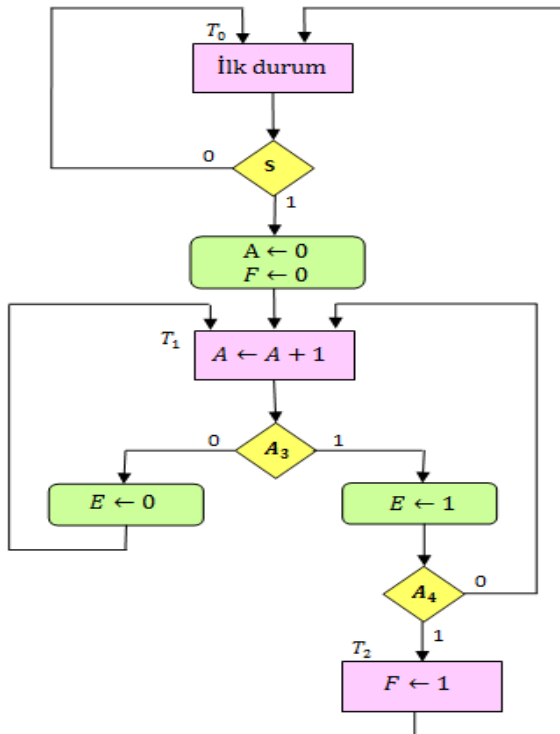
c) Devrenin işlem biriminin yapısını gösteriniz (E ve F flip-floplarının JK türü olduğunu varsayınız ve JK giriş fonksiyonlarını bulunuz).

d) Devrenin kontrol birimini her bir durum için bir flip-flop kullanarak tasarlayınız (D türü flip-flop kullanınız).

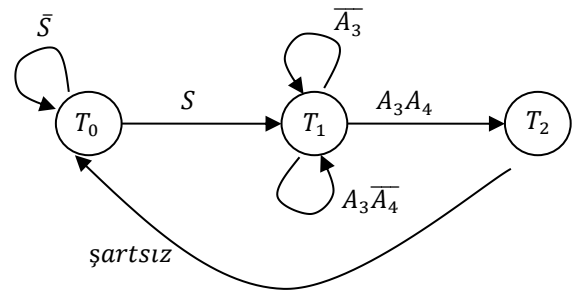
e) Devrenin kontrol birimini D türü flip-flop ve kod çözücü kullanarak tasarlayınız.

f) e şikkında kullandığınız D türü flip-flopların girişlerini MUX' lar kullanarak bulunuz.

a)



b)



$T_0S: A \leftarrow 0$

$T_0S: F \leftarrow 0$

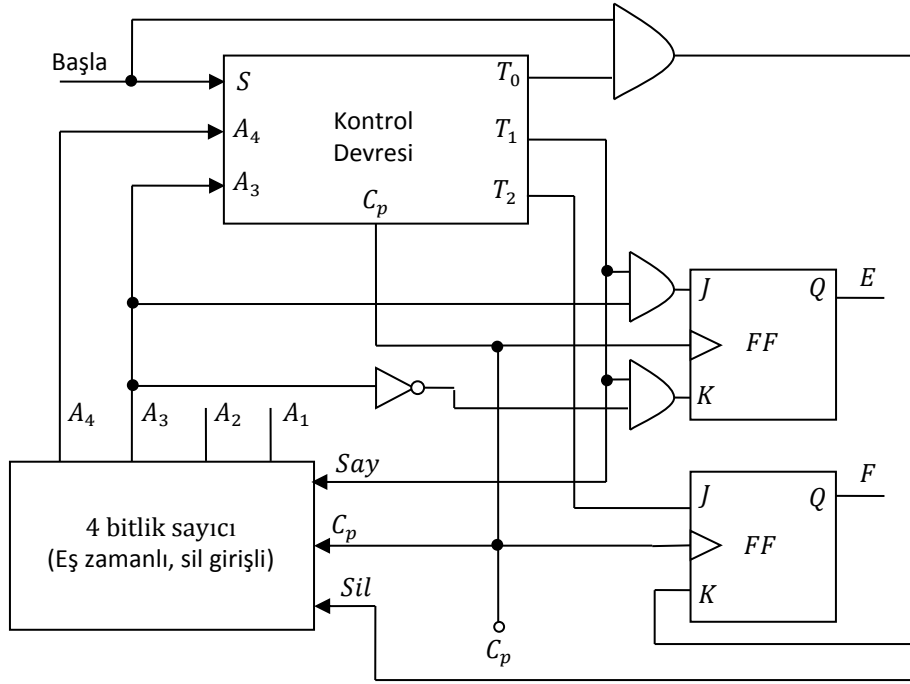
$T_1: A \leftarrow A + 1$

$T_1\overline{A_3}: E \leftarrow 0$

$T_1A_3: E \leftarrow 1$

$T_2: F \leftarrow 1$

c)

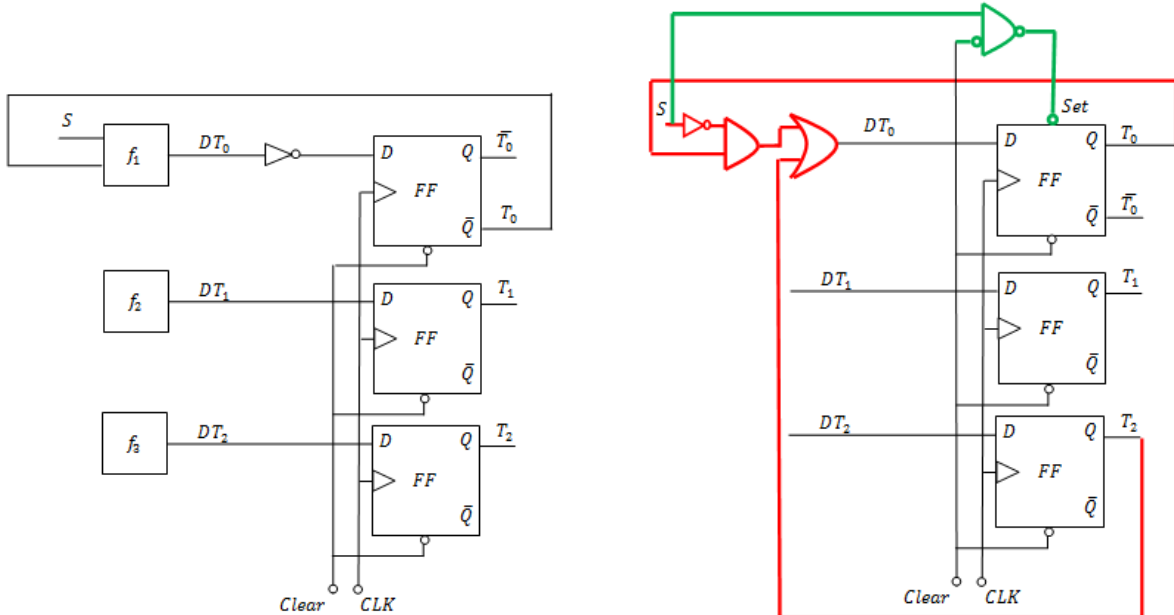


d) Durum diyagramı ve ASM şemasından hareketle

$$DT_0 = \bar{S}T_0 + T_2$$

$$DT_1 = ST_0 + \bar{A}_3T_1 + A_3\bar{A}_4T_1$$

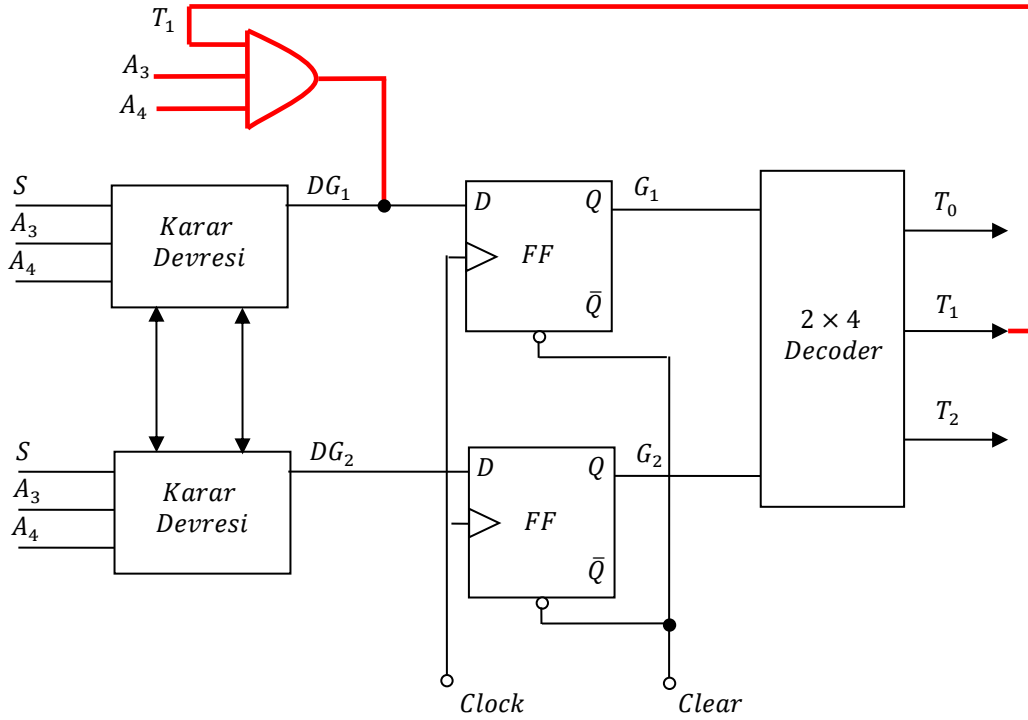
$$DT_2 = A_3A_4T_1$$



T_0 çıkışının tümleyen çıkışından alınması, T_0 için 1 sinyalini sağlar. $\bar{Q}$ tümleyenini (T_0 ' a ait D-FF) T_0 çıkışı olarak tutmak için D giriş fonksiyonuna fazladan bir inverter eklenir.

e)

Şimdiki Durum Sembolü	Şimdiki Durum $G_1 G_2$	Gelecek Durum $G_1 G_2$	Girişler $S A_3 A_4$	Giriş Değerleri	MUX-1	MUX-2	Çıkışlar $T_0 T_1 T_2$
T_0	0 0	0 0	0 X X	$\bar{S}$	0	---	1 0 0
T_0	0 0	0 1	1 X X	S	0	S	1 0 0
T_1	0 1	0 1	X 0 X	$\bar{A}_3$	---	$\bar{A}_3$	0 1 0
T_1	0 1	0 1	X 1 0	$A_3 \bar{A}_4$	---	$A_3 \bar{A}_4$	0 1 0
T_1	0 1	1 0	X 1 1	$A_3 A_4$	$A_3 A_4$	---	0 1 0
T_2	1 0	0 0	X X X	---	0	0	0 0 1

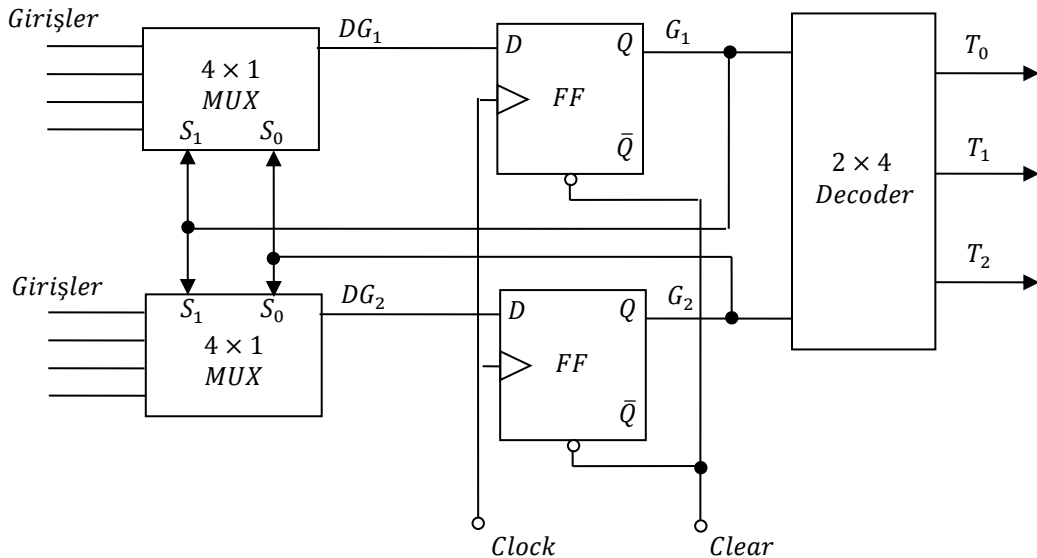


Tablodan “Şimdiki durum”, “Sonraki durum” ve “Girişler” sütunları kullanılarak aşağıdaki lojik ifadeler elde edilir.

$$DG_1 = T_1 A_3 A_4$$

$$DG_2 = T_0 S + T_1 \bar{A}_3 + T_1 A_3 \bar{A}_4$$

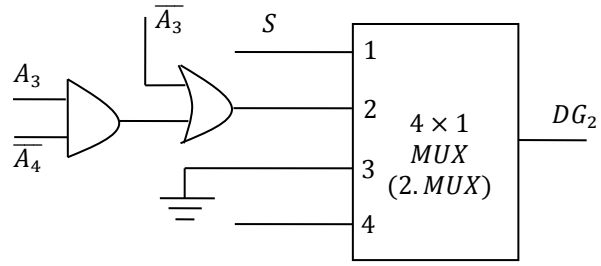
f)



$$D_{10} = 0 \quad D_{20} = S$$

$$D_{11} = A_3 A_4 \quad D_{21} = \overline{A_3} + A_3 \overline{A_4}$$

$$D_{12} = 0 \quad D_{22} = 0$$



MUX girişleri şu şekilde belirlenir:

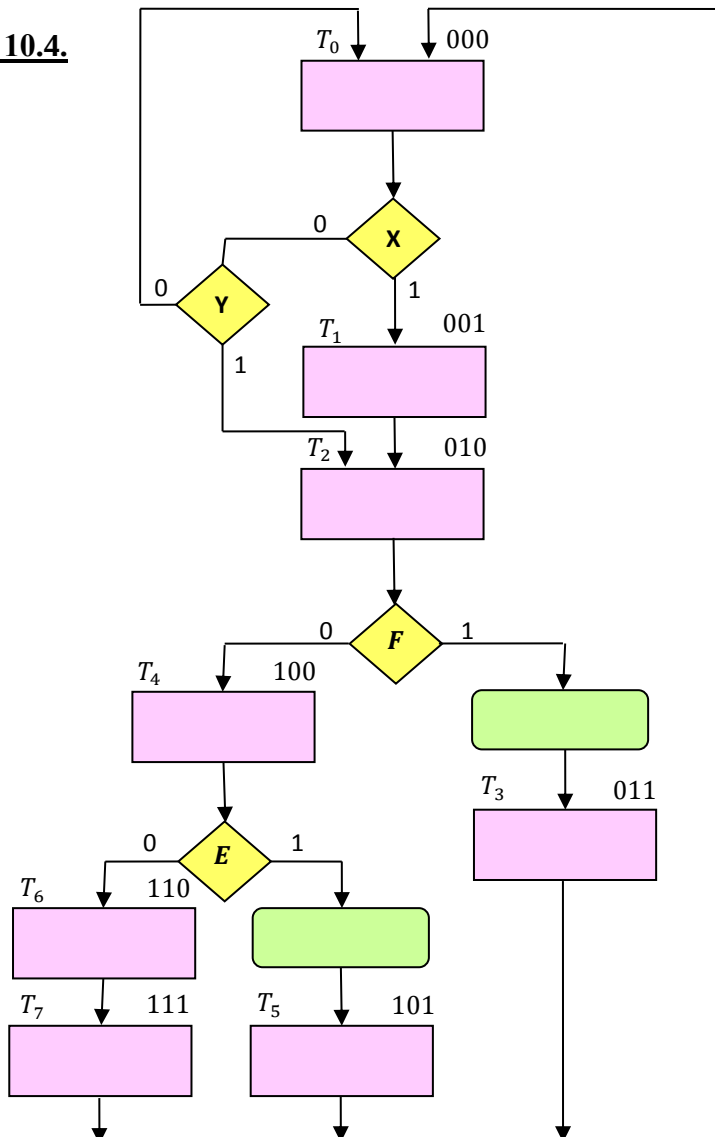
a) Herhangi bir durum süresince (örneğin T_0 'ın ilk iki durumu) gelecek durumdaki G_1 değerlerinin hepsi Lojik 0/1 ise bu G' ye ait MUX girişi Lojik 0/1 alınır. Yukarıdaki tabloda T_0 'a ait ilk iki G_1 durumunda MUX'a ait giriş değerleri "0" olduğu gibi.

b) Gelecek durumdaki G' lerin Lojik 1 olduğu duruma ait giriş değeri bu G' ye bağlı olan MUX'un girişini oluşturur. Örneğin yukarıdaki tabloda T_0 'a ait G_2 'ye ait gelecek durumda $G_2 = 1$ için MUX 2'nin girişi S olur.

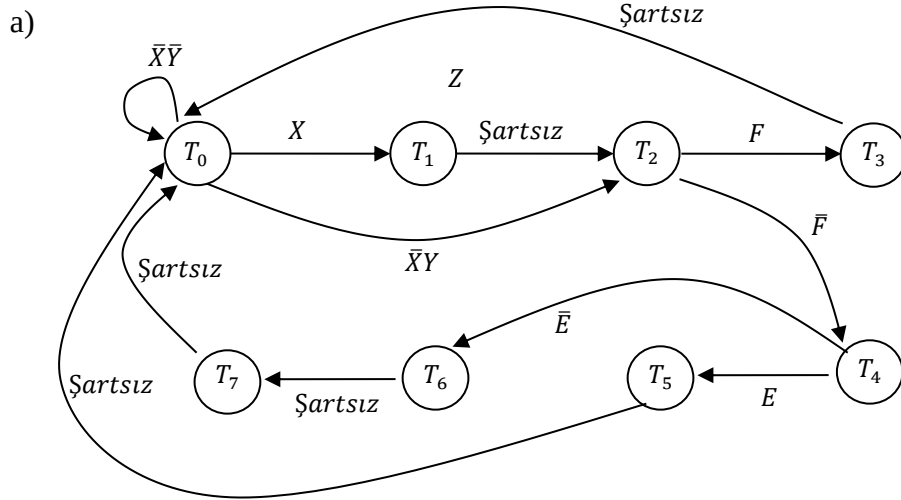
Bu durumda MUX 1'in ilk girişi Lojik 0, MUX 2'nin ilk girişi S olacaktır. T_0, T_1 ve T_2 olmak üzere 3 durum olduğuna göre 4×1 MUX kullanılır ve 4. girişleri boşta kalır (Kullanılan MUX'un giriş sayısı durum sayısına eşittir).

c) Seçilecek MUX'da MUX'un giriş sayısı problemdeki durum sayısına eşittir.

Örnek 10.4.



- Durum diyagramını çıkartarak kontrol birimini tasarlayınız.
- Her bir durum için bir FF kullanarak kontrol birimini tasarlayınız.
- FF-Decoder kullanarak tasarlayınız.
- MUX-FF kullanarak tasarlayınız.

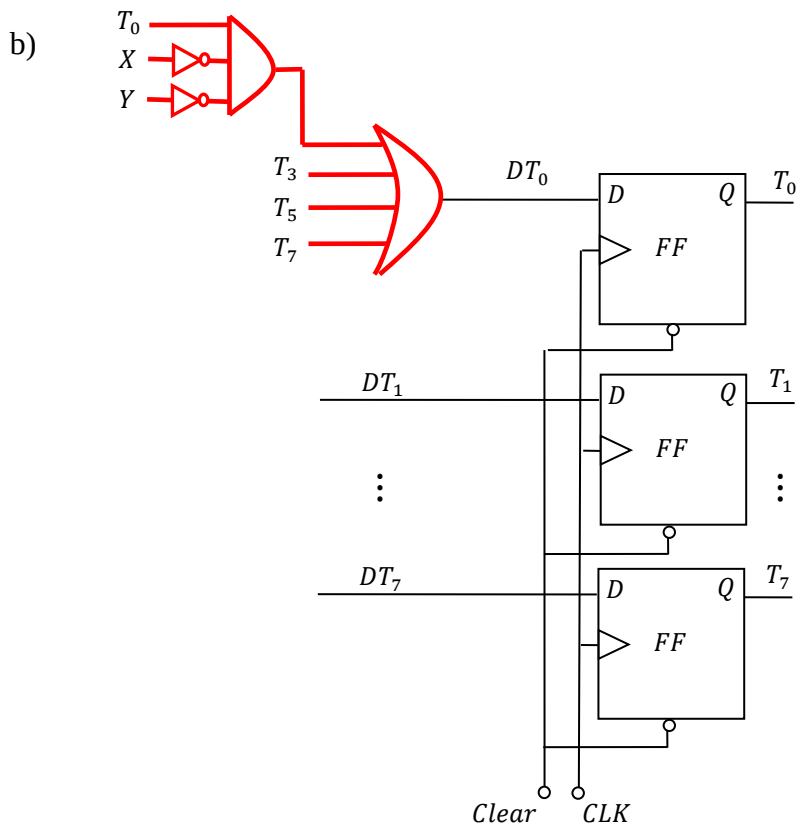


$$DT_0 = \bar{X}\bar{Y}T_0 + T_3 + T_5 + T_7,$$

$$DT_1 = XT_0, \quad DT_2 = \bar{X}YT_0 + T_1$$

$$DT_3 = FT_2, \quad DT_4 = \bar{F}T_2,$$

$$DT_5 = ET_4, \quad DT_6 = \bar{E}T_4, \quad DT_7 = T_6$$



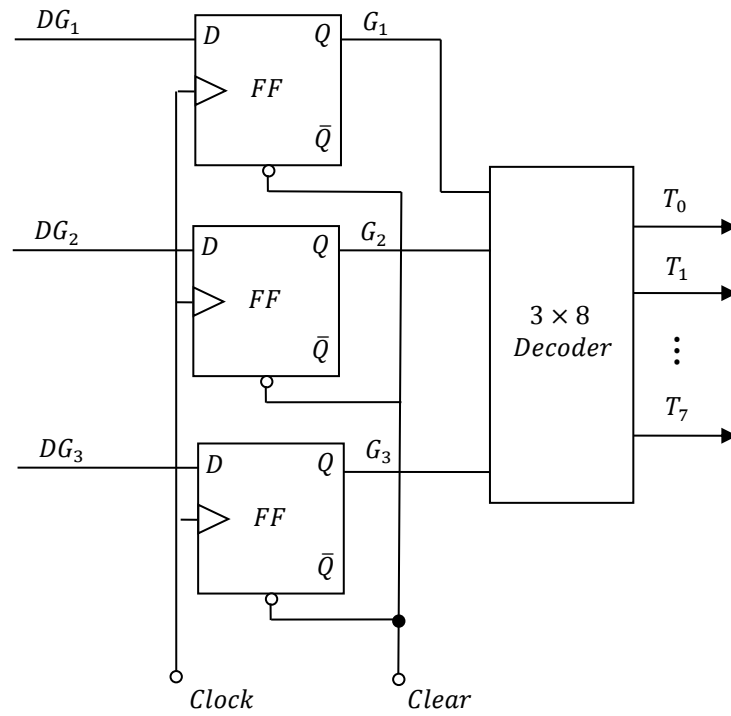
c)

Şimdiki Durum Sembolü	Şimdiki Durum $G_1G_2G_3$	Girişler $x y E F$	Gelecek Durum $G_1G_2G_3$	MUX-1	MUX-2	MUX-3	Çıkışlar $T_0 T_1 \dots T_7$
T_0	0 0 0	0 0 X X	0 0 0	0	---	---	1 0 ... 0
T_0	0 0 0	1 X X X	0 0 1	0	---	X	1 0 ... 0
T_0	0 0 0	0 1 X X	0 1 0	0	$\bar{X}Y$	---	1 0 ... 0
T_1	0 0 1	X X X X	0 1 0	0	1	0	0 1 ... 0
T_2	0 1 0	X X X 1	0 1 1	---	F	F	
T_2	0 1 0	X X X 0	1 0 0	$\bar{F}$	---	---	
T_3	0 1 1	X X X X	0 0 0	0	0	0	
T_4	1 0 0	X X 1 X	1 0 1	1	---	E	
T_4	1 0 0	X X 0 X	1 1 0	1	$\bar{E}$	---	
T_5	1 0 1	X X X X	0 0 0	0	0	0	
T_6	1 1 0	X X X X	1 1 1	1	1	1	
T_7	1 1 1	X X X X	0 0 0	0	0	0	

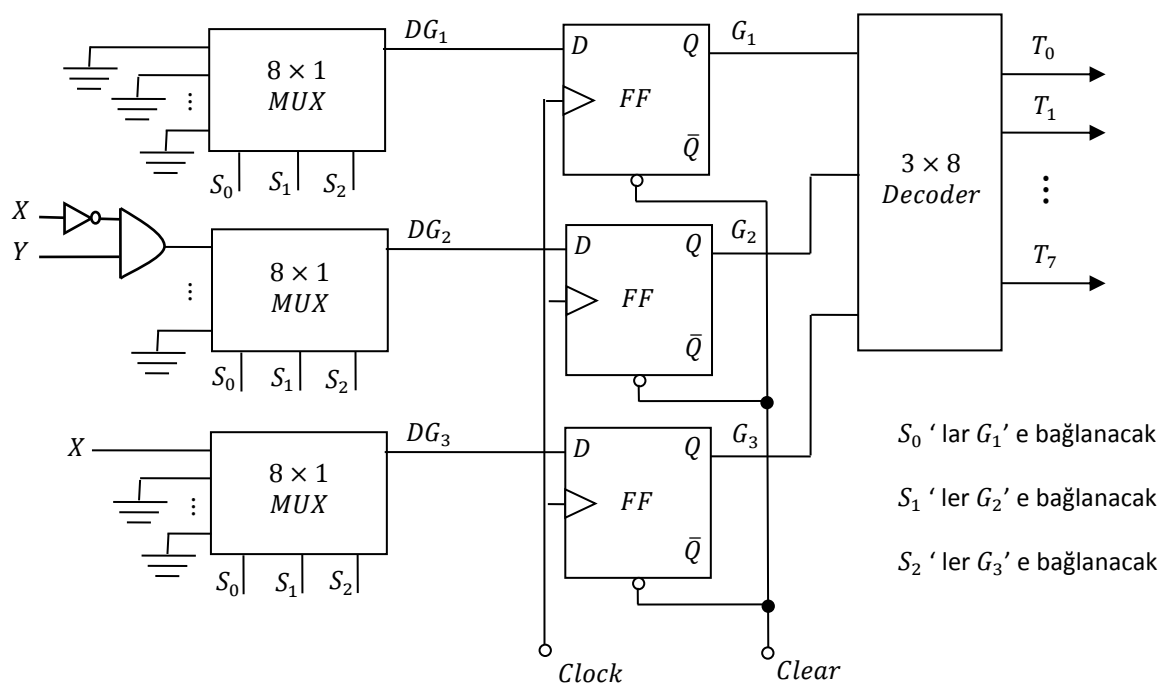
$$DG_1 = \bar{F}T_2 + T_4 + T_6$$

$$DG_2 = \bar{X}YT_0 + T_1 + FT_2 + \bar{E}T_4 + T_6$$

$$DG_3 = XT_0 + FT_2 + ET_4 + T_6$$



d)

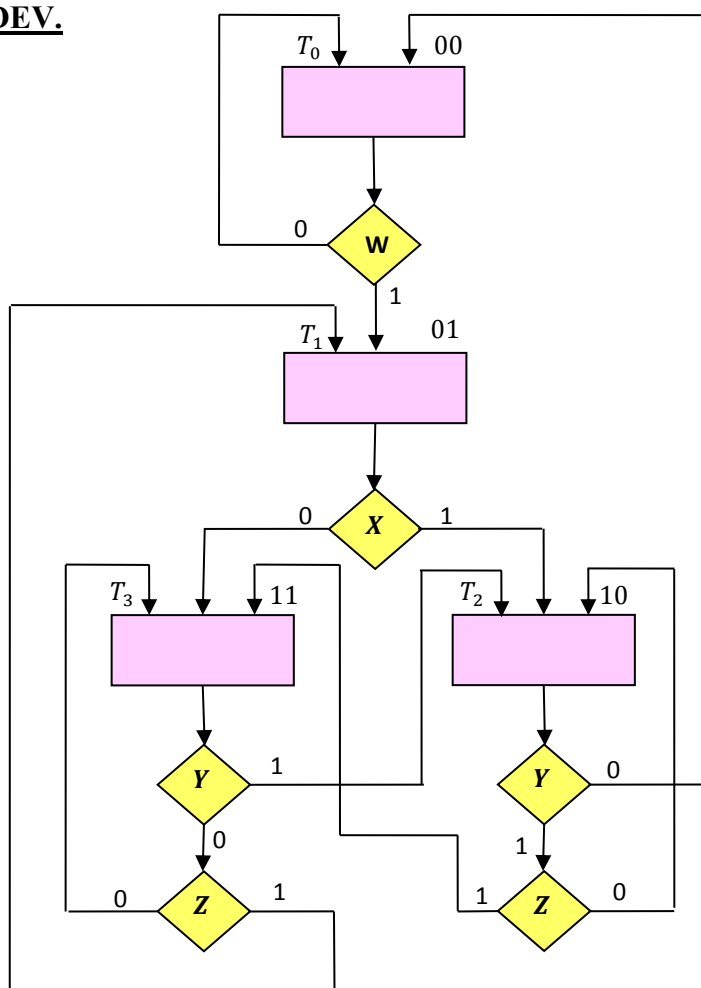


S_0 'lar G_1 ' e bağlanacak

S_1 'ler G_2 ' e bağlanacak

S_2 'ler G_3 ' e bağlanacak

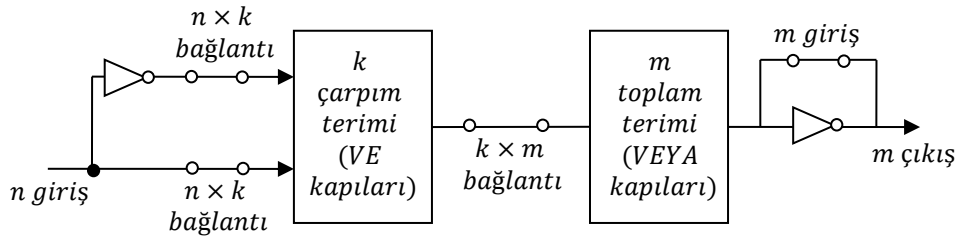
ÖDEV.



- Durum diyagramını çıkartarak kontrol birimini tasarlayınız.
- Her bir durum için bir FF kullanarak kontrol birimini tasarlayınız.
- FF-Decoder kullanarak tasarlayınız.
- MUX-FF kullanarak tasarlayınız.

11. PROGRAMLANABİLİR MANTIK DİZİSİ (PLA)

Gerçekleştirilecek lojik fonksiyonda değişken yada keyfi değer sayısının çok olduğu durumlarda “Programlanabilir Mantık Dizisi (programmable logic array-PLA)” elemanının kullanılması daha ekonomiktir. PLA’ da boole fonksiyonları çarpımların toplamı şeklinde uygulanır (yani fonksiyon minimum terimler kanonik açılımına göre yazılmalıdır). PLA’ ya ait blok gösterim Şekil 11.1’ deki gibidir.

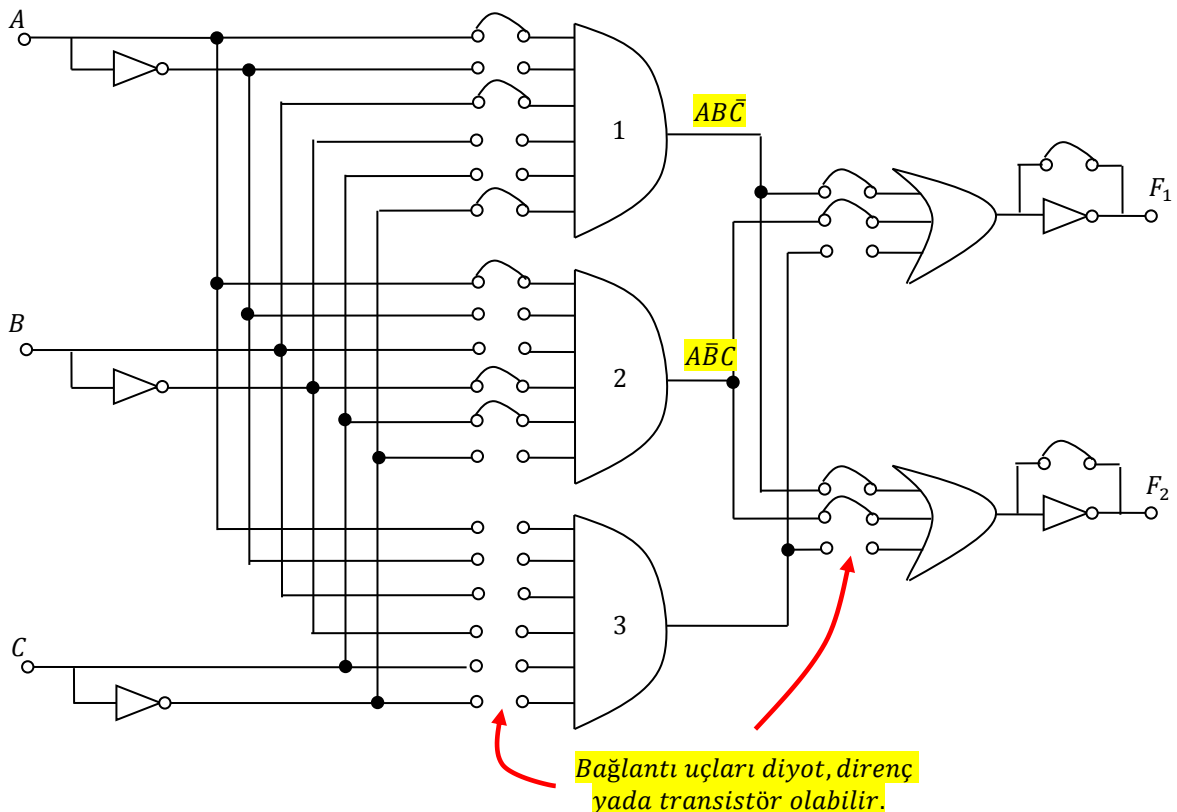


Şekil 11.1 PLA blok gösterim

PLA’ nın büyüklüğü girişlerin, çarpım terimlerinin ve çıkışların sayısı ile tanımlanır (Toplam terimlerinin sayısı çıkış sayısına eşittir). Tipik bir PLA’ da 16 giriş, 48 çarpım terimi ve 8 çıkış vardır (TTL IC tipi 82S100).

Programlı bağlantıların sayısı $(2n \times k) + (k \times m) + m$ kadardır.

Örnek 11.1. 3 girişli, 3 çarpım terimli ve 2 çıkışlı bir PLA devresini gerçekleyelim.



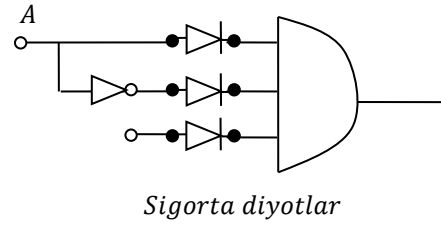
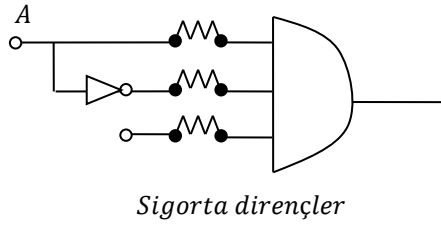
Yukarıdaki PLA' da gerçekleştirilen fonksiyon şu şekildedir:

$$F_1 = AB\bar{C} + A\bar{B}C$$

Eğer bu PLA' da $F_1 = AB + \bar{B}C$ ve $F_2 = A\bar{C} + \bar{B}\bar{C}$ fonksiyonu da gerçekleştirilirse program tablosu şu şekilde olacaktır:

Terimler		Girişler			Çıkışlar	
		A	B	C	F ₁	F ₂
AB	1	1	1	--	1	0
$\bar{B}C$	2	--	0	1	1	0
$A\bar{C}$	3	1	--	0	0	1
$\bar{B}\bar{C}$	4	--	0	0	0	1

Fonksiyonlardaki terim sayısı terimler sütununa yazılıyor. Girişler sütununda değişkenlerin aldığı değerler 1 veya 0 şeklinde gösteriliyor. Çıkışlarda da F' leri 1 yapan değerler alınıyor.



Örnek 11.2. $F_1(A, B, C) = \sum(3,5,6,7)$ $F_2(A, B, C) = \sum(0,2,4)$

PLA program tablosunu çıkarınız.

Önce fonksiyon sadeleştirilir.

BC \ A	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$F_1 = AC + BC + AB$$

BC \ A	00	01	11	10
0	1	0	0	1
1	1	0	0	0

$$F_2 = \bar{B}\bar{C} + \bar{A}\bar{C}$$

Aynı zamanda F_1 ve F_2 ' nin "Lojik 0" olduğu durumlarda Karnough diyagramına aktarılır ve kullanılacak VE (çarpım) kapılarının azaltılıp azaltılamayacağı dikkate alınır. Bunun için $\bar{F}_1$ yazılırsa ($\bar{F}_1$ bulmak için Karnaugh' da 0' lar yerine 1 yazılır ve tekrar lojik ifade çıkarılır);

$$\bar{F}_1 = \bar{A}\bar{C} + \bar{B}\bar{C} + \bar{A}\bar{B}$$

$$F_2 = \bar{B}\bar{C} + \bar{A}\bar{C}$$

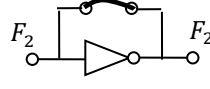
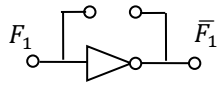
$\bar{F}_1$ ve F_2 ' nin iki teriminin aynı olduğu görülür. Böylece kullanılacak kapı sayısı 6' dan 4' e düşürülmüş oluyor. PLA çıkışında ise F_1 inversli çıkıştan alınır.

$$\overline{(\bar{F}_1)} = \overline{(\bar{A}\bar{C} + \bar{B}\bar{C} + \bar{A}\bar{B})}$$

$$F_2 = \bar{B}\bar{C} + \bar{A}\bar{C}$$

Buna göre PLA' nın program tablosu şu şekilde olacaktır:

Terimler		Girişler			Çıkışlar	
		A	B	C	F ₁	F ₂
$\overline{B}\overline{C}$	1	--	0	0	1	1
$\overline{A}\overline{C}$	2	0	--	0	1	1
$\overline{A}\overline{B}$	3	0	0	--	1	0

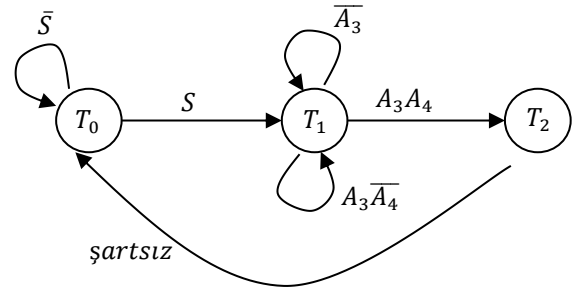


T D

T: F₁'in tersi olduğunu göstermektedir

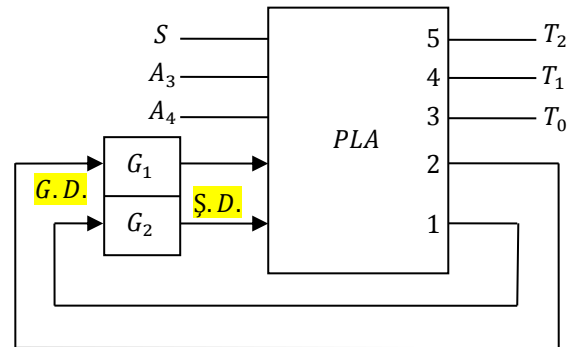
11.1. Örnek Bir Problemin PLA ile Çözümü

Yandaki şekilde durum diyagramı verilen devreyi PLA kullanarak tasarlayınız.



Durum tablosu aşağıdaki gibidir:

Şimdiki Durum Sembolü	Şimdiki Durum G ₁ G ₂	Gelecek Durum G ₁ G ₂	Girişler S A ₃ A ₄	Çıkışlar T ₀ T ₁ T ₂
T ₀	0 0	0 0	0 X X	1 0 0
T ₀	0 0	0 1	1 X X	1 0 0
T ₁	0 1	0 1	X 0 X	0 1 0
T ₁	0 1	0 1	X 1 0	0 1 0
T ₁	0 1	1 0	X 1 1	0 1 0
T ₂	1 0	0 0	X X X	0 0 1



PLA program tablosu şu şekildedir:

Şimdiki Durum Sembolü	Çarpım Terimi	Şimdiki Durum G ₁ G ₂	Girişler S A ₃ A ₄	Gelecek Durum G ₁ G ₂	Ş. D.' nin Çıkışlar T ₀ T ₁ T ₂	Açıklama
T ₀	1	0 0	0 -- --	0 0	1 -- --	S = 0' da T ₀ = 1
T ₀	2	0 0	1 -- --	0 1	1 -- --	S = 1' de T ₀ = T ₁
T ₁	3	0 1	-- 0 --	0 1	-- 1 --	A ₃ = 0' da T ₁ = T ₁
T ₁	4	0 1	-- 1 0	0 1	-- 1 --	A ₃ A ₄ = 10' da T ₁ = T ₁
T ₁	5	0 1	-- 1 1	1 0	-- 1 --	A ₃ A ₄ = 11' de T ₁ = T ₂
T ₂	6	1 0	-- -- --	0 0	-- -- 1	Şartsız T ₂ = T ₀ T ₀ = 1

$$G_1^+ = T_1 A_3 A_4 \Rightarrow G_1^+ = \overline{G_1} G_2 A_3 A_4$$

$$G_2^+ = T_0 S + T_1 \overline{A_3} + T_1 A_3 \overline{A_4} \Rightarrow G_2^+ = \overline{G_1} G_2 S + \overline{G_1} G_2 \overline{A_3} + \overline{G_1} G_2 A_3 \overline{A_4}$$

Önce girişler yazılır. Buna göre gelecek durumda bulunacak $G_1 G_2$ yazılır. Buna ait T_0, T_1, T_2 çıkışlarının 1 olduğu değerlerini yazıyoruz. Keyfi terimler çok fazla olduğundan Karnough diyagramına gerek yoktur. T_0 'ın 1 olduğu durumlar için 0 yazılır.

$$T_0 = \overline{G_1} \overline{G_2} \overline{S} + \overline{G_1} \overline{G_2} S$$

$$T_1 = \overline{G_1} G_2 \overline{A_3} + \overline{G_1} G_2 A_3 \overline{A_4} + \overline{G_1} G_2 A_3 A_4$$

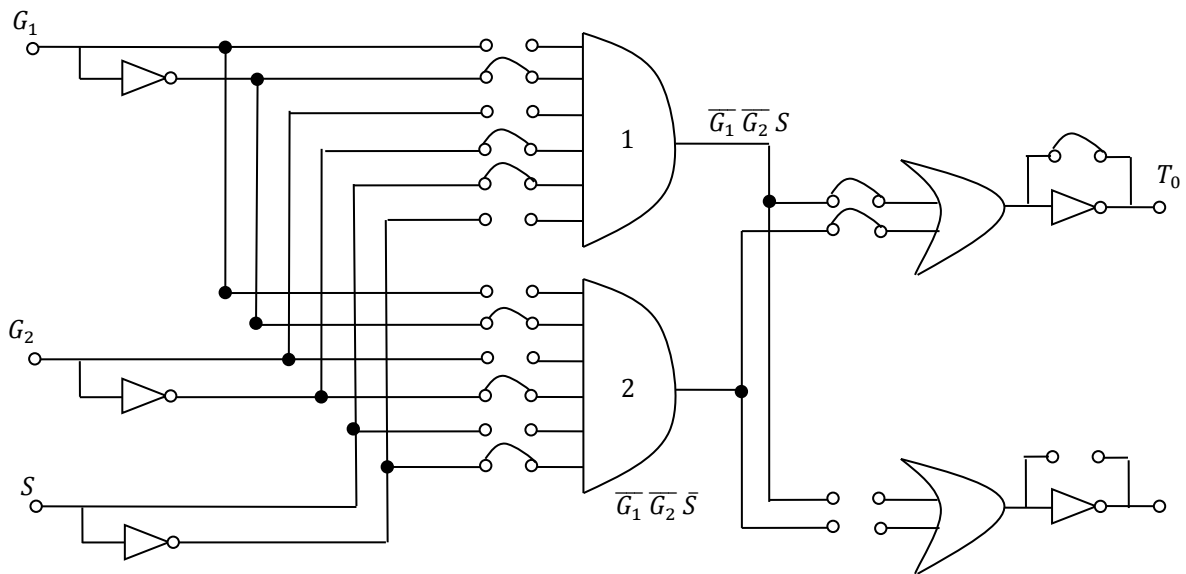
$$T_2 = G_1 \overline{G_2}$$

5 girişli PLA kullanılabilir (G_1, G_2, S, A_3, A_4)

Yada T_0 ve T_1 için ayrı ayrı PLA kullanılabilir. Eğer ayrı ayrı PLA kullanılacak olursa T_0 için 3 girişli 1 çıkışlı PLA kullanılabilir.

Eğer 5 girişli 3 çıkışlı bir PLA bulunabilirse bir PLA yeterli olur.

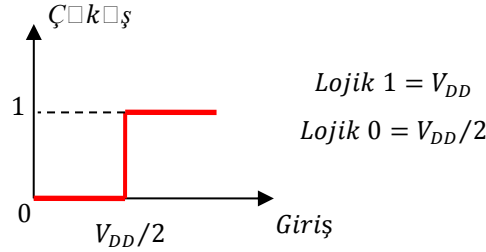
3 girişli 1 çıkışlı PLA için T_0 'ın gerçekleştirilmesi aşağıdaki gibidir:



Diğerleri de benzer şekilde tasarlanabilir. Bu örnekteki devre pratikte PLA ile uygulanamayacak kadar küçüktür. Burada sadece örnekleme amacıyla verilmiştir. Ticari piyasada mevcut tipik bir PLA' da 10' dan fazla giriş ve 50 kadar çarpım terimi olacaktır. Bu kadar çok değişkenli bir uygulama için bilgisayar destekli bir sadeleştirme programına ihtiyaç vardır.

12. LOJİK KAPILARDA FAN-OUT HESABI

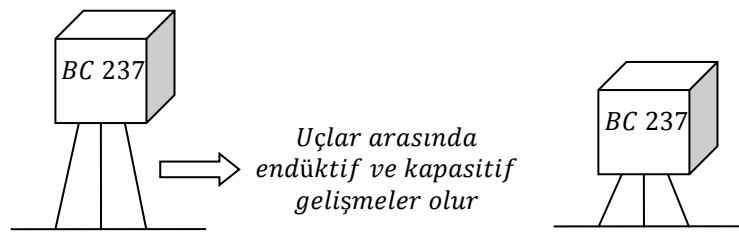
Bir lojik kapının çıkışına bağlanabilecek aynı türden maksimum lojik kapının sayısına fan-out değeri denir. İdeal bir lojik elemanın karakteristiği aşağıdaki şekilde olmalıdır.



Şekil 12.1. İdeal bir lojik kapının karakteristiği

12.1. Gürültü ve Gürültü Sınırı (Noisy Margin)

Sayısal devrelerde “Lojik 1” veya “Lojik 0” değerine etki eden sinyallere gürültü denir. Bu gürültü lojik değerleri değiştirilebilir. Kritik gürültü değerlerine gürültü sınırı (noisy margin) denir. Gürültüler direnç, yarı iletken elemanların gürültüsü ve dış etmenlerden kaynaklanan gürültüler şeklindedir. Direnç ve yarı iletken gürültüleri elemanın içindeki elektron hareketinden kaynaklanır ve buna “beyaz gürültü” denir. Dışarıdan hiçbir etki olmasa da bu gürültü oluşur. Aynı zamanda montajı yapılan elemanların bacak bağlantıları kart üzerinden çok yüksekte ise uçlar arası kapasitif yada endüktif etki oluşacağından gürültüye neden olur.



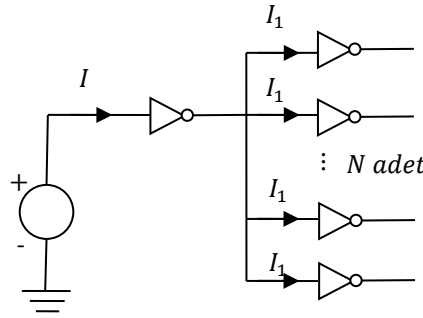
Şekil 12.2. Malzeme montajından gürültü oluşumu

12.2. Fan-Out Hesabı

Şekil 12.3' de görüldüğü gibi bir kapının çıkışına yine aynı türden kapılar bağlansın. Bu durumda çıkışa bağlanan kapılar eşit miktarda (I_1) akım çeker. Bu devrede

$$I = N \cdot I_1$$

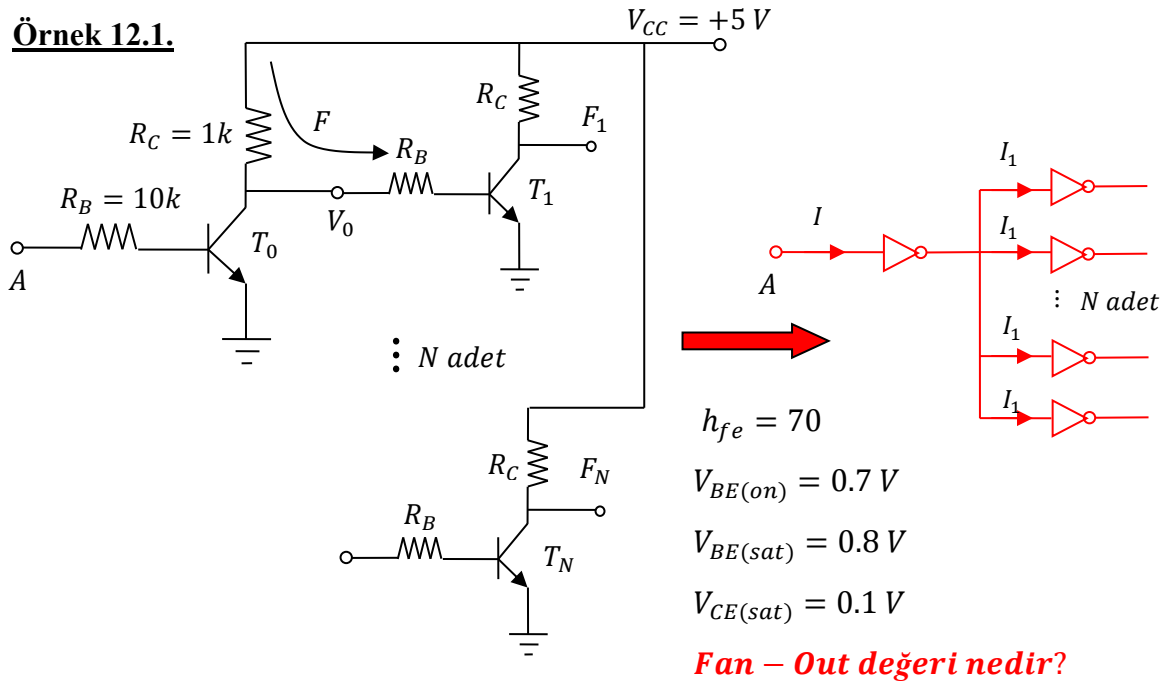
olacaktır. Örneğin hesaplanan maksimum kapı sayısı yani fan-out değeri 300 ise bu değer aynı tür kapılar için hesaplanan değerdir. Farklı tür kapı bağlanırsa bu değer daha da düşecektir.



Şekil 12.3. Bir kapı çıkışına aynı türde kapıların bağlanması

12.2.1. RTL kapılarında fan-out hesabı

Örnek 12.1.



T_0 saturasyonda ise I_C toprağa akar, T_0 kesimde ise akım F kolunu takip eder.

Devrenin analizi yapılacak olursa;

1. A girişine "lojik 0" verilirse T_0 transistörü iletime geçmeyeceği için bir I_C akımı akmayacaktır. Dolayısıyla V_0 çıkışında "lojik 1" değeri görülecektir ve T_1 iletime geçer.

2. A girişine “lojik 1” verilirse T_0 transistörü iletime geçer (T_0 kesime gider) ve bir I_C akımı akar. Dolayısıyla V_0 çıkışında $V_0 = V_{CC} - I_C R_C$ değeri yani “lojik 0” görülecektir.

Birinci yol, NM_H kullanarak fan-out hesabıdır.

$$I_{C(sat)} = \frac{V_{CC} - V_{CE(sat)}}{R_C} \quad I_{C(sat)} = h_{fe} \cdot I_{B(sat)}$$

Girişteki gerilimin maksimum değerini (V_{IH}) alması durumunda T_0 transistörü iletime geçer ve giriş gerilimi aşağıdaki değerini alır.

$$V_{IH} = I_{B(sat)} \cdot R_B + V_{BE(sat)} = \frac{V_{CC} - V_{CE(sat)}}{R_C} \cdot \frac{R_B}{h_{fe}} + V_{BE(sat)} \Rightarrow V_{IH} = 1.5 \text{ V}$$

Çıkışta oluşabilecek maksimum gerilim değeri (V_{OH}) T_0 transistörünün kesimde olduğu zaman oluşur. Bu durumda T_1 transistörü iletime geçer ve akım F yolundan akar.

$$V_{OH} = I \cdot R_B + V_{BE(sat)} = \frac{V_{CC} - V_{BE(sat)}}{R_C + R_B} \cdot R_B + V_{BE(sat)} \Rightarrow V_{OH} = 4.6 \text{ V}$$

$NM_H = V_{OH} - V_{IH} = 4.6 - 1.5 = 3.1 \text{ V}$ olarak bulunur.

$NM_H = 3.1 \text{ V}$ değeri 1 kapı sürülürse bulunan değerdir ve bu devre gürültüden oldukça az etkilenir. Ancak soruda maksimum bağlanabilecek kapı sayısı istenmektedir. Maksimum kapı sayısı $NM_H = 0$ değeri için (En kötü hal için) hesaplanır. Bu durumda

$NM_H = V_{OH} - V_{IH} = 0 \Rightarrow V_{OH} = V_{IH}$ olacaktır. Bu durumda çıkışa N adet kapı bağlı olduğu göz önüne alınarak eşitlik yeniden yazılırsa;

$$V_{OH} = V_{IH}$$

$$\frac{V_{CC} - V_{BE(sat)}}{R_C + \frac{R_B}{N}} \cdot \frac{R_B}{N} + V_{BE(sat)} = \frac{V_{CC} - V_{CE(sat)}}{R_C} \cdot \frac{R_B}{h_{fe}} + V_{BE(sat)}$$

$$h_{fe} \cdot \frac{V_{CC} - V_{BE(sat)}}{NR_C(1 + \frac{R_B}{NR_C})} = \frac{V_{CC} - V_{CE(sat)}}{R_C}$$

$$h_{fe} \cdot \frac{V_{CC} - V_{BE(sat)}}{V_{CC} - V_{CE(sat)}} = N + \frac{R_B}{R_C} \Rightarrow N = h_{fe} \cdot \frac{V_{CC} - V_{BE(sat)}}{V_{CC} - V_{CE(sat)}} - \frac{R_B}{R_C}$$

Bu formül kullanarak N değeri hesaplanırsa

$$N = 70 \cdot \frac{5-0.8}{5-0.1} - \frac{10k}{1k} \Rightarrow N = 50 \text{ kapı olarak bulunur.}$$

İkinci yol, akımları kullanarak fan-out hesabıdır.

Süren kapının çıkışına bağlanan kapının yani T_1 transistörünün çekeceği akım I_{B1} kadardır.

Çıkışa bağlanacak kapıların çekeceği akım göz önüne alınarak süren kapı çıkışındaki akımın

$$I_0 = I_{B1} + I_{B2} + I_{B3} + \dots + I_{BN}$$

olması beklenir. Eğer çıkışa bağlanan kapılar aynı türden ise bu durumda $I_{B1} = I_{B2} = \dots = I_{BN}$ olacağından yukarıdaki akım ifadesi

$$I_0 = N \cdot I_{B1}$$

olacaktır. Buna göre önce I_{B1} akımı bulunmalıdır. Bunun için T_1 transistörünün iletim durumunda olacağı göz önüne alınarak I_{C1} akımı hesaplanırsa;

$$I_{C1} = \frac{V_{CC} - V_{CE(sat)}}{R_C} \Rightarrow I_{C1} = \frac{5 - 0.1}{1k} \Rightarrow I_{C1} = 4.9 \text{ mA}$$

$$I_{B1} = I_{C1}/h_{fe} \Rightarrow I_{B1} = 4.9 \text{ mA}/70 \Rightarrow I_{B1} = 70 \mu A$$

$$I_0 = \frac{V_{CC} - V_0}{R_B}, \quad V_0 = R_{B1} \cdot I_{B1} + V_{BE(sat)} = 10k \cdot 70 \mu A + 0.8 = 1.5 \text{ V}$$

$$I_0 = \frac{V_{CC} - V_0}{R_B} = \frac{5 - 1.5}{10k} \Rightarrow I_0 = 3.5 \text{ mA}$$

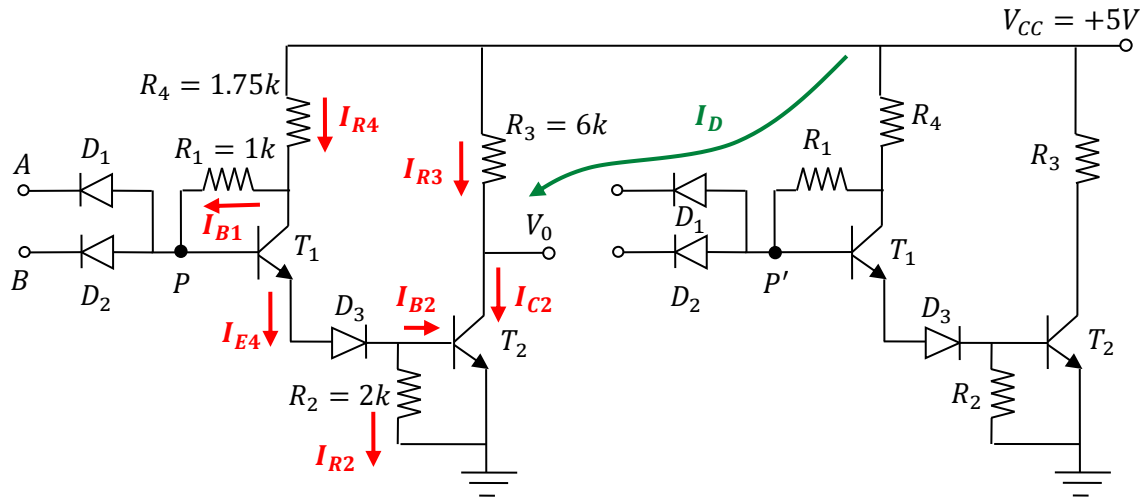
$$N = \frac{I_0}{I_{B1}} = \frac{3.5 \text{ mA}}{70 \mu A} \Rightarrow N = 50 \text{ kapı}$$

bulunur.

NOT: N değeri 50.2 yada 50.7 çıkabilirdi. Bu durumda da yine sonucun tamsayı kısmı maksimum kapı sayısı olarak alınır yani fan-out değeri yine 50 olurdu.

12.2.2. DTL kapılarında fan-out hesabı

Örnek 12.2. Aşağıda verilen devrenin analizini yaparak fan-out değerini hesaplayınız.



$$h_{fe} = 20, V_{BE(off)} = 0.6 V, V_{BE(sat)} = 0.8 V, V_{CE(sat)} = 0.2 V, V_{D(on)} = 0.7 V, V_{D(off)} = 0.6 V$$

A ve B girişlerine Lojik 0 verilirse $\Rightarrow D_1$ ve D_2 ilettime geçer
 T_1, D_3 ve T_2 kesime gider $\Rightarrow$ Bu durumda $V_0 = 1$ olur.

A ve B girişlerine Lojik 1 verilirse $\Rightarrow D_1$ ve D_2 kesime gider
 T_1, D_3 ve T_2 ilettime geçer $\Rightarrow$ Bu durumda $V_0 = 0$ olur.

V_0 çıkışına bağlanan kapının sürülebilmesi için $V_0 = 0$ olması gerekmektedir. Yani T_2 transistörünün iletimde olması gereklidir. Bu durumda T_2 transistörünün kolektör kolundan I_{C2} akımı akacaktır. Çıkışa N adet kapı bağlandığında, her bir kapı I_D akımını çekerse

$$I_{C2} = I_{R3} + N \cdot I_D$$

ifadesi elde edilir.

I_{C2} akımını bulmak için devre üzerinde işaretlenen akımların sırasıyla bulunması gerekmektedir.

$$V_{CC} - V_{R4} - V_{R1} - V_p = 0 \Rightarrow V_{CC} - V_p = V_{R4} + V_{R1}$$

$$V_p = V_{BE(sat)} + V_{D(on)} + V_{BE(sat)} \Rightarrow V_p = 0.8 + 0.7 + 0.8 = 2.3 V$$

$$V_{CC} - V_p = R_4 \cdot I_{R4} + R_1 \cdot I_{B1} \quad \text{burada } I_{R4} = I_{B1} + I_{C1} \text{ ifadesi yerine yazılırsa}$$

$$V_{CC} - V_p = R_4(I_{B1} + I_{C1}) + R_1 \cdot I_{B1} \text{ burada } I_{C1} = h_{fe} \cdot I_{B1} \text{ 'dir.}$$

$$5 - 2.3 = 1.75k(I_{B1} + 20 \cdot I_{B1}) + 1k \cdot I_{B1} \Rightarrow I_{B1} = 0.07 \text{ mA}$$

$$I_{C1} = 20 \times 0.07 \Rightarrow I_{C1} = 1.4 \text{ mA} \quad I_{E1} = I_{B1} + I_{C1} \Rightarrow I_{E1} = 1.47 \text{ mA}$$

$$I_{E1} = I_{B2} + I_{R2} \Rightarrow 1.47 \text{ mA} = I_{B2} + \frac{V_{BE(sat)}}{R_2} \Rightarrow I_{B2} = 1.07 \text{ mA}$$

$$I_{C2} = h_{fe} \cdot I_{B2} = 20 \times 1.07 \Rightarrow I_{C2} = 21.4 \text{ mA} \text{ bulunur.}$$

$$V_0 = V_{CE(sat)} = 0.2 \text{ V} \text{ dur. Buna göre } I_{R3} = \frac{V_{CC} - V_{CE(sat)}}{R_3} = \frac{5 - 0.2}{6k} = 0.8 \text{ mA} \text{ bulunur.}$$

$$I_D = \frac{V_{CC} - (V_{CE(sat)} + V_{D(on)})}{R_1 + R_4} = \frac{5 - (0.2 + 0.7)}{1k + 1.75k} \Rightarrow I_D = 1.09 \text{ mA} \text{ olarak bulunur.}$$

Bulunan bu değerlere göre ilk formül kullanılarak çıkış hesaplanabilir.

$$I_{C2} = I_{R3} + N \cdot I_D \Rightarrow N = \frac{I_{C2} - I_{R3}}{I_D} \Rightarrow N = \frac{21.4 - 0.8}{1.09} = 18.89 \Rightarrow N = 18 \text{ kapı} \quad \text{olarak bulunur.}$$

Ayrıca girişte oluşacak maksimum gerilim değeri V_{IH} şu şekilde hesaplanır. V_{IH} değeri oluştuğunda T_1 , D_3 ve T_2 iletime geçer. Buna göre

$$V_{IH} = -V_{D(off)} + V_{BE(sat)} + V_{D(on)} + V_{BE(sat)} = -0.6 + 0.8 + 0.7 + 0.8 = 1.7 \text{ V}$$

olarak bulunur.

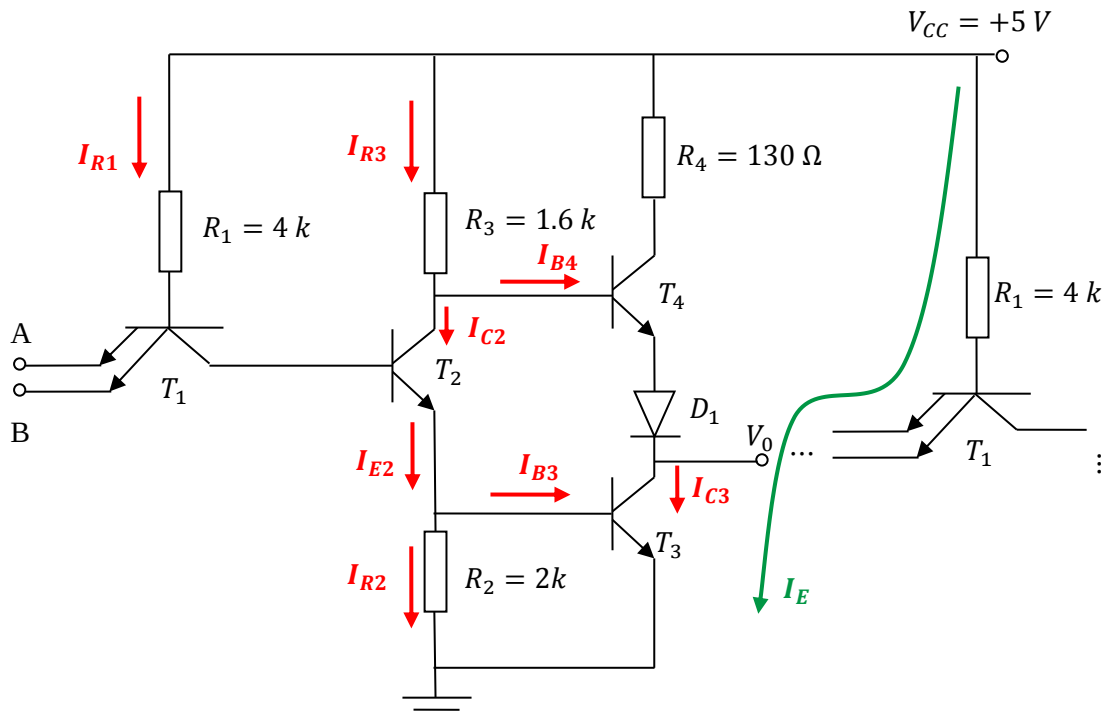
Ayrıca girişte oluşacak minimum gerilim değeri V_{IL} şu şekilde hesaplanır. V_{IL} değeri oluştuğunda T_1 , D_3 ve T_2 kesime gider. Buna göre

$$V_{IL} = -V_{D(on)} + V_{BE(off)} + V_{D(off)} + V_{BE(off)} = -0.7 + 0.6 + 0.6 + 0.6 = 1.1 \text{ V}$$

olarak bulunur.

12.2.3. TTL kapılarında fan-out hesabı

Örnek 12.3. Aşağıda verilen devrenin analizini yaparak fan-out değerini hesaplayınız.



$$h_{fe} = 20, V_{BC(sat)} = 0.5 V, V_{BE(sat)} = 0.7 V, V_{CE(sat)} = 0.2 V, V_{D(on)} = 0.7 V, V_{D(off)} = 0.6 V$$

A ve B girişlerine Lojik 0 verilirse $\Rightarrow T_1$ ilettime geçer

$$\begin{array}{ll} T_2 \text{ ve } T_3 \text{ kesime gider} & \Rightarrow \text{Bu durumda} \\ T_4 \text{ ve } D_1 \text{ ilettime geçer} & V_0 = 1 \text{ olur.} \end{array}$$

A ve B girişlerine Lojik 1 verilirse $\Rightarrow T_1$ kesime gider

$$\begin{array}{ll} T_2 \text{ ve } T_3 \text{ ilettime geçer} & \Rightarrow \text{Bu durumda} \\ T_4 \text{ ve } D_1 \text{ kesime gider} & V_0 = 0 \text{ olur.} \end{array}$$

V_0 çıkışına bağlanan kapının sürülebilmesi için $V_0 = 0$ olması gerekmektedir. Yani T_3 transistörünün iletimde olması gereklidir. Bu durumda T_3 transistörünün kolektör kolundan I_{C3} akımı akacaktır. Çıkışa N adet kapı bağlandığında, her bir kapı I_E akımını çekerse

$$I_{C3} = N \cdot I_E \Rightarrow N = \frac{I_{C3}}{I_E}$$

formülü ile fan-out sayısı bulunur.

$$V_{CC} - V_{B1} - I_{B1} \cdot R_1 = 0 \quad V_{B1} = V_{BE(sat)} + V_{BC(sat)} + V_{BE(sat)} = 0.7 + 0.5 + 0.7 = 1.9 V$$

$$V_{CC} - V_{B1} = I_{B1} \cdot R_1 \Rightarrow I_{B1} = \frac{V_{CC} - V_{B1}}{R_1} = \frac{5 - 1.9}{4k} \Rightarrow I_{B1} = 0.78 mA$$

$$I_{R3} = \frac{V_{CC} - (V_{CE(sat)} + V_{BE(sat)})}{R_3} = \frac{5 - (0.2 + 0.7)}{1.6 k} \Rightarrow I_{R3} = 2.56 mA$$

$$I_{R3} = I_{C2} + I_{B4} \quad T_4 \text{ transistörü kesimde olduğu için } I_{B4} = 0 \text{ olacaktır ve } I_{R3} = I_{C2} \text{ olur.}$$

$$I_{C2} = 2.56 mA' \text{ dir. Buna göre } I_{B2} = I_{C2}/h_{fe} \Rightarrow I_{B2} = 2.56/20 = 0.128 mA' \text{ dir.}$$

$$I_{E2} = I_{C2} + I_{B2} \Rightarrow I_{E2} = 2.56 + 0.128 \Rightarrow I_{E2} = 2.69 mA$$

$$I_{B3} = I_{E2} - I_{R2} = 2.69 mA - \frac{V_{BE(sat)}}{R_2} = 2.69 mA - \frac{0.7}{2k} \Rightarrow I_{B3} = 2.34 mA$$

$$I_{C3} = I_{B3} \cdot h_{fe} = 2.34 mA \cdot 20 \Rightarrow I_{C3} = 46.8 mA \text{ olarak bulunur.}$$

$$I_E = \frac{V_{CC} - (V_{CE(sat)} + V_{BE(sat)})}{R_1} = \frac{5 - (0.2 + 0.7)}{4 k} \Rightarrow I_D = 1.025 mA \text{ olarak bulunur.}$$

$$N = \frac{I_{C3}}{I_E} \Rightarrow N = \frac{46.8 mA}{1.025 mA} \Rightarrow N = 45.6 \Rightarrow N = 45 \text{ kapı olarak bulunur.}$$